SHANGHAI JIAO TONG UNIVERSITY

APEX 数据和知识管理实验室
DATA & KNOWLEDGE MANAGEMENT LAB

UCL

Empowering Recommender Systems with Sequences

Jiarui Jin

Ph.D. Student, Shanghai Jiao Tong University

https://jinjiarui.github.io/

alimama

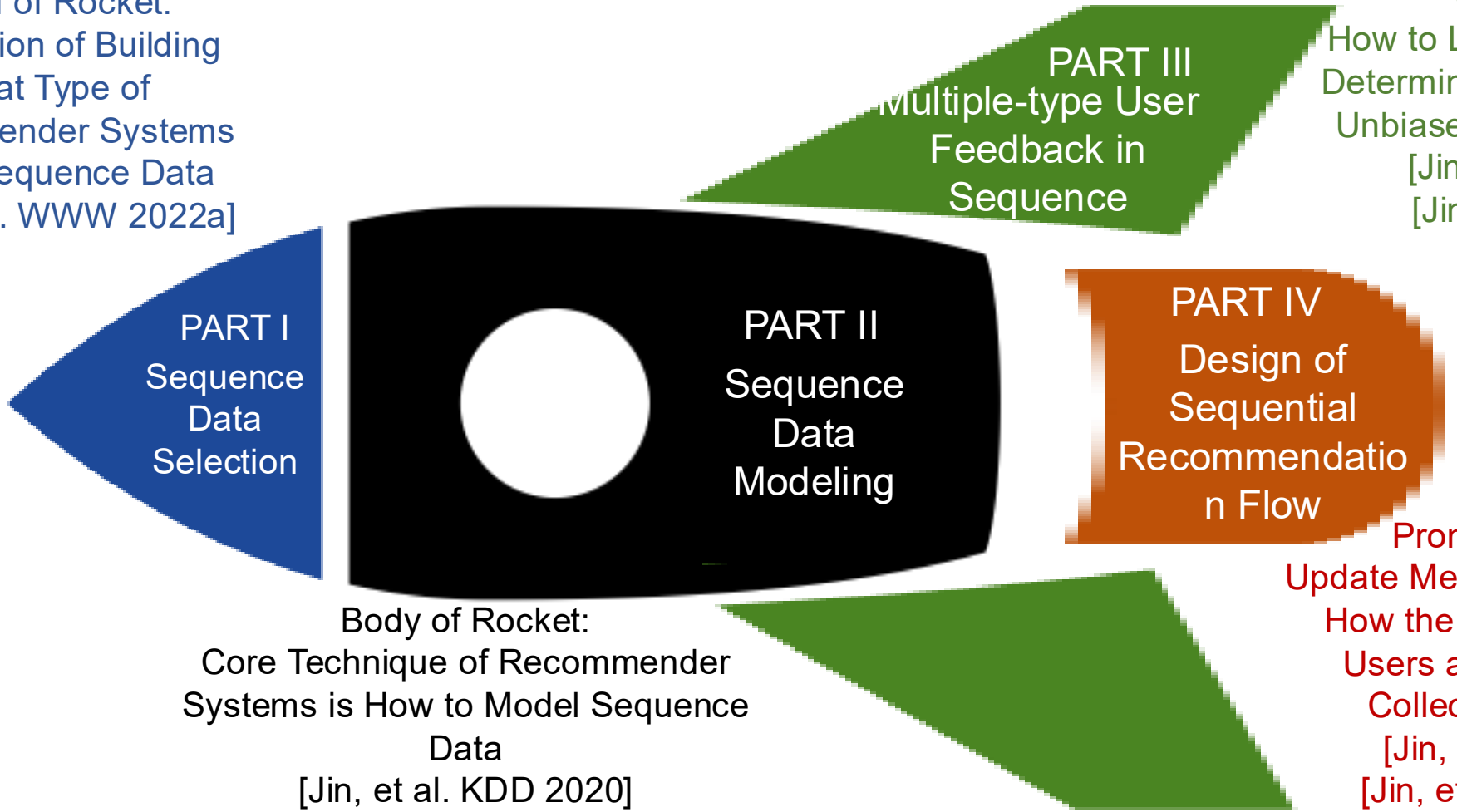
aws

招商银行 CHINA MERCHANTS BANK

淘宝直播 TAOBAO LIVE

Content

Head of Rocket:
Orientation of Building What Type of Recommender Systems using Sequence Data
[Jin, et al. WWW 2022a]



Wings of Rocket:
How to Leverage User Feedback Determine Whether the System is Unbiased and Aligned with Goal
[Jin, et al. SIGIR 2020]
[Jin, et al. CIKM 2022]

Body of Rocket:
Core Technique of Recommender Systems is How to Model Sequence Data
[Jin, et al. KDD 2020]
[Jin, et al. TOIS 2022]
[Jin, et al. WWW 2022b]

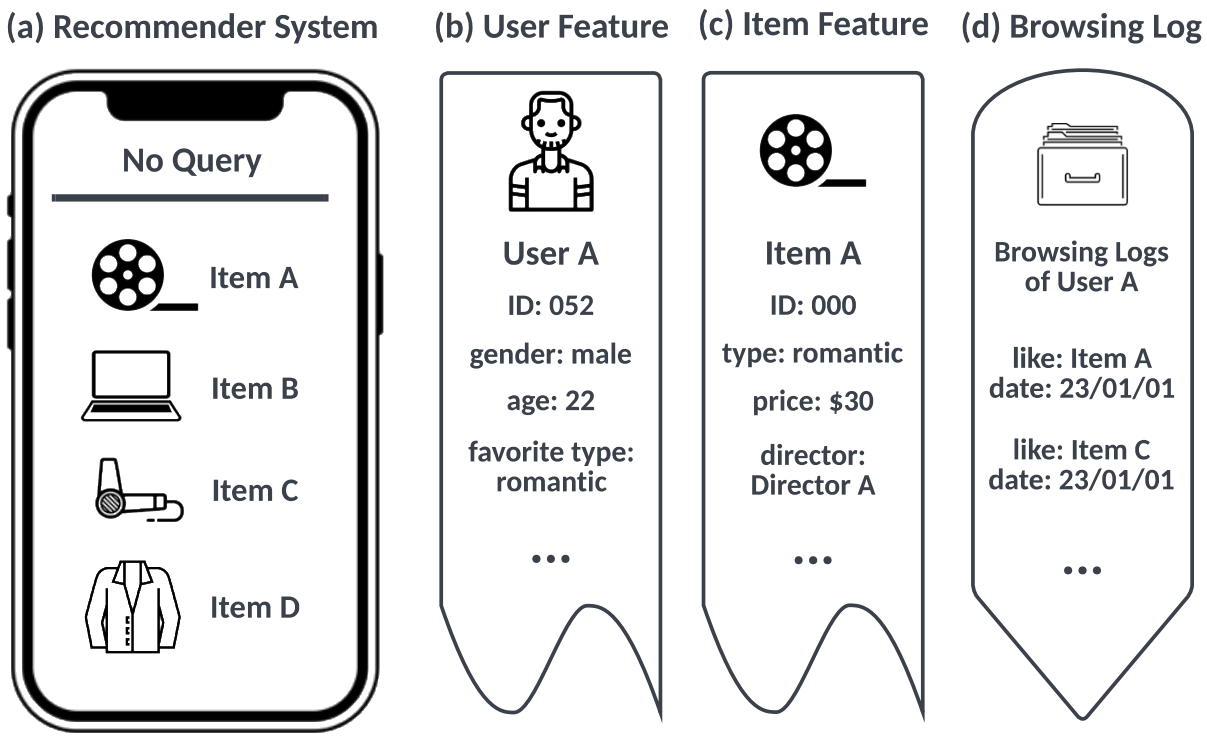
Promotion of Rocket:
Update Mechanism is caring about How the System Responds to Users and How the System Collects Sequence Data
[Jin, et al. CIKM 2023]
[Jin, et al. NeurIPS 2023]

Recommender Systems in Our Life

How do Recommender Systems work?

Learning From Feature

Feature and Pipeline in Recommender Systems



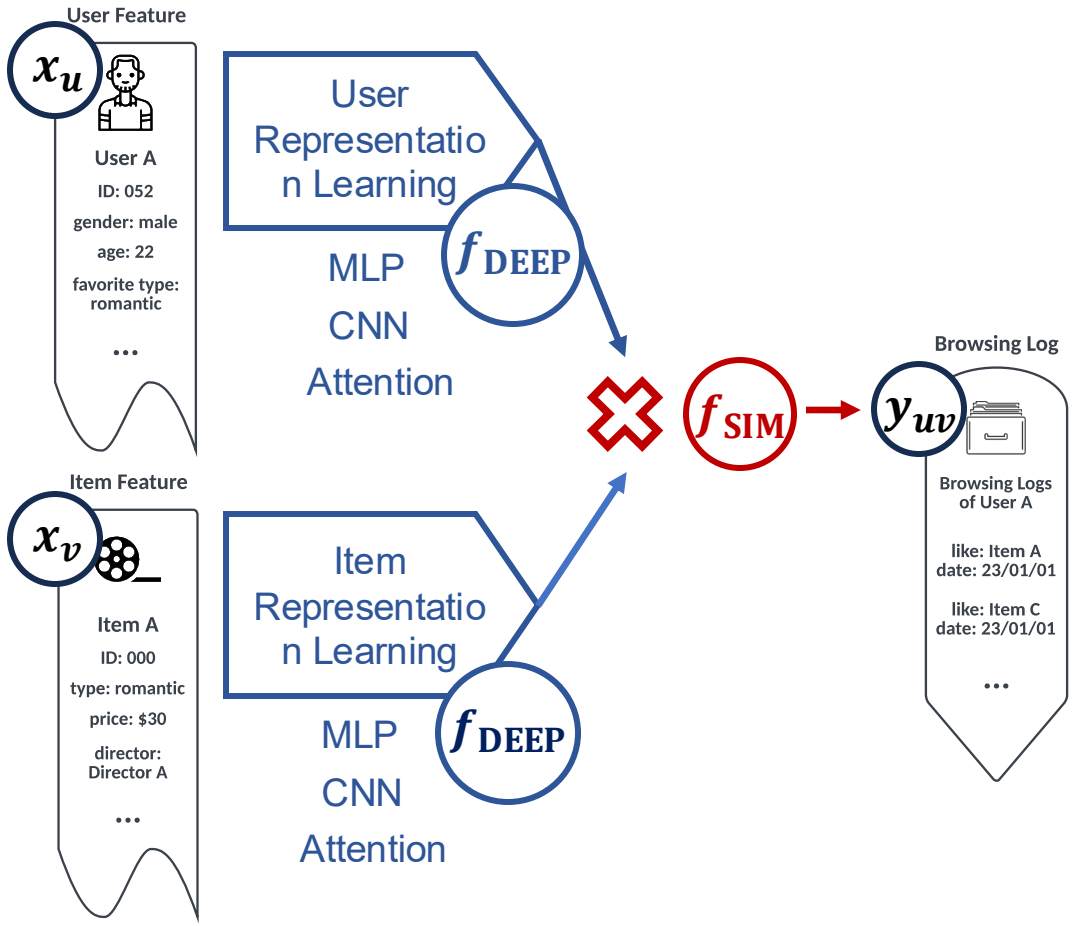
(a) Recommendation Platform for Data Collections and System Responses.

(b) Static Feature of User

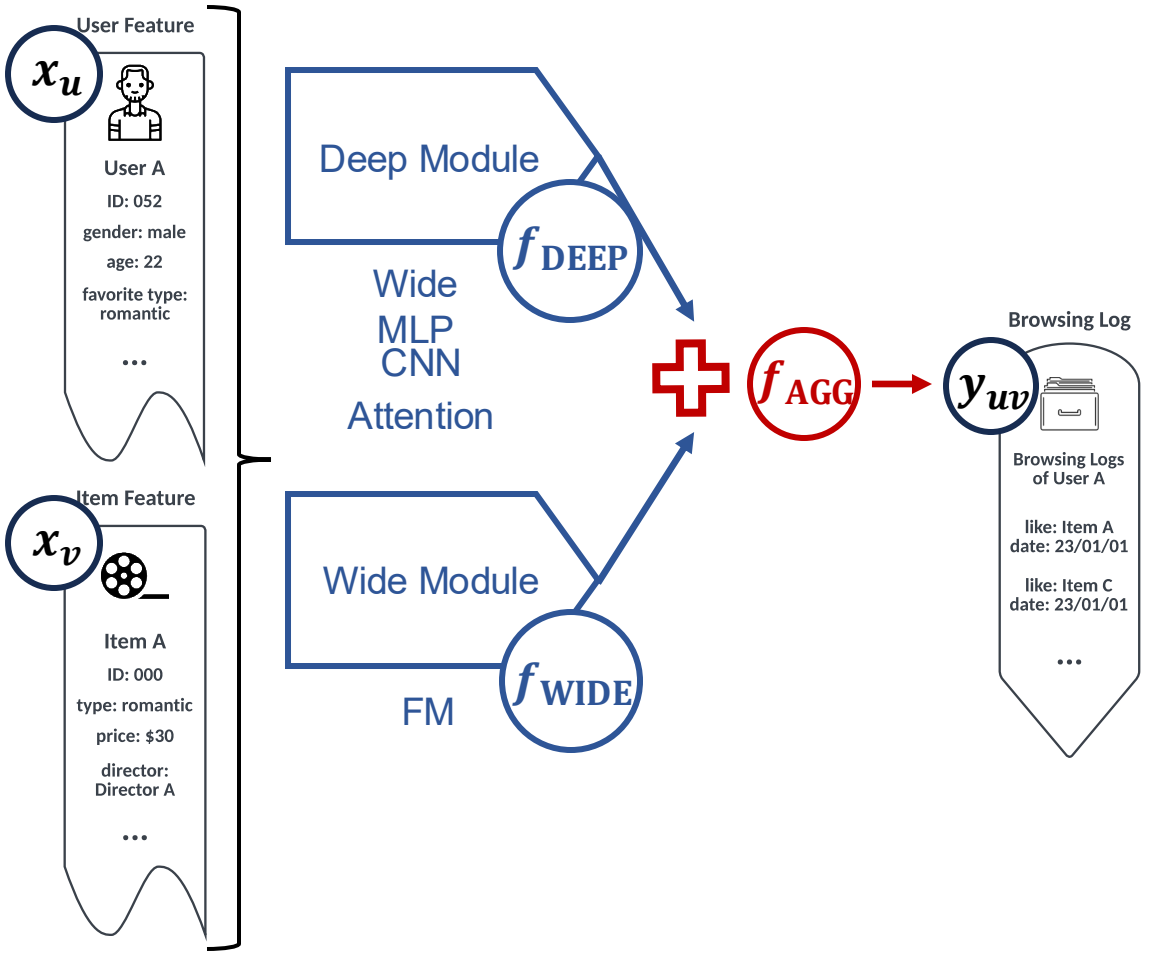
(c) Static Feature of Item

(d) Dynamic Feature of User (Bridge of Users and Items)

Algorithms in Recommender Systems



$$f_{SIM}(f_{DEEP}(x_u), f_{DEEP}(x_v))$$



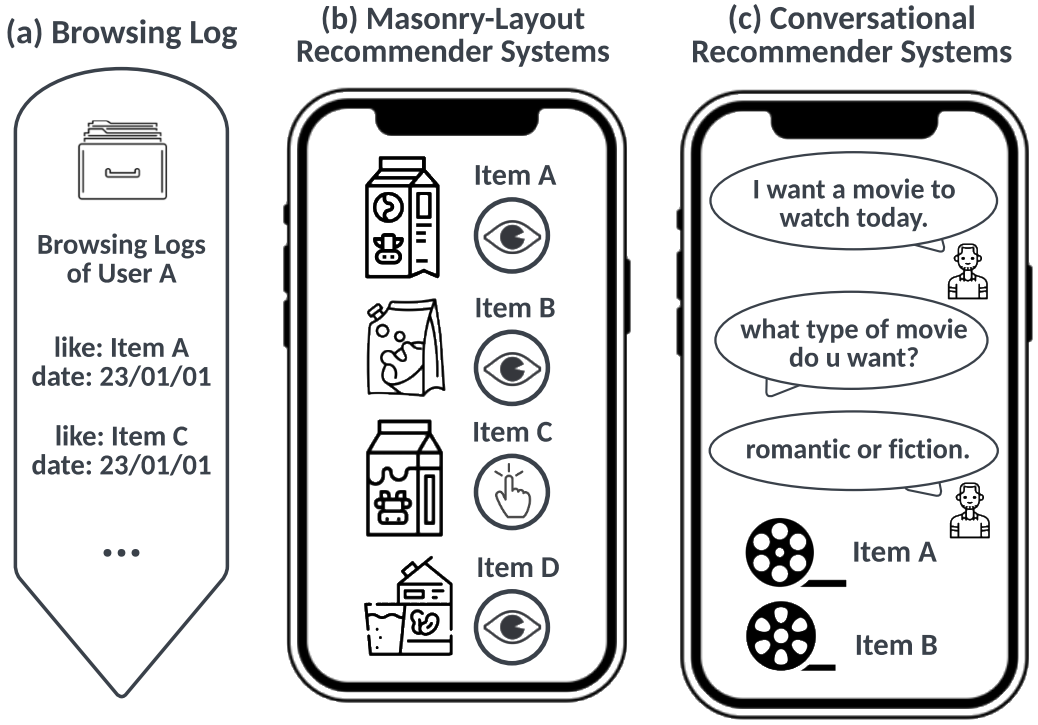
$$f_{AGG}(f_{DEEP}([x_u; x_v]), f_{WIDE}([x_u; x_v]))$$

Why We Need Sequence Representations in Recommender Systems

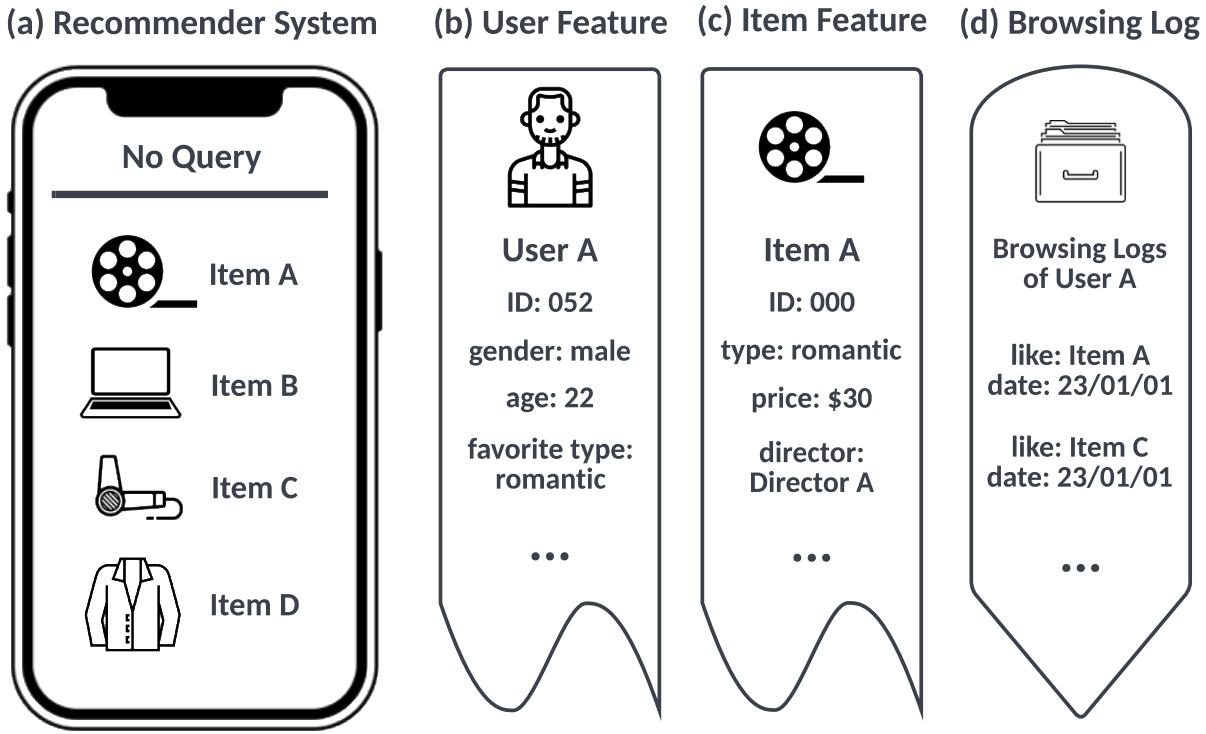
(a) Users' preference and items' popularity are Dynamic rather than Static over time.

(b) User-item Masonry-layout interactions usually happen under a certain Sequential Context.

(c) User-item Conversational interactions requires Sequential Update.



Feature and Pipeline in Recommender Systems

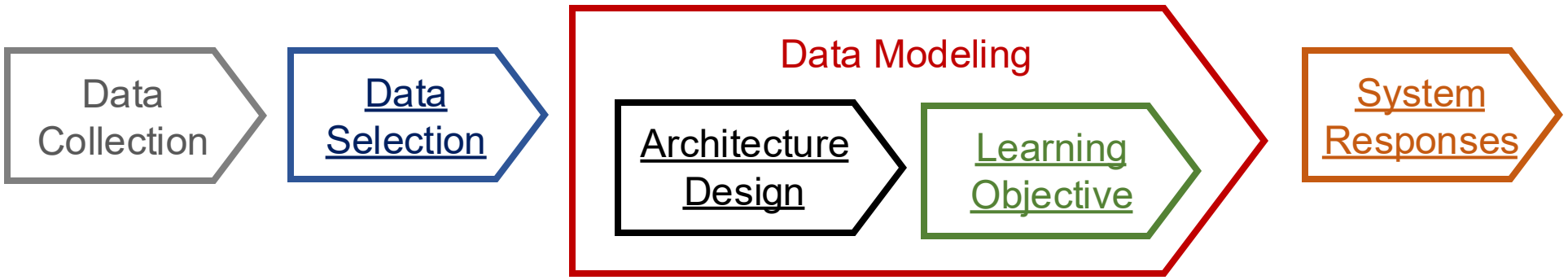


(a) Recommendation Platform for Data Collections and System Responses.

(b) Static Feature of User

(c) Static Feature of Item

(d) Dynamic Feature of User (Bridge of Users and Items)



Observations from Industrial Recommendation Platforms

(a) [Qi, et al. CIKM 2020] developed to search Relevant Data from the whole data as the input for the subsequential model in Data Selection.

(b) [Rendle. ICDM 2010] discovered introducing Interaction Operations for the input feature in Architecture Design.

(c) [Ma, et al. KDD 2018] designed to model Relationship over User Feedback (i.e., Task Objective) from data in Learning Objective.

(d) [ChatGPT. Technical Report 2023] introduced ChatGPT which can empower the recommender systems to establish a Conversation for each user in System Response.

[Qi, et al. CIKM 2020] Pi Qi, et al. Search-based User Interest Modeling with Lifelong Sequential Behavior Data for Click-Through Rate Prediction. WWW 2020.

[Rendle, ICDM 2010] Steffen Rendle. Factorization Machines. ICDM 2010.

[Ma, et al. KDD 2018] Jiaqi Ma, et al. Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts. KDD 2018.

[ChatGPT. Technical Report] OpenAI. GPT-4 Technical Report. 2023.

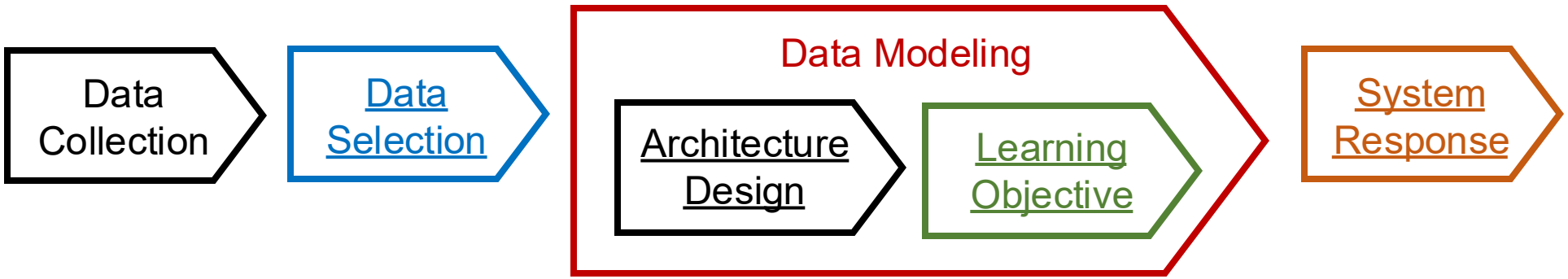
Observations from Industrial Recommendation Platforms

(a) Relevant Data in Data Selection.

(b) Interaction Operations in Architecture Design.

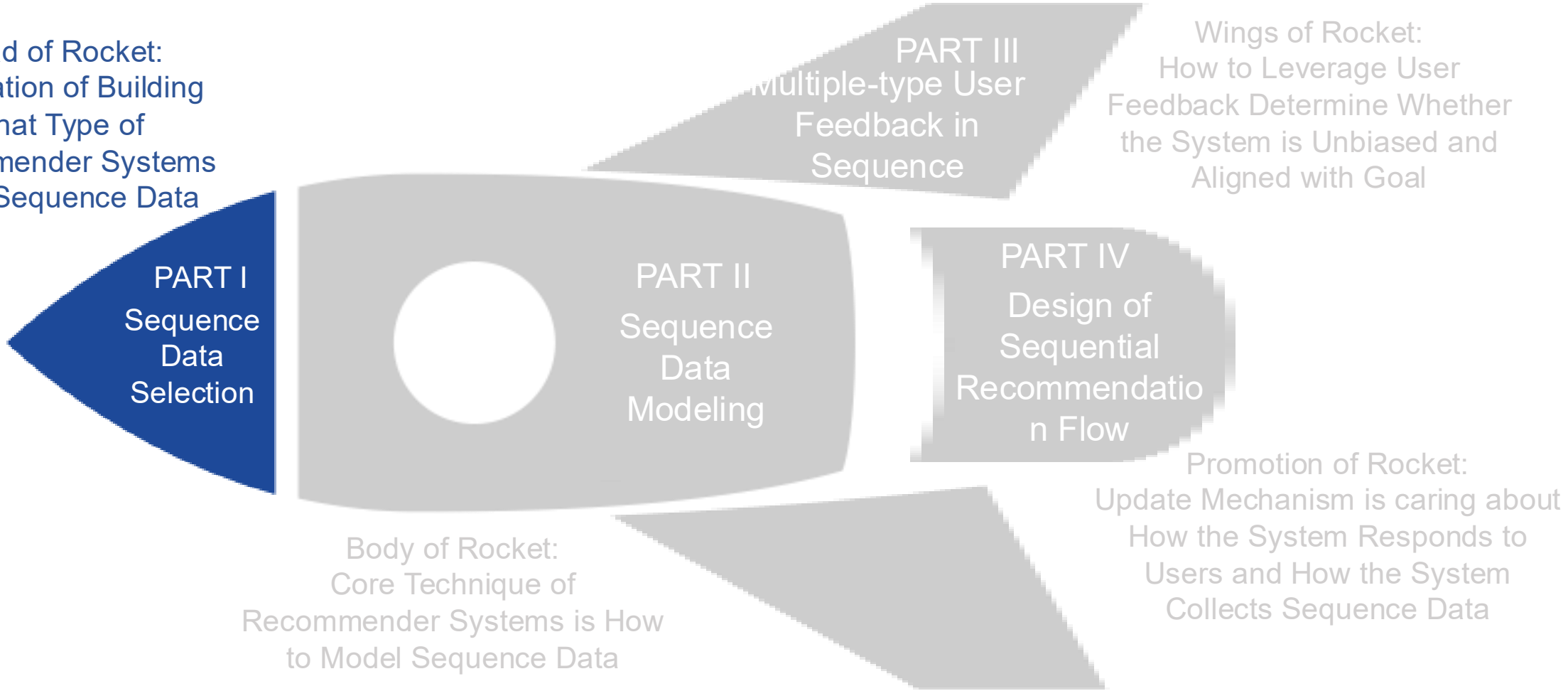
(c) Relationship over User Feedback (i.e., Task Objective) in Learning Objective.

(d) Conversations in System Response.



Content

Data Collection → Data Selection



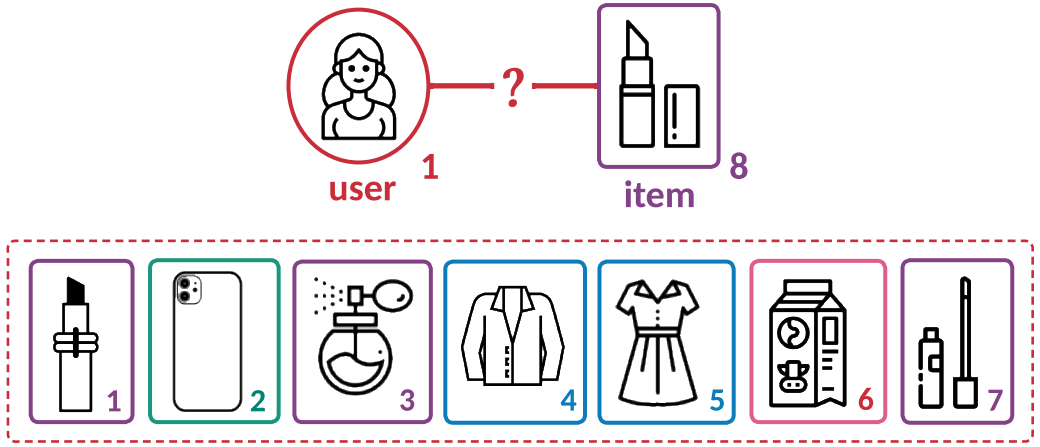
PART I: Sequence Data Selection

What Kind of Sequence Data are Useful?

Maybe We Can Call Them "Relevant"

What are "Relevant" Data

- Our answer is Recent and Similar Data regarding Target User-Item Pair.
- Recent represents the impact of Time, where more recently viewed items are usually more representative for user interests, as user interests are drifting.
- Similar is measured by the Category of item, where items belonging to the same or similar category are usually more representative for user interest on the target item.



[Jin, et al. WWW 2022a] Jiarui Jin, et al. Learn over Past, Evolve for Future: Search-based Time-aware Recommendation with Sequential Behavior Data. WWW 2020.

How to Measure "Similar"

- Similar is measured by the Category of item, where items belonging to the same or similar category are usually more representative for user interest on the target item.
- Hard-Search strategy: for each user-item pair $\langle u_m, i_n \rangle$, we construct a set of similar items from user u_m's browsed items $\mathcal{H}_m$ following:

$$\widehat{\mathcal{H}}_m := \{i_{n'} | i_{n'} \in \mathcal{H}_m \wedge f_{\mathrm{HARD}}(c_{n'}, c_n) \geq \epsilon\}$$

where $f_{\mathrm{HARD}}(c_n, c_{n'}) := -|c_{n'} - c_n|$ and $\epsilon = 0$. $c_{n'}$ and c_n are one-hot vectors directly represent their categories without any learnable parameter.

- Hard-search strategy does not work in the following case where a teen purchases a beautiful dress and she needs lipstick to make up.

[Jin, et al. WWW 2022a] Jiarui Jin, et al. Learn over Past, Evolve for Future: Search-based Time-aware Recommendation with Sequential Behavior Data. WWW 2020.

How to Measure "Similar"

- Similar is measured by the Category of item, where items belonging to the same or similar category are usually more representative for user interest on the target item.
- Soft-Search strategy leverages the embedding vectors of category of items (NOT the embedding vector of items): $f_{\mathrm{SOFT}}(x_{n'}, x_n) \coloneqq \cos(x_{n'}, x_n) = (x_{n'}^{\mathrm{T}} \cdot x_n)/(|x_{n'}^{\mathrm{T}}| \cdot |x_n|)$ where $\cos(\cdot,\cdot)$ denotes cosine similarity. One can obtain retrieved items by the soft-search strategy through replacing $f_{\mathrm{HARD}}(c_{n'}, c_n)$ by $f_{\mathrm{SOFT}}(x_{n'}, x_n)$ and assigning $0 < \epsilon < 1$.
- Adaptive-Search strategy combines the advantage of hard- and soft-search strategies:

$$f_{\mathrm{ADA}}(c_n, c_n, x_{n'}, x_n) \coloneqq -\frac{\mathrm{sign}(|c_{n'} - c_n|)}{1-\tau} + f_{\mathrm{SOFTMAX}}(\cos(x_{n'}, x_n), \tau)$$

where $\tau \in (0,1)$ denotes temperature hyper-parameter.

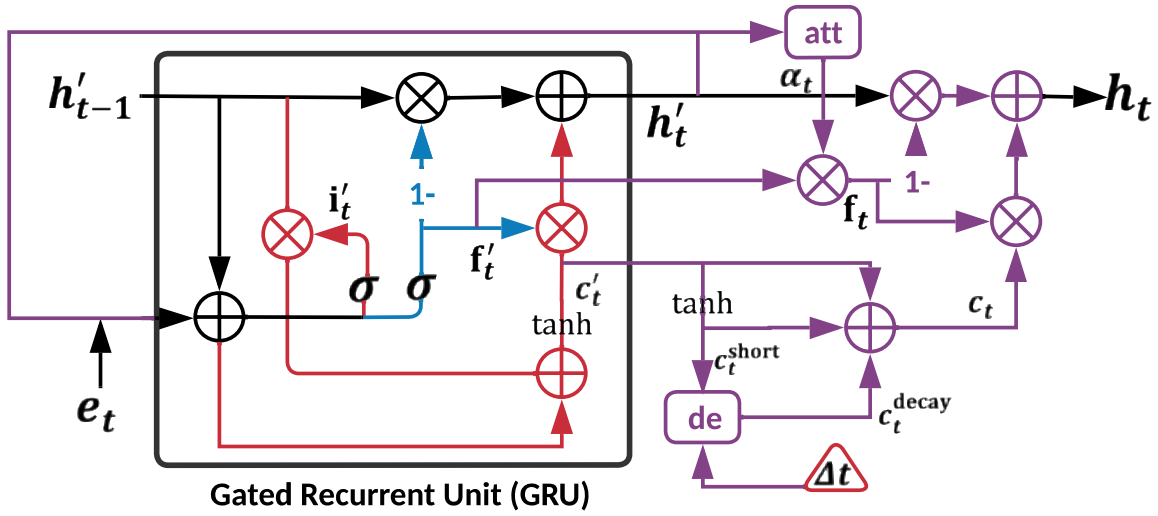
[Jin, et al. WWW 2022a] Jiarui Jin, et al. Learn over Past, Evolve for Future: Search-based Time-aware Recommendation with Sequential Behavior Data. WWW 2020.

Insert “Time” into “Similar”

- Similar items are searched but Time Interval information is missing.
- To model sequence data, we introduce a Gated Recurrent Unit (GRU) (where $\odot$ is element-wise product operator):

$$\mathrm{f}'_t = \sigma(W_{\mathrm{f}'} e_t + U_{\mathrm{f}'} h'_{t-1}),\ i'_t = \sigma(W_{\mathrm{i}'} e_t + U_{\mathrm{i}'} h'_{t-1})$$

$$c'_t = \tanh(W_{c'} e_t + i'_t \odot U_{c'} h'_{t-1}),\ h'_t = \mathrm{f}'_t \odot c'_t + (1 - \mathrm{f}'_t) \odot h'_{t-1}$$



[Jin, et al. WWW 2022a] Jiarui Jin, et al. Learn over Past, Evolve for Future: Search-based Time-aware Recommendation with Sequential Behavior Data. WWW 2020.

Insert "Time" into "Similar"

- Similar items are searched but Time Interval information is missing.
- We further use attention mechanism to model the evolution of user interest and consider the effect from time intervals as:

$$c_t^{\text{short}} = \tanh(W_c c_t' + b_c),\ c_t^{\text{decay}} = c_t^{\text{short}} \cdot \text{de}(\Delta t),\ c_t = c_t' - c_t^{\text{short}} + c_t^{\text{decay}}$$

$$\alpha_t' = \exp(h_t' W_{\alpha'} e_t) \Big/ \sum_{t'=1}^{\hat{T}+1} \exp(h_t' W_{\alpha'} e_t),\ \text{f}_t = \alpha_t' \cdot \text{f}_t',\ h_t = \text{f}_t \odot c_t + (1 - \text{f}_t) \odot h_t'$$

where Δt is the elapsed time between items i_{t-1} and i_t, and $\text{de}(\cdot)$ denotes a heuristic decaying function. We use $\text{de}(\Delta t) = 1/\Delta t$ for datasets with small amount of elapsed time and $\text{de}(\Delta t) = 1/\log(e + \Delta t)$ for those with large elapsed time in practice.

[Jin, et al. WWW 2022a] Jiarui Jin, et al. Learn over Past, Evolve for Future: Search-based Time-aware Recommendation with Sequential Behavior Data. WWW 2020.

Insert "Time" into "Similar"

- Similar items are searched but Time Interval information is missing.
- For each sequence $\widehat{\mathcal{H}}_m$, we can obtain a set of hidden states $\{h_t | i_t \in \widehat{\mathcal{H}}_m\}$, and we employ an attention model as an aggregation function to fuse these embeddings:

$$e_m = f_{\mathrm{AGG}}(\{h_t | i_t \in \widehat{\mathcal{H}}_m\}) = \sigma(W_m \cdot \left(\sum_{i_t \in \widehat{\mathcal{H}}_m} \alpha_t (h_t W_t) \right) + b_m)$$

$$\alpha_t = \exp(h_t W_\alpha) \,/\, \sum_{t'=1}^{\hat{T}+1} \exp(h_t W_\alpha)$$

[Jin, et al. WWW 2022a] Jiarui Jin, et al. Learn over Past, Evolve for Future: Search-based Time-aware Recommendation with Sequential Behavior Data. WWW 2020.

Combine "Recent" and "Similar"

- We separately use a convolutional GRU to model the recently viewed items ("Recent"), and use our modified GRU to model the searched items ("Similar").
- We then combine these results to get the final prediction for user-item pair $\langle u_m, i_n \rangle$:

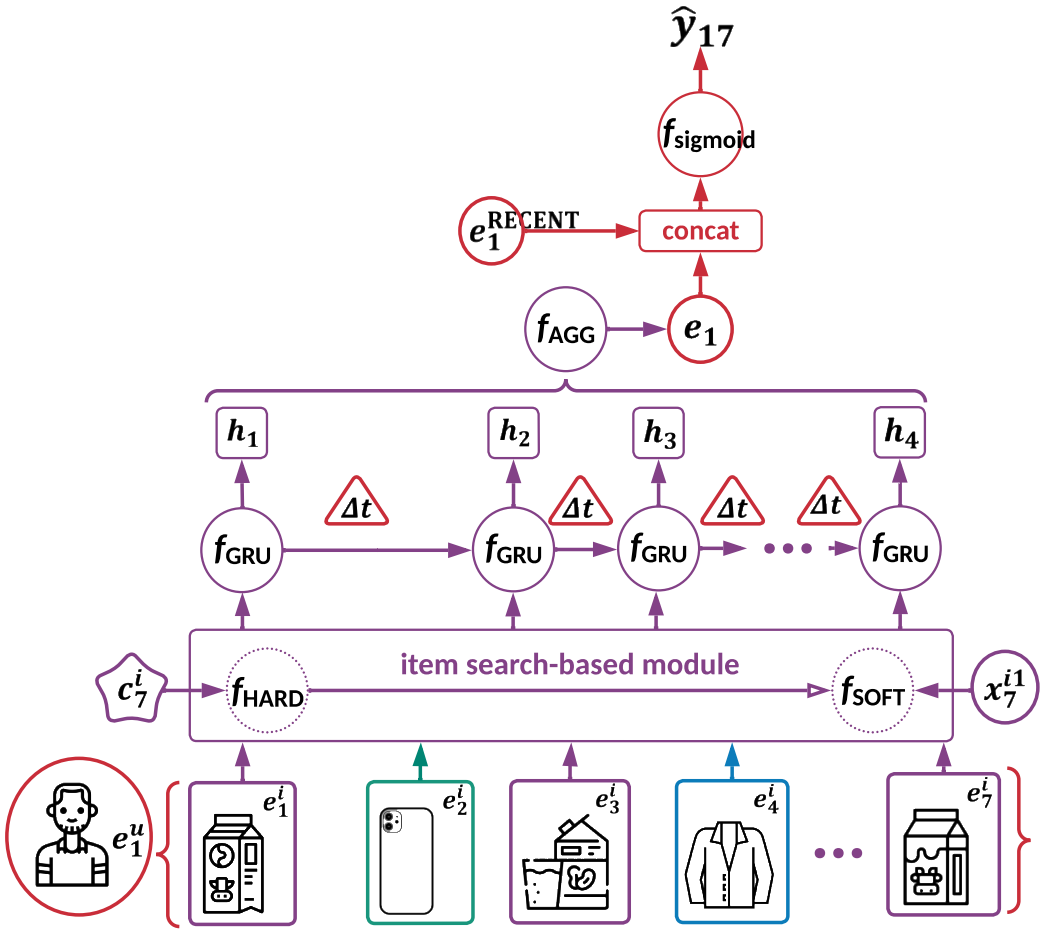
$$\hat{y}_{mn} = \sigma(f_{\text{MLP}}([e_m; e_m^{\text{RECENT}}]))$$

where the predictions are supervised by a log loss:

$$\mathcal{L} = -\sum_{\langle u_m, i_n \rangle \in \mathcal{D}} (y_{mn} \cdot \log \hat{y}_{mn} + (1 - y_{mn}) \cdot \log(1 - \hat{y}_{mn}))$$

[Jin, et al. WWW 2022a] Jiarui Jin, et al. Learn over Past, Evolve for Future: Search-based Time-aware Recommendation with Sequential Behavior Data. WWW 2020.

Overview of our STARec Model



- The figure shows the case where our STARec model predicts whether user u_1 likes item i_7.
- Compared to solely using recently viewed items e_1^{RECENT}, our method leverages the similar items in the long sequential historical data and model them with a new GRU variant considering the time interval information.

[Jin, et al. WWW 2022a] Jiarui Jin, et al. Learn over Past, Evolve for Future: Search-based Time-aware Recommendation with Sequential Behavior Data. WWW 2020.

Offline Evaluation of our STARec Model

Recommender	Tmall			Alipay			Taobao		
	AUC	ACC	LogLoss	AUC	ACC	LogLoss	AUC	ACC	LogLoss
LSTM	0.6973	0.7054	0.5854	0.8357	0.7697	0.4713	0.5912	0.4516	0.4411
LSTM$^{+}_{\text{label}}$	**0.7662**	**0.7324**	**0.5291**	**0.9052**	**0.8449**	**0.3738**	**0.6450**	**0.5780**	**0.4094**
RRN	0.6973	0.7073	0.5866	0.8429	0.7145	0.4636	0.5102	0.1502	0.4398
Time-LSTM	0.6962	0.6796	0.5865	0.8439	0.8057	0.4874	0.5945	0.4393	0.4397
NHP	0.6979	0.6969	0.5849	0.8490	0.7922	0.4743	0.6003	0.3205	0.4393
DUPN	0.5551	0.6796	0.6269	0.8021	0.7552	0.5243	0.5525	0.1921	0.4471
NARM	0.6396	0.6839	0.6025	0.8422	0.7695	0.4860	0.5972	0.3426	0.4494
STAMP	0.6753	0.6946	0.5829	0.8178	0.7491	0.5137	0.6012	0.3592	0.4448
ESMM	0.5189	0.6796	0.6312	0.7131	0.6856	0.6080	0.5487	0.1774	0.4573
ESM^2	0.5149	0.6796	0.6310	0.7241	0.6930	0.5996	0.5030	0.1520	0.4594
MMoE	0.5060	0.6796	0.6313	0.7119	0.6834	0.6085	0.5501	0.1719	0.4565
DIN	0.6878	0.6946	0.5915	0.8496	0.7692	0.4717	0.5978	0.4422	0.4388
DIEN	0.6892	0.6962	0.5833	0.8474	0.7949	0.4668	0.5963	0.4025	0.4412
SIM	0.7005	0.7094	0.5698	0.8549	0.8069	0.4623	0.6045	0.4538	0.4271
STARec$^{-}_{\text{time}}$	0.6999	0.7097	0.5684	0.8527	0.8046	0.4547	0.6035	0.4525	0.4312
STARec$^{-}_{\text{recent}}$	0.7013	0.7081	0.5632	0.8536	0.8012	0.4672	0.6021	0.4561	0.4355
STARec	**0.7204***	**0.7150***	**0.5471***	**0.8624***	**0.8142***	**0.4410***	**0.6126***	**0.4629***	**0.4211***
STARec$^{+}_{\text{label}}$	**0.7986***	**0.7502***	**0.5059***	**0.9201***	**0.8661***	**0.3423***	**0.6771***	**0.6039***	**0.3862***

- STARec$^{-}_{\text{time}}$ is a variant using a standard GRU as the sequential encoder.
- STARec$^{-}_{\text{recent}}$ is a variant where recently viewed items are not considered.
- STARec$^{+}_{\text{label}}$ is a variant using our proposed label trick to use the user feedback as input (which will be later introduced in PART III).

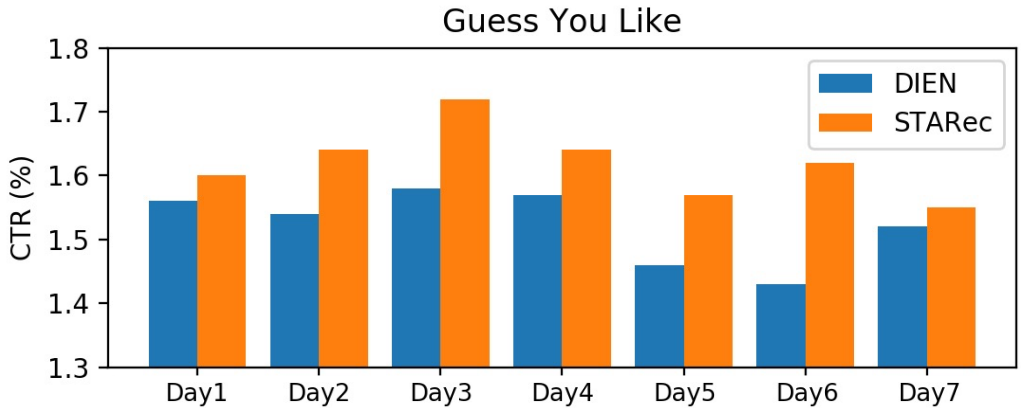
[Jin, et al. WWW 2022a] Jiarui Jin, et al. Learn over Past, Evolve for Future: Search-based Time-aware Recommendation with Sequential Behavior Data. WWW 2020.

Online A/B Test of our STARec Model

招商银行
CHINA MERCHANTS BANK

- We conduct A/B testing in “Guess You Like” and “Information Flow” scenarios of China Merchants Bank E-commerce Recommendation Platform. The whole online experiment lasts a week, from October 14, 2021 to October 21, 2021.
- STARec gains average performance improvement of around 6% and 1.5% in terms of CTR.

Recommender	Guess You Like		Information Flow	
	AUC	CTR	AUC	CTR
DIN	0.7828	1.47%	0.8409	1.26%
DIEN	0.8068	1.52%	0.8493	1.30%
STARec	**0.8909**	**1.61%**	**0.9159**	**1.32%**

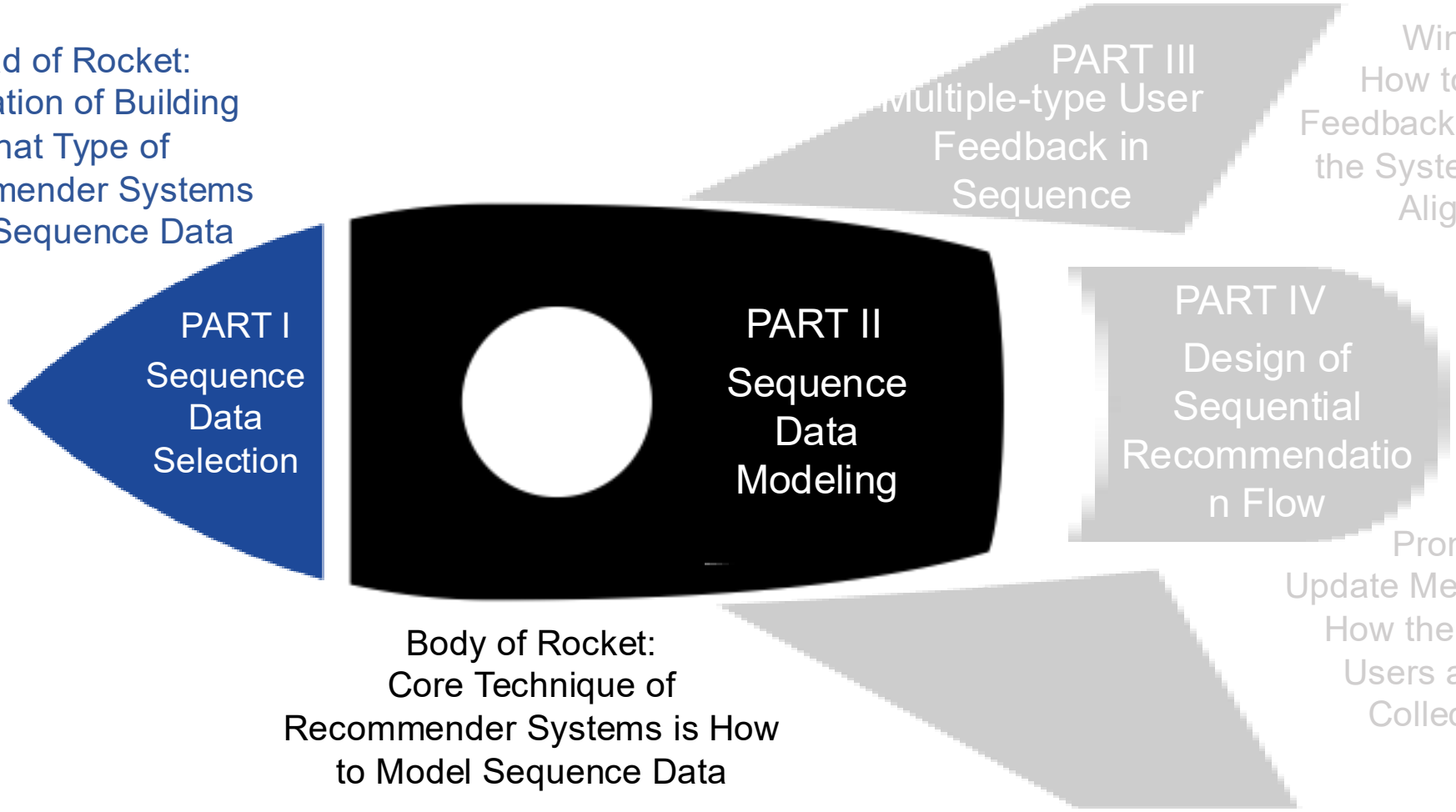


[Jin, et al. WWW 2022a] Jiarui Jin, et al. Learn over Past, Evolve for Future: Search-based Time-aware Recommendation with Sequential Behavior Data. WWW 2020.

Content

Data Collection → Data Selection → Architecture Design

Head of Rocket:
Orientation of Building What Type of Recommender Systems using Sequence Data



Body of Rocket:
Core Technique of Recommender Systems is How to Model Sequence Data

Wings of Rocket:
How to Leverage User Feedback Determine Whether the System is Unbiased and Aligned with Goal

Promotion of Rocket:
Update Mechanism is caring about How the System Responds to Users and How the System Collects Sequence Data

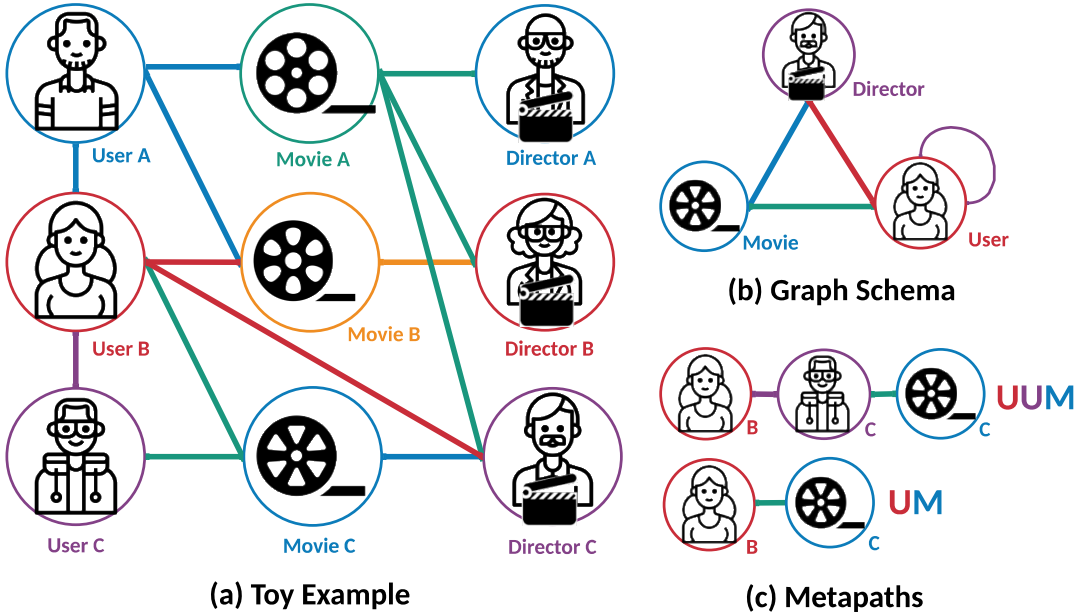
PART II: Sequence Data Modeling

First-Aggregating-Then-Interacting?

Maybe We Can First-Interacting-Then-Aggregating

Sequence in Heterogeneous Graph

- Graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ consisting of a set of nodes $\mathcal{V}$ and a set of edges $\mathcal{E}$. We can build a node type mapping function $\mathcal{V} \rightarrow \mathcal{A}$ and an edge type mapping function $\mathcal{E} \rightarrow \mathcal{R}$.
- Heterogeneous Graph are those graphs satisfying $|\mathcal{A}| + |\mathcal{R}| > 2$.



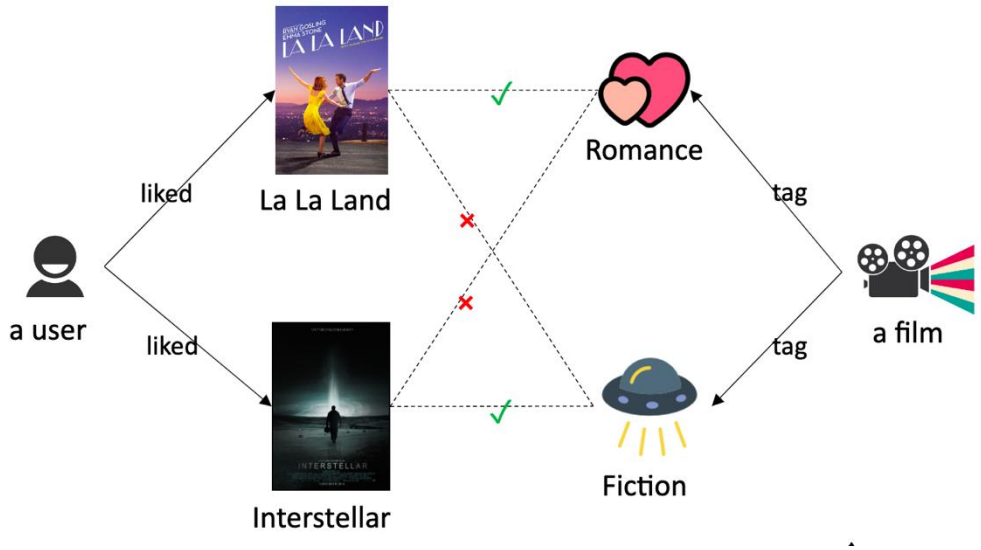
- Metapath ρ defined on a graph schema $\mathcal{S} = (\mathcal{A}, \mathcal{R})$, which is normally given for specific datasets.
- Metapath can be denoted as $A_1 \rightarrow (R_1) \rightarrow A_2 \rightarrow \cdots \rightarrow A_{l+1}$ (abbr. $A_1 A_2 \ldots A_{l+1}$) or $R_1 \circ R_2 \circ \cdots \circ R_l$.
- We can sample sequences to represent neighborhood by using defined metapaths.

[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

Why We Need Interaction Operation

- Early Summarization Issue: existing graph modeling approaches (e.g., graph neural networks) compress rich structural information into only two nodes and single edge.

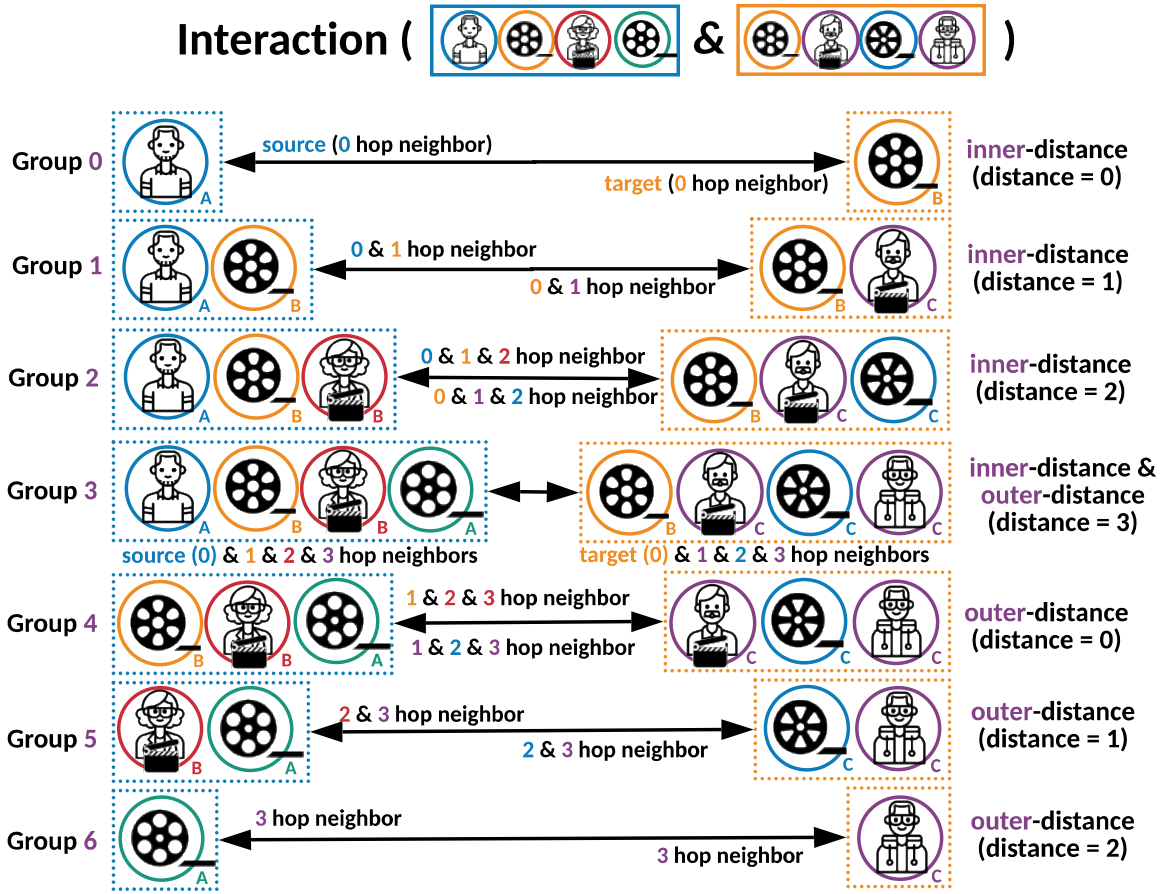


- For example, in the figure, the connection between La La Land and romance, and the connection between Interstellar and fiction could be helpful.; meanwhile, the connection between La La Land and fiction and the connection between Interstellar and romance are nonsense and not expected.

[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

What is Interaction Operation

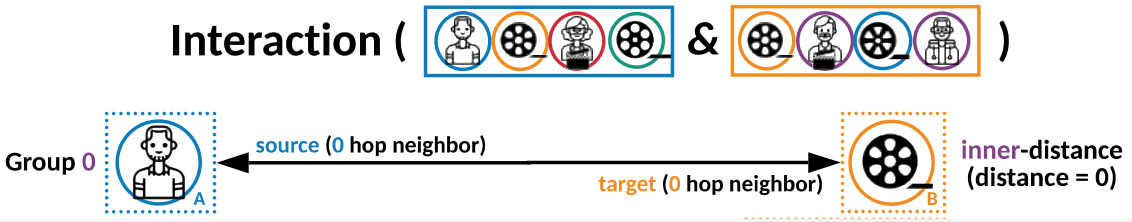


- We group the sampled neighbors according to the distance to the source or target node.
- Our interaction operation is "AND" operation.
- Interaction is only applied between corresponding neighborhood in the same group.
- We find that the whole operation can be formulated as recursively shift, interaction, and shift.

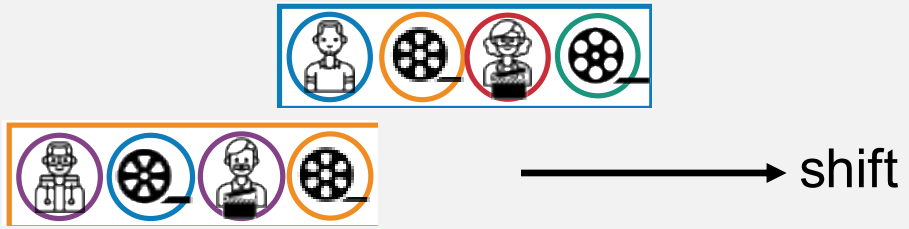
[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

What is Interaction Operation



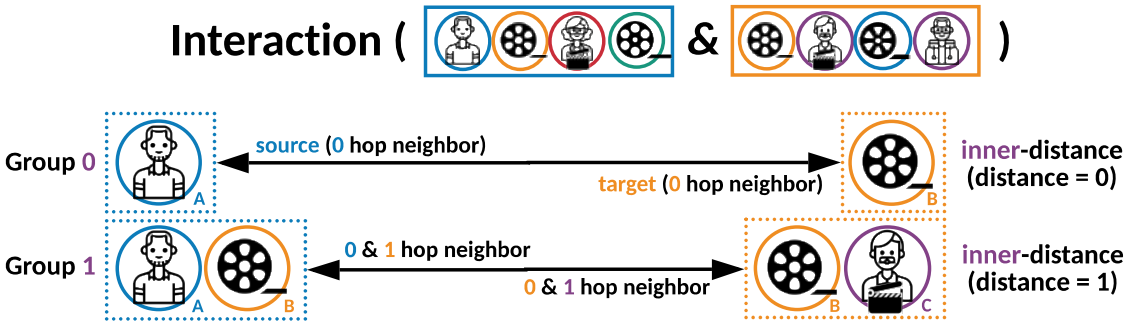
An illustration of interaction process with convolution operation.



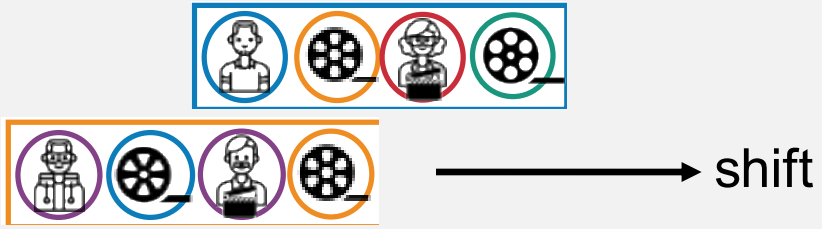
[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

What is Interaction Operation



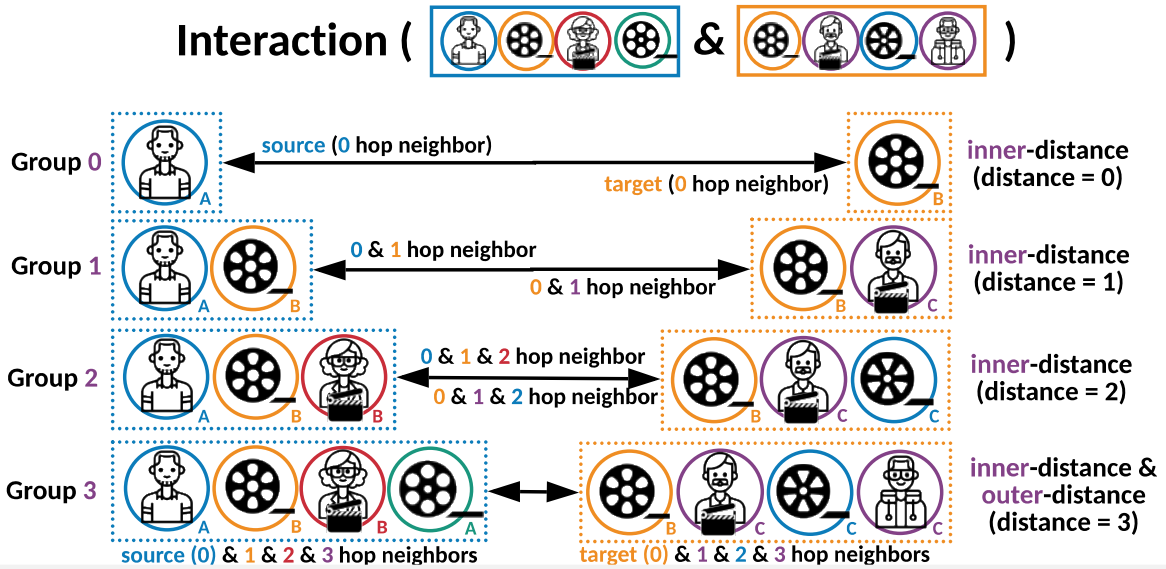
An illustration of interaction process with convolution operation.



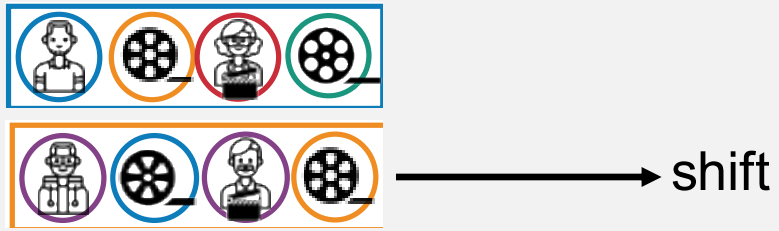
[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

What is Interaction Operation



An illustration of interaction process with convolution operation.



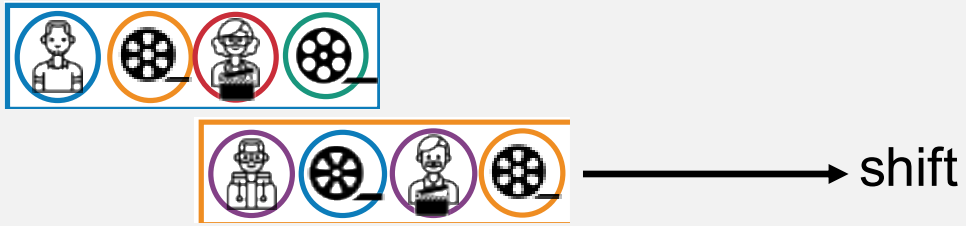
[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

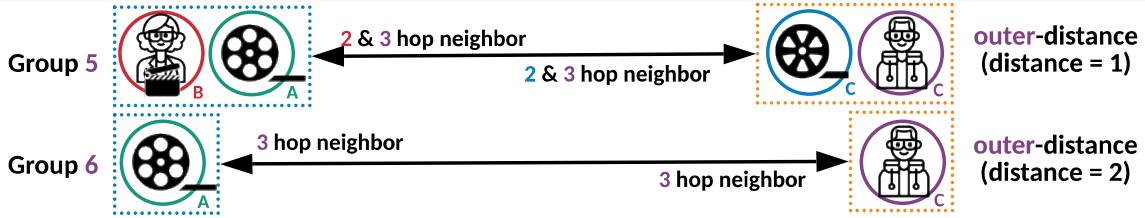
[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

What is Interaction Operation

Interaction ( & )

An illustration of interaction process with convolution operation.





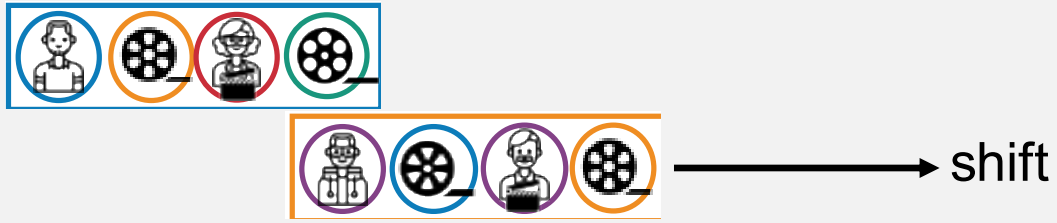
[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

What is Interaction Operation

Interaction ( & )

An illustration of interaction process with convolution operation.

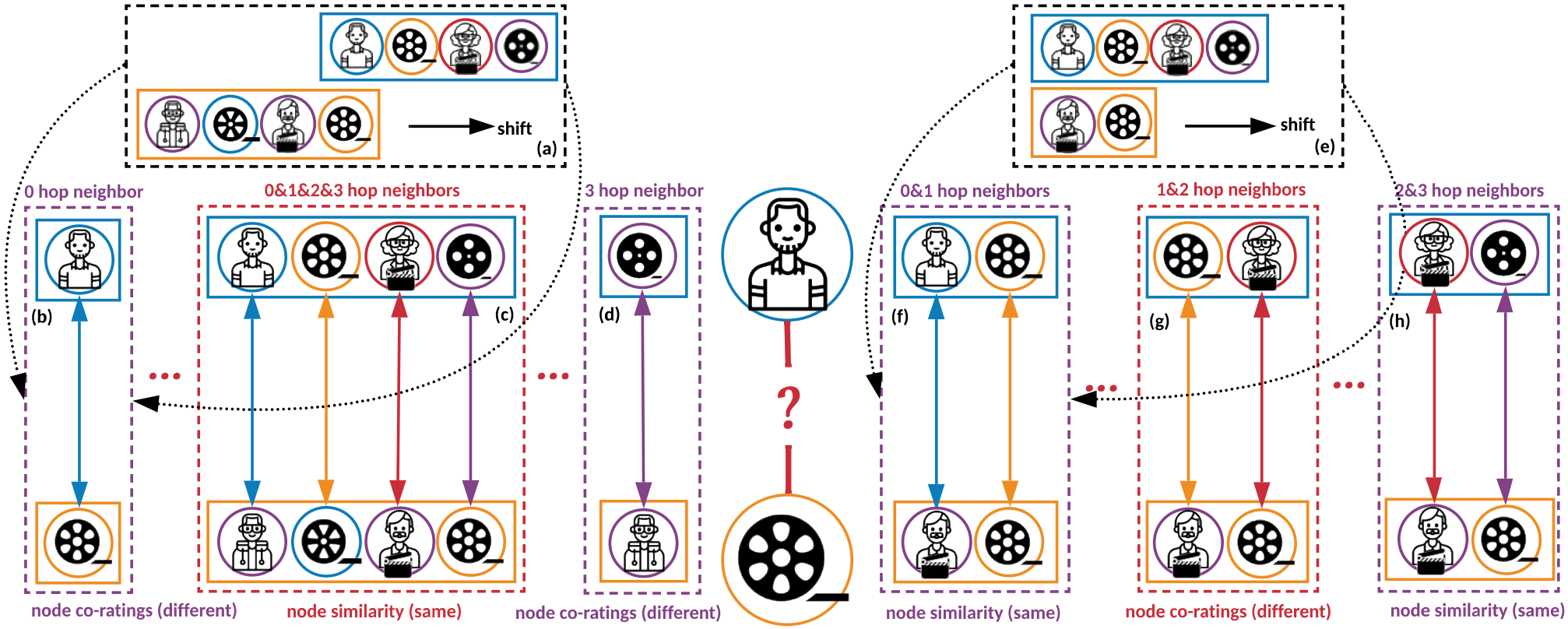


[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

What is Interaction Operation

Our interaction process also can be generalized into pairs of sequences with different length.

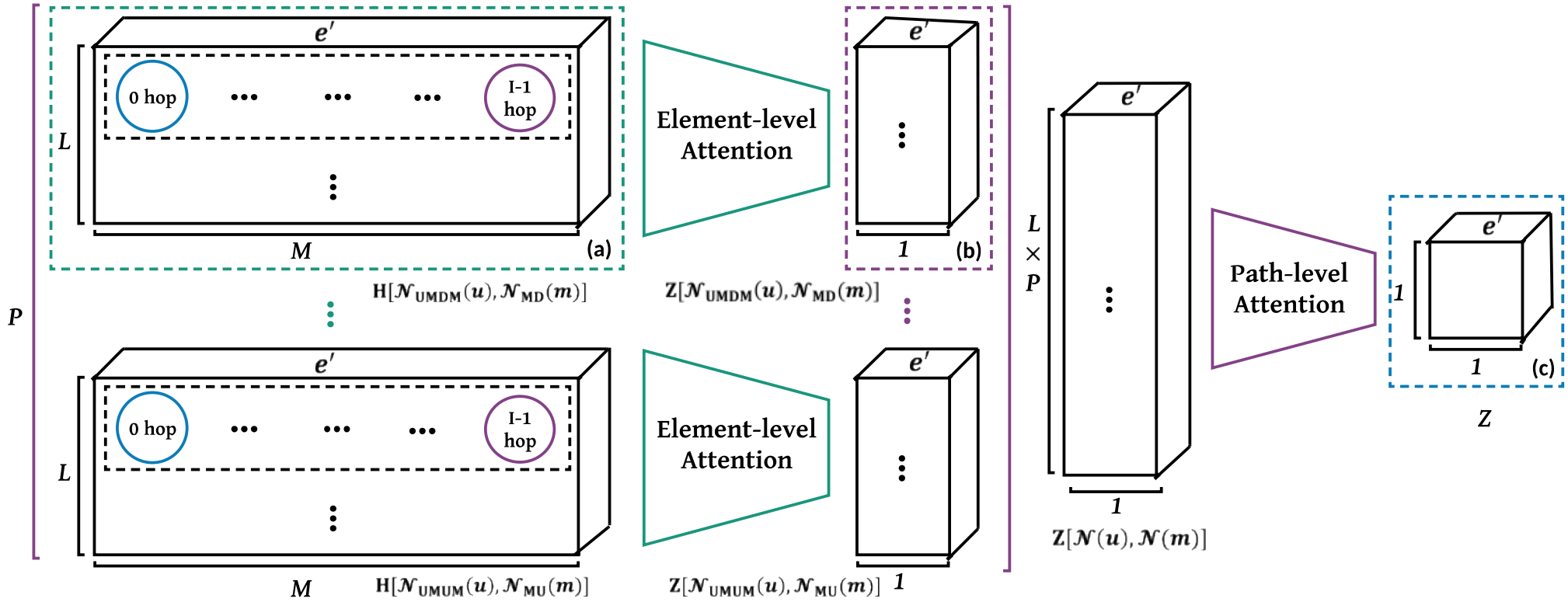


[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

Aggregating Interactions

After getting the interaction results, we first aggregate information within each metapath (from (a) to (b)), and then aggregate information across metapaths (from (b) to (c)).



[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

Aggregating Interactions

- Element-level Aggregation. Let ρ_p^l denote the l-th path guided by metapath ρ_p. We leverage an attention mechanism to learn the weights among various kinds of interaction elements in path ρ_p^l as:

$$h_{0j} = (h_0 W_T)^{\mathrm{T}} \cdot (h_j W_S)$$

where W_T, W_S are trainable weights and h_j is an element of interaction matrix $\mathrm{H}[\mathcal{N}_{\rho_p^l}]$. h_{0j} can show how important interaction element h_j will be for interaction element h_0. h_0 is the first element of $\mathrm{H}[\mathcal{N}_{\rho_p^l}]$, which denotes the interaction result of source node s and target node t. We further normalized this value in path scope as (where ι is the temperature factor):

$$\alpha_j = \mathrm{softmax}(h_{0j}/\iota) = \exp(h_{0j}/\iota) \,/\, \sum_{i=0}^{M-1} \exp(h_{0i}/\iota)$$

[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

Aggregating Interactions

- Element-level Aggregation. We leverage the multi-head attention to extend the observation as:

$$z = \sigma(W_q \cdot \left(\frac{1}{H} \sum_{n=1}^{H} \sum_{j \in \mathcal{N}_\rho} \alpha_{jn} \left(h_j W_{Cn} \right) \right) + b_q)$$

where H is the number of attention heads, and W_{Cn}, W_q, b_q are trainable parameters. z can be written as $z^{\rho_p^l}$. We can build $\mathrm{Z}\left[\mathcal{N}_{\rho_p}\right]$ as the concatenation of the set of $z^{\rho_p^l}$s with L paths. Our element-level aggregation $\mathrm{Z}[\mathcal{N}]$ is the concatenation of the set of $\{\mathrm{Z}[\mathcal{N}_{\rho_0}], \mathrm{Z}[\mathcal{N}_{\rho_1}], \dots, \mathrm{Z}[\mathcal{N}_{\rho_{\mathrm{P}-1}}]\}$ with different metapaths.

[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

Aggregating Interactions

- Path-level Aggregation. We use a self-attention mechanism to learn the attention vector among various path embeddings in $\mathrm{Z}[\mathcal{N}]$. For each path $z_i \in \mathrm{Z}[\mathcal{N}]$, after an MLP, it is fed to a softmax layer:

$$\beta_i = \mathrm{softmax}(z_i/v) = \exp(z_i/v) \,/ \sum_{j=0}^{L\times P-1} \exp(z_j/v)$$

where v denotes the temperature factor. With the learned weights as coefficients, we can fuse these semantic-specific embeddings to obtain the final embedding Z via

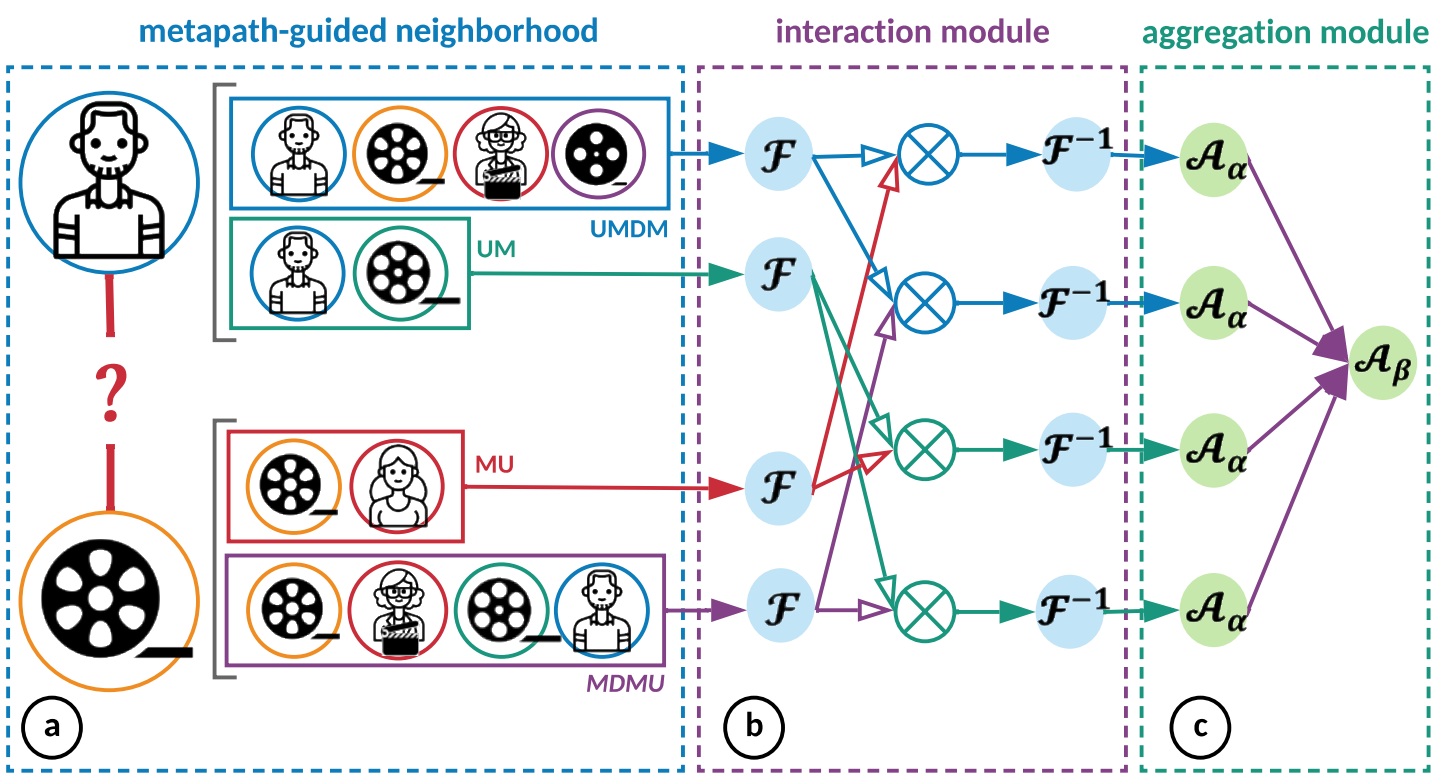
$$Z = \sum_{j=0}^{L\times P-1} \beta_j \cdot z_j$$

The final prediction result can be derived from Z via a nonlinear projection (i.e., MLP). The loss function is a log loss.

[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

Overview of our GraphHINGE Model

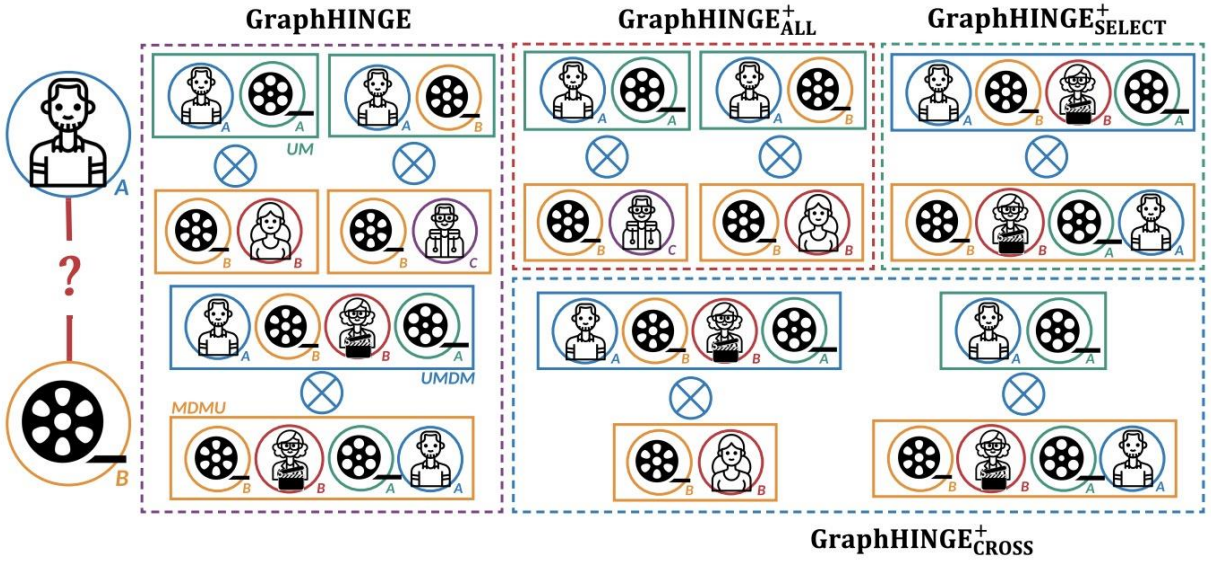


GraphHINGE first samples metapath-guided neighborhood (a); next constructs interactive information via interaction layer (b), where the interaction operations can be accelerated by fast Fourier transform; finally combines rich information through aggregation layer (c).

[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

Offline Evaluation of our GraphHINGE Model



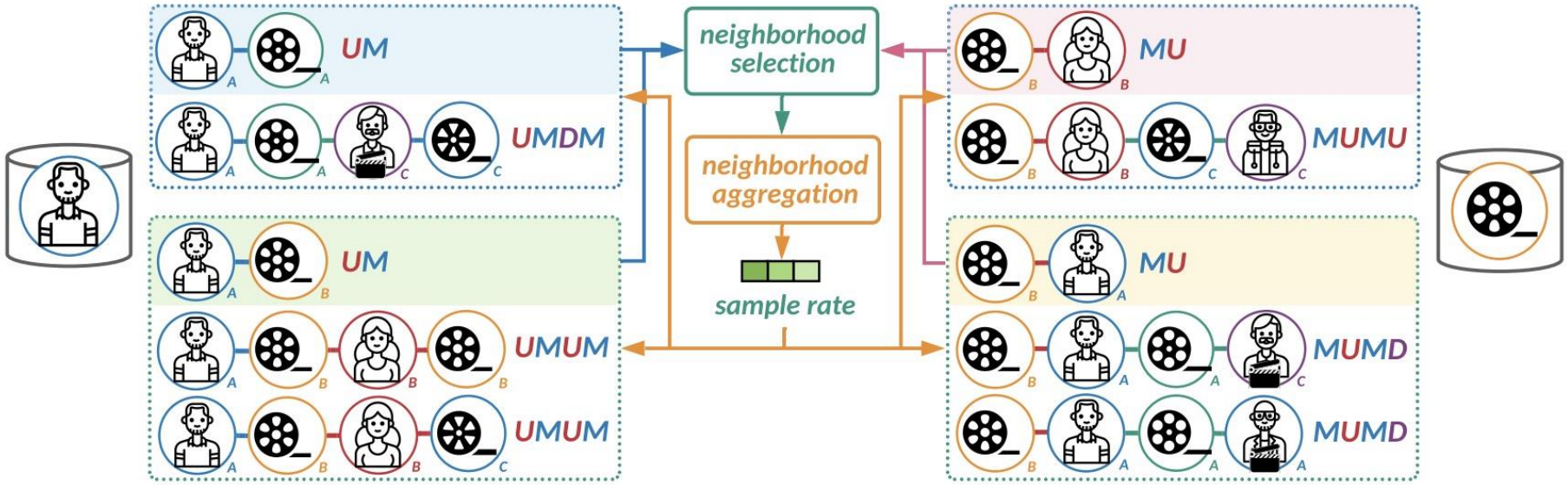
- GraphHINGE only does interaction operations between aligned paths under metapaths with the same length.
- GraphHINGE$_{\mathrm{CNN}}$ does not operate interactions. It directly fuses the neighborhood of each source and target node via Convolutional Neural Network (CNN).
- GraphHINGE$_{\mathrm{GCN}}$ replace the aggregation module by standard Graph Convolutional Network (GCN).
- GraphHINGE$^{+}_{\mathrm{ALL}}$ operates interactions for all paths under metapaths with the same length but suffers from high time complexity.
- GraphHINGE$^{+}_{\mathrm{CROSS}}$ operates interactions for all paths under metapaths even with the different lengths.

[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

Offline Evaluation of our GraphHINGE Model

- GraphHINGE$^{+}_{\mathrm{SELECT}}$ uses the performance of low-order neighborhoods to adaptively sample paths with high-order neighborhoods. For example, using the performance of paths under metapath UM to determine the sample rate for paths under metapaths UMDM and UMUM.



[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

Offline Evaluation of our GraphHINGE Model

Model	Movielens			Amazon			Yelp			DBLP		
	ACC	F1	LogLoss	ACC	F1	LogLoss	ACC	F1	LogLoss	ACC	F1	LogLoss
TAHIN [2]	0.8811	0.8847	0.3029	0.8229	0.8267	0.4363	0.8904	0.8919	0.2666	0.9792	0.9793	0.1954
HAN [62]	0.8713	0.8751	0.3210	0.8481	0.8490	0.4282	0.8894	0.8901	0.2697	0.9752	0.9751	0.2881
HetGNN [71]	0.8510	0.8567	0.3528	0.8313	0.8310	0.4402	0.8719	0.8843	0.2817	0.9512	0.9514	0.3125
IPE [32]	0.8645	0.8693	0.3223	0.8430	0.8411	0.4366	0.8809	0.8876	0.2744	0.9570	0.9589	0.3099
NeuMF [19]	0.8125	0.8421	0.4265	0.7510	0.7143	0.4907	0.7812	0.7742	0.4175	0.8125	0.8012	0.4103
LGRec [20]	0.8144	0.8481	0.4290	0.7127	0.7100	0.5589	0.8308	0.8130	0.3642	0.8158	0.8172	0.3988
MCRec [21]	0.8125	0.8421	0.4234	0.7156	0.7114	0.5557	0.8113	0.8116	0.3302	0.8203	0.8215	0.3937
NEM [66]	0.8032	0.8322	0.4311	0.7021	0.7012	0.5832	0.7909	0.8087	0.3455	0.8167	0.8110	0.4022
GF [70]	0.8100	0.8277	0.4455	0.7018	0.7103	0.5764	0.8002	0.8045	0.3400	0.8185	0.8155	0.4001
MF [29]	0.8098	0.8419	0.4433	0.7571	0.7135	0.4971	0.7841	0.7857	0.4151	0.7155	0.7381	0.5856
DeepFM [15]	0.7284	0.7748	0.5371	0.6931	0.6893	0.5836	0.7162	0.7342	0.5122	0.7212	0.7897	0.5102
ItemKNN [46]	0.8066	0.8062	0.3890	0.6274	0.6355	0.6781	0.5512	0.5339	0.5914	0.7274	0.6940	0.5008
UserKNN [57]	0.8061	0.8045	0.3920	0.6574	0.6471	0.6688	0.5974	0.5877	0.5577	0.7321	0.7319	0.4967
GraphHINGE$_{CNN}$	0.8482	0.8550	0.3549	0.8322	0.8317	0.4493	0.8751	0.8811	0.2233	0.9582	0.9585	0.3051
GraphHINGE$_{GCN}$	0.8727	0.8756	0.3120	0.8813	0.8815	0.3088	0.9014	0.9011	0.2213	0.9879	0.9842	0.1924
GraphHINGE	0.8837*	0.8854*	0.2806*	0.8939*	0.8912*	0.2575*	0.9194*	0.9106*	0.2175*	0.9978*	0.9979*	0.0119*
GraphHINGE$^{+}_{ALL}$	0.8857*	**0.8868***	**0.2749***	0.8960*	0.8930*	0.2531*	0.9201*	0.9109*	0.2178*	0.9983*	**0.9982***	**0.0009***
GraphHINGE$^{+}_{CROSS}$	0.8854*	0.8865*	0.2844*	**0.8965***	**0.8958***	**0.2528***	**0.9225***	**0.9131***	**0.2064***	0.9979*	0.9980*	0.0013*
GraphHINGE$^{+}_{SELECT}$	**0.8858***	0.8860*	0.2778*	0.8945*	0.8922*	0.2530*	0.9217*	0.9110*	0.2146*	**0.9984***	0.9981*	0.0010*

[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

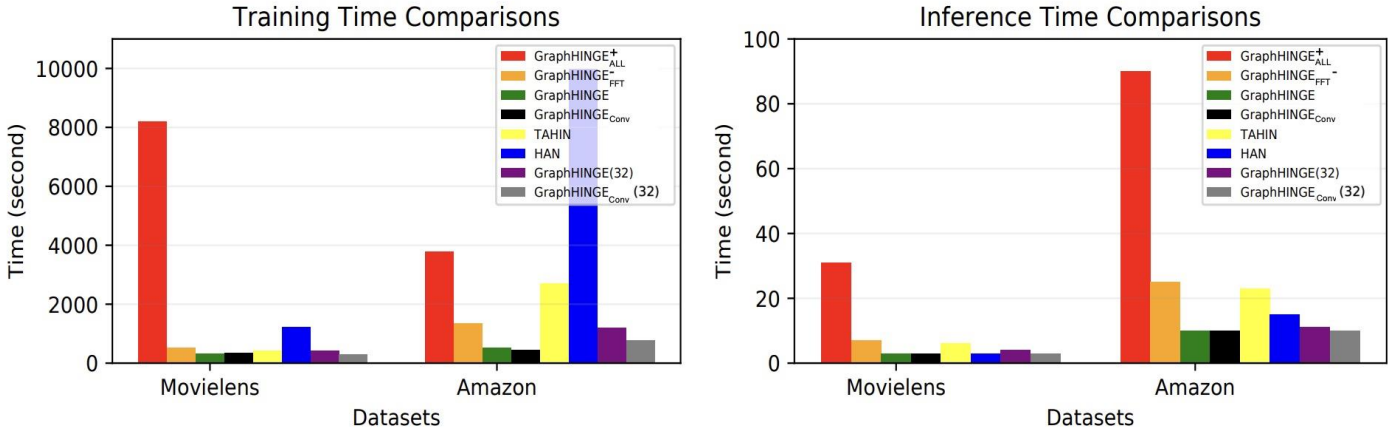
Offline Evaluation of our GraphHINGE Model

Model	Movielens			Amazon			Yelp			DBLP		
	MAP	NDCG@3	NDCG@5	MAP	NDCG@3	NDCG@5	MAP	NDCG@3	NDCG@5	MAP	NDCG@3	NDCG@5
TAHIN [2]	0.6010	0.8908	0.8976	0.7029	0.9396	0.9312	0.7780	0.9523	0.9600	0.8261	0.9780	0.9734
HAN [71]	0.5930	0.8611	0.8723	0.7111	0.9489	0.9370	0.7677	0.9509	0.9432	0.8238	0.9689	0.9695
HetGNN [32]	0.5906	0.8593	0.8699	0.7045	0.9340	0.9410	0.7459	0.9446	0.9409	0.8179	0.9502	0.9508
IPE [62]	0.5910	0.8630	0.8725	0.7093	0.9389	0.9412	0.7503	0.9500	0.9406	0.8182	0.9509	0.9511
NeuMF [19]	0.5816	0.8513	0.8675	0.6433	0.8906	0.8956	0.6501	0.8414	0.8403	0.7853	0.9029	0.9044
LGRec [20]	0.5827	0.8504	0.8634	0.6127	0.8311	0.8435	0.7308	0.8930	0.8904	0.7821	0.9022	0.9042
MCRec [21]	0.5823	0.8565	0.8662	0.6242	0.8618	0.8756	0.7063	0.8717	0.8629	0.7946	0.9119	0.9140
NEM [66]	0.5712	0.8422	0.8499	0.6087	0.8540	0.8579	0.6978	0.8700	0.8599	0.7812	0.9054	0.9023
GF [70]	0.5801	0.8498	0.8510	0.6123	0.8578	0.8665	0.7001	0.8709	0.8602	0.7899	0.9003	0.9104
MF [29]	0.5197	0.8062	0.8090	0.6474	0.8955	0.8981	0.6512	0.8439	0.8414	0.7074	0.7940	0.8008
DeepFM [15]	0.5002	0.7908	0.7982	0.5874	0.7855	0.7881	0.6514	0.8419	0.8397	0.7123	0.8076	0.8098
ItemKNN [46]	0.5161	0.8008	0.8015	0.6074	0.7876	0.7871	0.5409	0.6339	0.6845	0.7120	0.8012	0.8101
UserKNN [57]	0.5153	0.8022	0.8037	0.6043	0.7871	0.7892	0.5506	0.6339	0.6903	0.7121	0.8023	0.8019
GraphHINGE$_{CNN}$	0.5749	0.8535	0.8740	0.7035	0.9488	0.9355	0.7432	0.9534	0.9424	0.8136	0.9516	0.9522
GraphHINGE$_{GCN}$	0.5890	0.8730	0.8885	0.7302	0.9521	0.9414	0.7798	0.9578	0.9608	0.8260	0.9833	0.9850
GraphHINGE	0.6066*	0.9015*	0.9074*	0.7530*	0.9653*	0.9525*	0.7810*	0.9638*	0.9622*	0.8293*	0.9892*	0.9888*
GraphHINGE$^{+}_{ALL}$	**0.6098***	0.9037*	0.9076*	0.7599*	0.9644*	0.9526*	**0.7919***	0.9701*	0.9704*	0.8304*	0.9901*	0.9919*
GraphHINGE$^{+}_{CROSS}$	0.6068*	**0.9043***	**0.9099***	**0.7769***	**0.9670***	**0.9538***	0.7901*	**0.9747***	**0.9714***	0.8303*	0.9944*	0.9918*
GraphHINGE$^{+}_{SELECT}$	0.6070*	0.9038*	0.9085*	0.7755*	0.9666*	0.9532*	0.7909*	0.9745*	0.9706*	**0.8305***	**0.9945***	**0.9920***

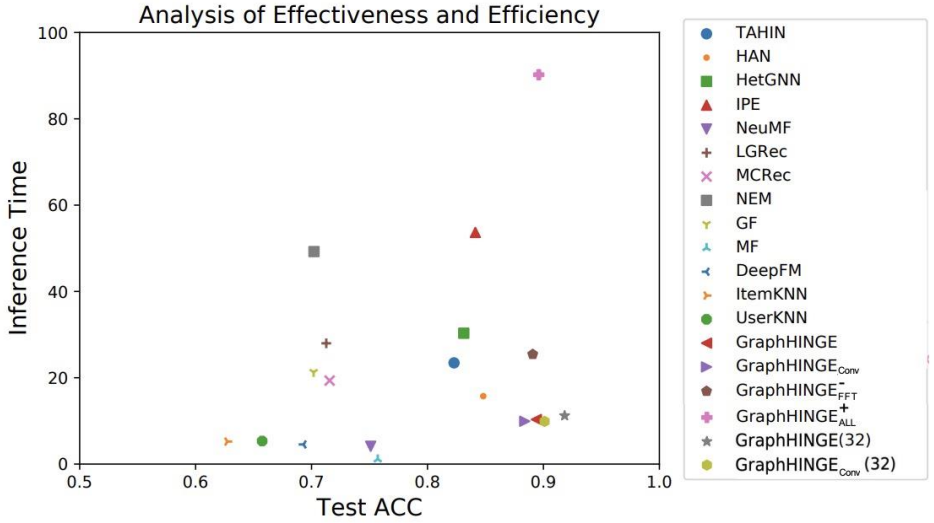
[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

Offline Evaluation of our GraphHINGE Model



Time comparisons of GraphHINGE and its variants. Training time (left figure) and inference time (right figure) on MovieLens and Amazon datasets.



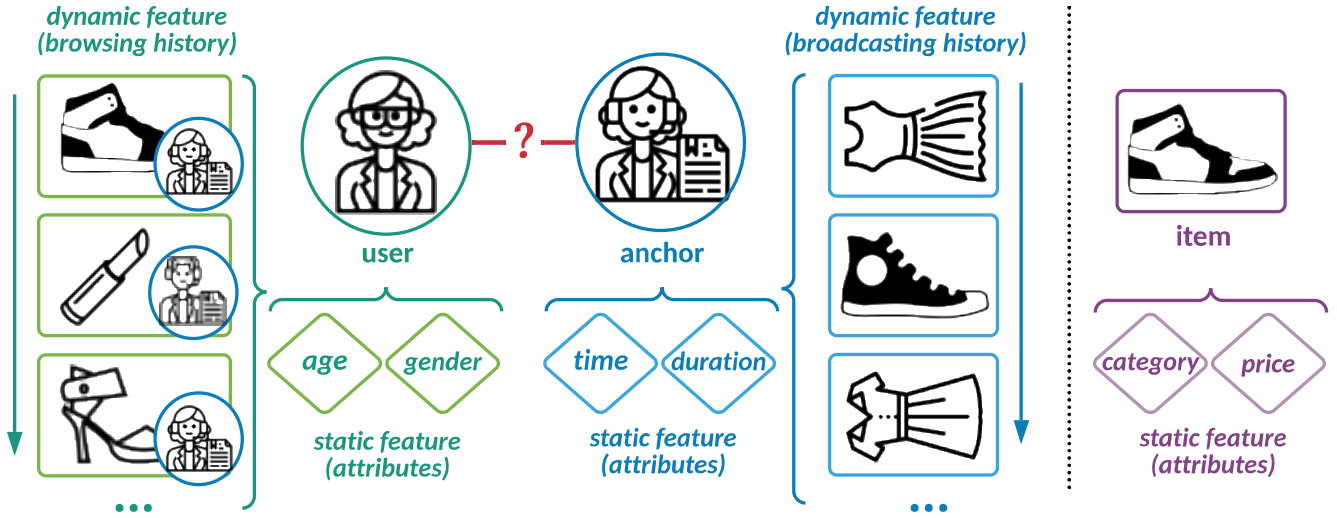
Analysis of the trade-off between effectiveness and efficiency of all the models in terms of ACC and inference time on Amazon dataset. We report the average inference time of each method, evaluated on the whole dataset (i.e., one epoch) after 10 runs.

[Jin, et al. KDD 2020] Jiarui Jin, et al. An Efficient Neighborhood-based Interaction Model for Recommendation on Heterogeneous Graph. KDD 2020.

[Jin, et al. TOIS 2022] Jiarui Jin, et al. GraphHINGE: Learning Interaction Models of Structured Neighborhood on Heterogeneous Information Network. TOIS 2022.

Sequence in Heterogeneous Sequence

- Live Broadcast Recommendation is based on the sequence of historical data of both users and anchors. For users, these sequence data include browsed items, whereas for anchors, these sequence data involve items for live broadcast.
- Live broadcast recommender systems are required to handle the interaction histories between triple objects (i.e., users, anchors, items) rather than the binary interactions between users and items.



[Jin, et al. WWW 2022b] Jiarui Jin, et al. Who to Watch Next: Two-side Interactive Networks for Live Broadcast Recommendation. WWW 2020.

What is Interaction Operation

- Item Aspect. By encoding the static and dynamic features in triple objects, we obtain the embedding vectors of the p-th user (i.e., e_p^u), the q-th item (i.e., h_q^i), and the n-th anchor (i.e., e_n^a). We develop a bi-attention network to better capture the interactive patterns between the sequence of items in user and anchor sides:

$$\alpha_{pq'nq''} = f_{\text{SOFTMAX}}(w_{pq'nq''}^{i\text{T}} \left[e_p^u; h_{q'}^i; e_n^a; h_{q''}^i\right] + b_{pq'nq''}^i)$$

$$y_{pn}^i = \sum_{q' \in \mathcal{H}_p^{ui}} \sum_{q'' \in \mathcal{H}_n^{ai}} \alpha_{pq'nq''} \, (h_{q'}^i \odot h_{q''}^i)$$

where $\odot$ is the element-wise product operator.

[Jin, et al. WWW 2022b] Jiarui Jin, et al. Who to Watch Next: Two-side Interactive Networks for Live Broadcast Recommendation. WWW 2020.

What is Interaction Operation

- Anchor Aspect. Similarly, we formulate the interaction operation as follows:

$$\alpha_{pnn'} = f_{\mathrm{SOFTMAX}}(w_{pnn'}^{a\mathrm{T}}[e_p^u; e_n^a; e_{n'}^a] + b_{pnn'}^a)$$

$$y_{pn}^a = \sum_{n' \in \mathcal{H}_p^{ua}} \alpha_{pnn'} (e_{n'}^a \odot e_n^a)$$

- For each user-anchor pair $(u_p a_n)$, we can obtain the final prediction by:

$$\hat{y}_{pn} = \sigma(f_{\mathrm{MLP}}([y_{pn}^i; y_{pn}^a; y_{pn}^i \odot y_{pn}^a]))$$

We then use a log loss as the objective function.

[Jin, et al. WWW 2022b] Jiarui Jin, et al. Who to Watch Next: Two-side Interactive Networks for Live Broadcast Recommendation. WWW 2020.

What are "Relevant" Data

- Hard-Search Co-Retrieval. We first construct a set of categories for user and anchor sides respectively. Namely $\mathcal{C}_p^u = \{x_{q'} | i_{q'} \in \mathcal{H}_p^{ui}\}$ and $\mathcal{C}_n^a = \{x_{q''} | i_{q''} \in \mathcal{H}_n^{ai}\}$. We then compute a set of the common categories as $\mathcal{C}_{pn}^{ua} = \mathcal{C}_p^u \cap \mathcal{C}_n^a$. We establish a retrieved set of $\mathcal{H}_p^{ui}$ and $\mathcal{H}_n^{ai}$ by following:

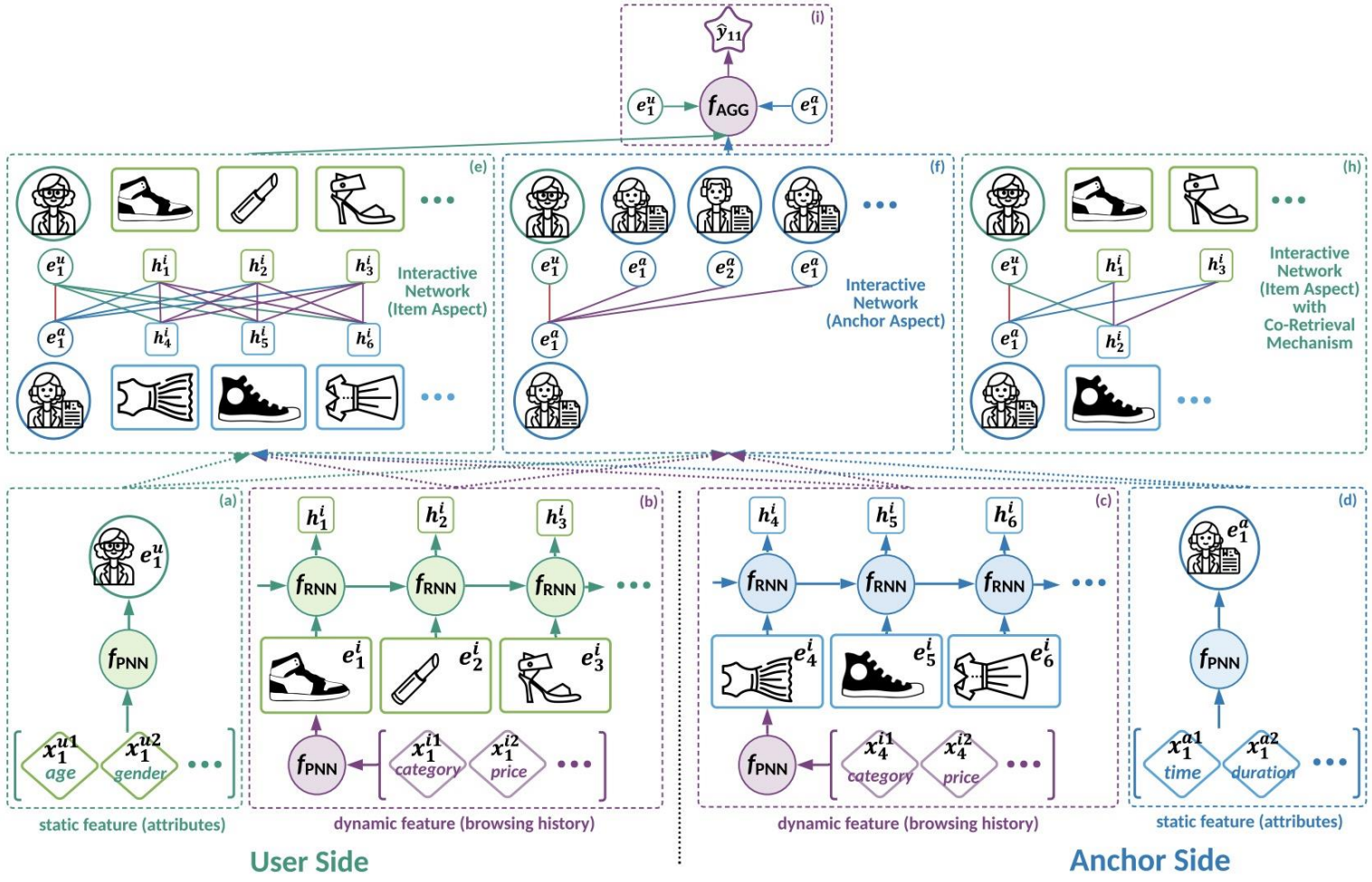
$$\hat{\mathcal{H}}_p^{ui} = \{i_{q'} | i_{q'} \in \mathcal{H}_p^{ui} \text{ and } x_{q'} \in \mathcal{C}_{pn}^{ua}\}$$

$$\hat{\mathcal{H}}_n^{ai} = \{i_{q''} | i_{q''} \in \mathcal{H}_n^{ai} \text{ and } x_{q''} \in \mathcal{C}_{pn}^{ua}\}$$

- As a consequence, we only need to operate interactions between those items whose categories appear in both user and anchor sides.

[Jin, et al. WWW 2022b] Jiarui Jin, et al. Who to Watch Next: Two-side Interactive Networks for Live Broadcast Recommendation. WWW 2020.

Overview of our TWINS Model



The bottom part (i.e., (a)-(d)) shows how we encode the static and dynamic features in user and anchor sides. The up part (i.e., (e)-(i)) illustrates the interactive networks to capture the interactive patterns from item and anchor aspects, which are further aggregated with user and anchor static features to make the final prediction.

[Jin, et al. WWW 2022b] Jiarui Jin, et al. Who to Watch Next: Two-side Interactive Networks for Live Broadcast Recommendation. WWW 2020.

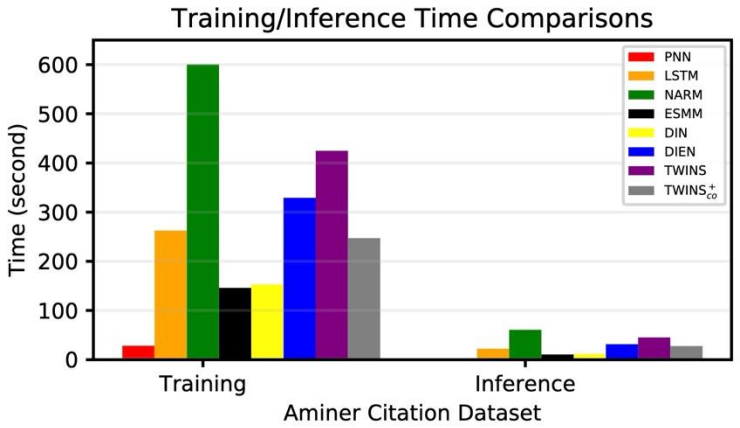
Offline Evaluation of our TWINS Model

- Existing baseline methods are originally proposed particularly for classical item-based recommendation task. Here, we introduce two versions of implementation. Suppose X is a baseline method. We use X^{-} for solely modeling the sequence of user browsed anchors; and we use X for leveraging the sequences of user browsed anchors, anchor broadcast items, and user browsed items.
- TWINS is our model without co-retrieval mechanism.
- TWINS_{i}^{-} is a variant of TWINS, applying the original model without the interactions from item aspect.
- TWINS_{a}^{-} is a variant of TWINS, applying the original model without the interactions from anchor aspect.
- $\text{TWINS}_{\text{co}}^{+}$ is a variant of TWINS using co-retrieval mechanism.

[Jin, et al. WWW 2022b] Jiarui Jin, et al. Who to Watch Next: Two-side Interactive Networks for Live Broadcast Recommendation. WWW 2020.

Offline Evaluation of our TWINS Model

Methods	Yelp Business Dataset			Trust Statement Dataset			Aminer Citation Dataset			Diantao Live Broadcast Dataset		
	LogLoss	ACC	AUC	LogLoss	ACC	AUC	LogLoss	ACC	AUC	LogLoss	ACC	AUC
FM	0.6677	0.5945	0.6233	0.6188	0.7003	0.7574	0.6510	0.6430	0.7071	0.6541	0.6732	0.6896
NeuMF	0.4895	0.7759	0.8424	0.4814	0.7835	0.8440	0.4797	0.7882	0.8542	0.5942	0.7124	0.7229
DeepFM	0.4594	0.7925	0.8658	0.4545	0.8054	0.8756	0.4410	0.8049	0.8826	0.5832	0.7231	0.7345
PNN	0.4581	0.7931	0.8668	0.4452	0.8144	0.8838	0.3932	0.8789	0.9399	0.5432	0.7367	0.7578
DIN$^-$	0.3256	0.8731	0.9420	0.4164	0.8457	0.9154	0.1847	0.9423	0.9799	0.5231	0.7564	0.7790
DIN	0.3156	0.8782	0.9451	0.3771	0.8394	0.9114	0.1212	0.9581	0.9892	0.5100	0.7784	0.7995
LSTM$^-$	0.3236	0.8736	0.9433	0.3931	0.8271	0.9012	0.2218	0.9204	0.9708	0.5334	0.7529	0.7602
LSTM	0.3204	0.8783	0.9445	0.3854	0.8325	0.9051	0.1214	0.9575	0.9891	0.5321	0.7602	0.7789
NARM$^-$	0.3132	0.8811	0.9463	0.3916	0.8306	0.9037	0.2156	0.9216	0.9720	0.5421	0.7432	0.7667
NARM	0.3137	0.8808	0.9463	0.3839	0.8339	0.9060	0.1200	0.9580	0.9893	0.5233	0.7756	0.7953
ESMM$^-$	0.3224	0.8722	0.9414	0.3959	0.8268	0.9008	0.2402	0.9110	0.9655	0.5334	0.7456	0.7698
ESMM	0.3150	0.8776	0.9448	0.4035	0.8262	0.8991	0.1241	0.9564	0.9887	0.5175	0.7753	0.7985
DIEN$^-$	0.3198	0.8723	0.9401	0.4018	0.8388	0.9091	0.2254	0.9183	0.9698	0.5252	0.7657	0.7854
DIEN	0.3291	0.8646	0.9395	0.3911	0.8308	0.9022	0.1242	0.9563	0.9887	0.5145	0.7843	0.8046
TWINS$_i^-$	0.2746	0.8879	0.9538	0.3685	0.8452	0.9174	0.1246	0.9616	0.9893	0.5012	0.7896	0.8010
TWINS$_a^-$	0.2608	0.8948	0.9583	0.3660	0.8446	0.9170	0.1233	0.9622	0.9895	0.4977	0.7920	0.8024
TWINS	**0.2603***	**0.9120***	**0.9659***	**0.3528***	**0.8501***	**0.9235***	**0.1081***	**0.9631***	**0.9913***	**0.4855***	**0.7934***	**0.8187***
TWINS$^+_{co}$	**0.2596***	**0.8962***	**0.9593***	**0.3579***	**0.8458***	**0.9194***	**0.1170***	**0.9612***	**0.9903***	**0.4731***	**0.8045***	**0.8205***



[Jin, et al. WWW 2022b] Jiarui Jin, et al. Who to Watch Next: Two-side Interactive Networks for Live Broadcast Recommendation. WWW 2020.

Online A/B Test of our TWINS Model

- We conduct A/B testing comparing the proposed model TWINS$^{+}_{\text{co}}$ with the current production method. The whole experiment lasts a week from September 25, 2021 to October 2, 2021.
- Online experiments on Diantao App show that TWINS gains average performance improvement around 8% on ACTR metric, 3% on UCTR metric, 3.5% on UCVR metric.

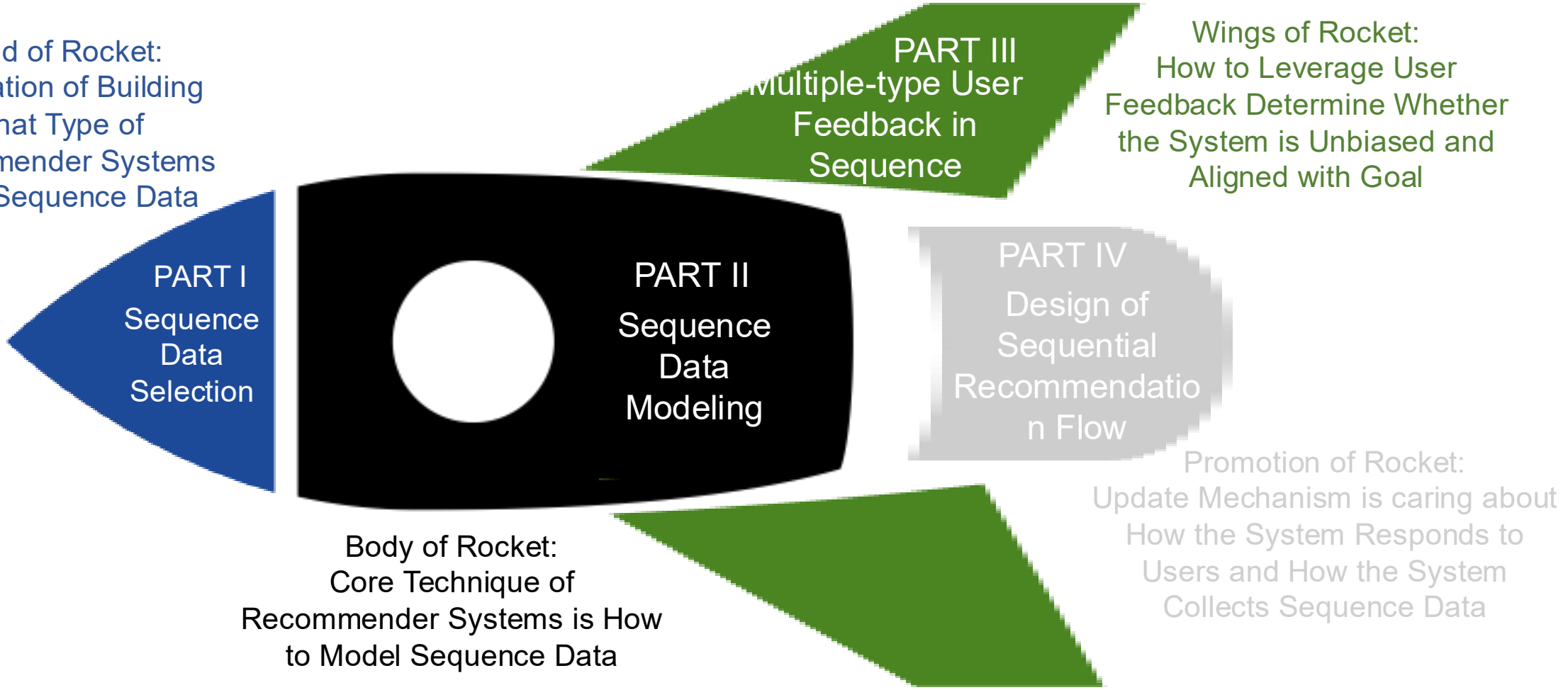
Recommender	ACTR	UCTR	UCVR
TWINS	**8.11%**	**2.01%**	**3.52%**

- ACTR is CTR metric from the anchor aspect defined as #clicks on anchors / #impressions on anchors.
- UCTR is CTR metric from the user aspect defined as #clicks on users / #impressions on users.
- UCVR is CVR metric from the user aspect defined as #conversions on users / # impressions on users.

[Jin, et al. WWW 2022b] Jiarui Jin, et al. Who to Watch Next: Two-side Interactive Networks for Live Broadcast Recommendation. WWW 2020.

Content

Data Collection → Data Selection → Architecture Design → Learning Objective



PART III: Multiple-type User Feedback in Sequence

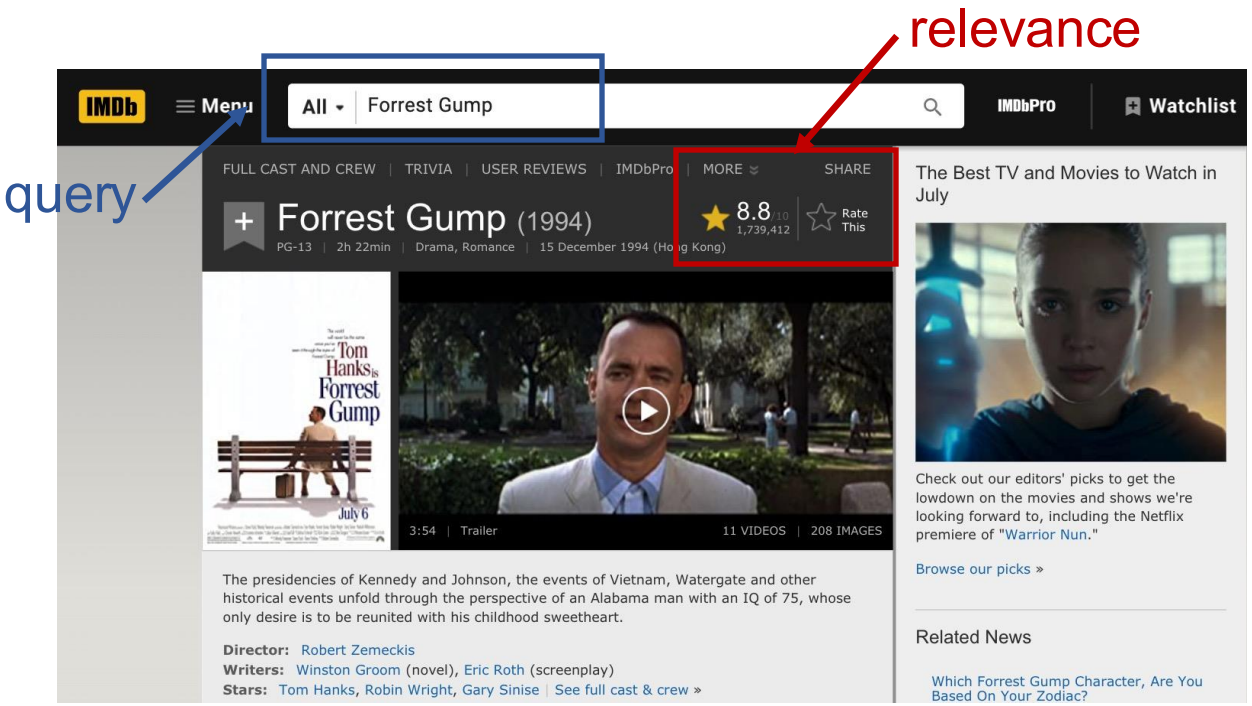
Does Directly Use Click Signals to Supervise Model a Good Choice?

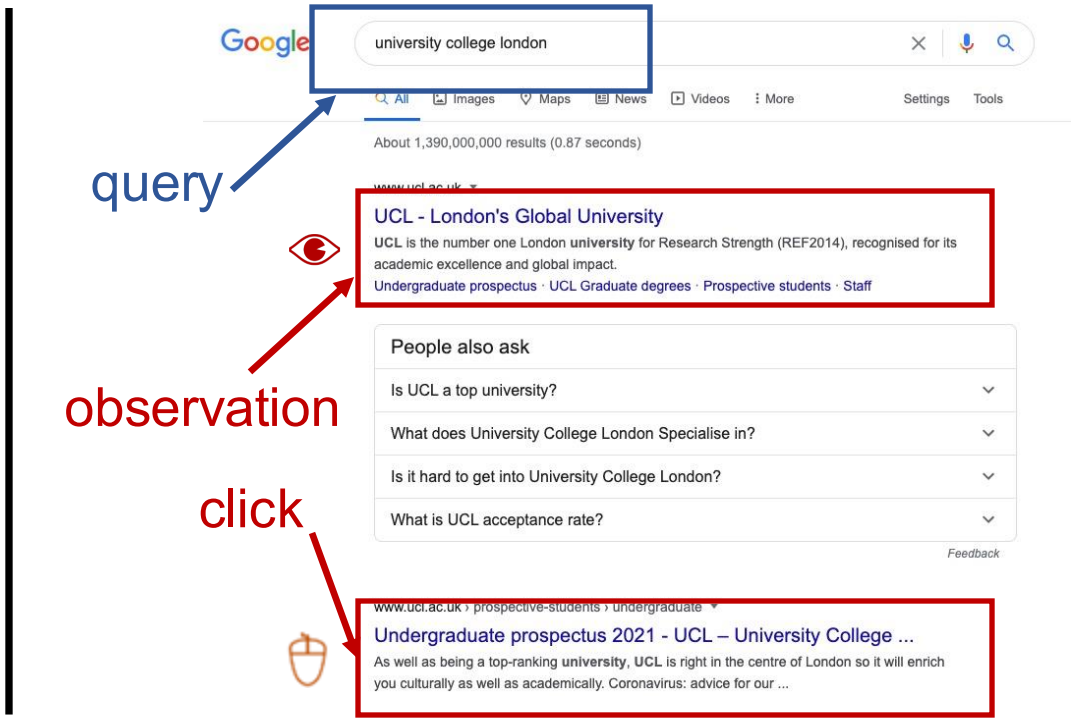
Maybe We Re-consider Usage of User Feedback

Use Them as Input or Consider Casual Dependence among Them

Explicit Feedback vs. Implicit Feedback

- The left figure is an example of an IMDB movie showing explicit feedback; whereas the right figure is an example of a Google search engine showing implicit feedback.





[Jin, et al. SIGIR 2020] Jiarui Jin, et al. A Deep Recurrent Survival Model for Unbiased Ranking. SIGIR 2020.

[Jin, et al. CIKM 2022] Jiarui Jin, et al. Multi-Scale User Behavior Network for Entire Space Multi-Task Learning. CIKM 2022.

From Observation to Click

- Position Bias indicates that users typically peruse presented item lists from top to bottom, with their attention diminishing rapidly along the way. Consequently, higher-ranked items receive more exposure and greater opportunities for observation and subsequent clicks.
- (Biased) learning-to-rank trains a ranker to directly estimate the Click probability, whereas unbiased learning-to-rank derives the Relevance probability from the click data according to the fact that a user clicks on an item only when it has been both observed and perceived as relevant.

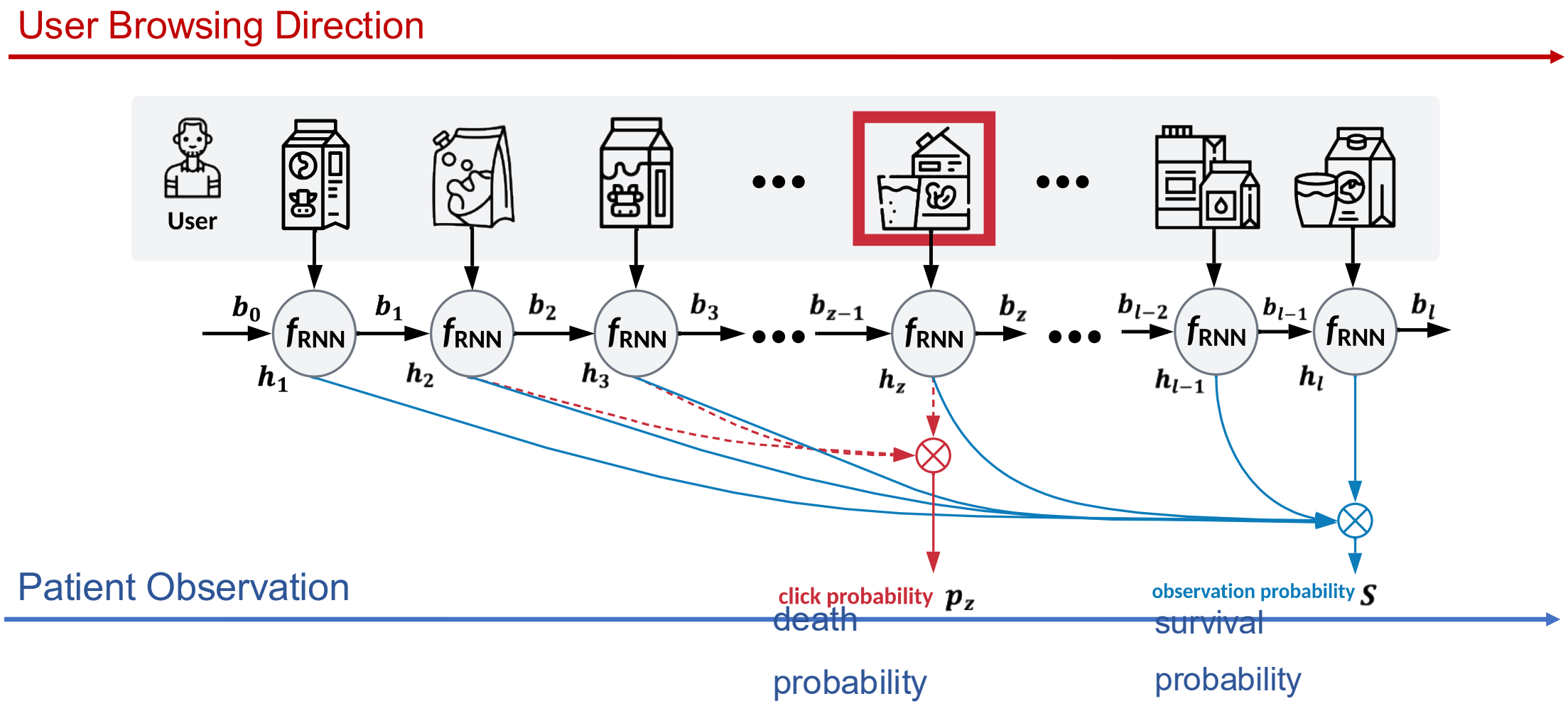
$$P(C = 1|X = x) = P(R = 1|X = x) \cdot P(O = 1|X = x)$$

- Motivation of Unbiased Learning-to-Rank is that blindly optimizing ranking performance based on implicit feedback data may advertently reinforce the existing presentation rather than learning personalized relevance.

[Jin, et al. SIGIR 2020] Jiarui Jin, et al. A Deep Recurrent Survival Model for Unbiased Ranking. SIGIR 2020.

[Jin, et al. CIKM 2022] Jiarui Jin, et al. Multi-Scale User Behavior Network for Entire Space Multi-Task Learning. CIKM 2022.

Overview of our DRSR Model



[Jin, et al. SIGIR 2020] Jiarui Jin, et al. A Deep Recurrent Survival Model for Unbiased Ranking. SIGIR 2020.

[Jin, et al. CIKM 2022] Jiarui Jin, et al. Multi-Scale User Behavior Network for Entire Space Multi-Task Learning. CIKM 2022.

Survival Model in Sequences

- Click probability (Death probability) is the Probability density function (PDF) of click occurring at i-th item, which can be defined as (z denotes the position of click event):

$$p_i = P(c_i = 1) = P(z = i)$$

- Observation probability (Survival probability) is the Cumulative Distribution Function (CDF) of click occurring at i-th item, since a user will keep browsing until she finds and clicks a favored one:

$$S_i = P(o_i = 1) = \sum_{\tau \geq i} P(z = \tau)$$

  - Relevance probability is the Conditional Click Probability, the click probability at an item given the item is observed:

$$h_i = P(r_i = 1) = \frac{P(c_i = 1)}{P(o_i = 1)} = \frac{P(z = i)}{P(z \geq i)} = \frac{p_i}{S_i}$$

[Jin, et al. SIGIR 2020] Jiarui Jin, et al. A Deep Recurrent Survival Model for Unbiased Ranking. SIGIR 2020.

[Jin, et al. CIKM 2022] Jiarui Jin, et al. Multi-Scale User Behavior Network for Entire Space Multi-Task Learning. CIKM 2022.

Survival Model in Sequences

- We apply Recurrent Neural Networks (RNNs) to model relevance probability as:

$$h_i = P(z = i | z \geq i, x) = f_{\text{RNN}}\left(x_i \,|b_{i-1}\right)$$

- Observation probability (survival probability) can be derived as:

$$S_i = P(z \geq i | x) = P(z \neq 1, z \neq 2, \dots, z \neq i-1 | x)$$

$$= P(z \neq 1 | x_1) \cdot P(z \neq 2 | z \neq 1, x_2) \cdots P(z \neq i-1 | z \neq 1, z \neq 2, \dots, z \neq i-2, x_{i-1})$$

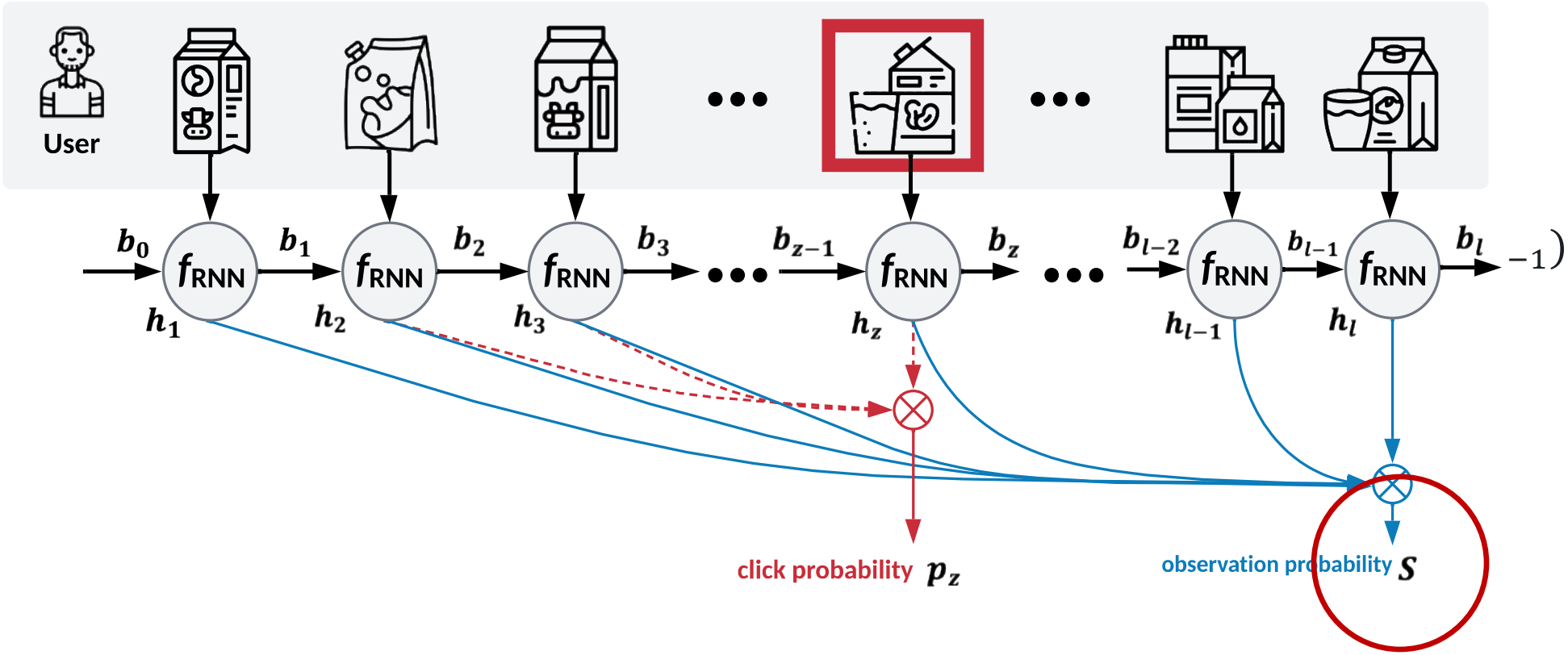
$$= \prod_{\tau:\tau<i} (1 - P(z = \tau | z \geq \tau, x_\tau)) = \prod_{\tau:\tau<i} (1 - h_\tau)$$

[Jin, et al. SIGIR 2020] Jiarui Jin, et al. A Deep Recurrent Survival Model for Unbiased Ranking. SIGIR 2020.

[Jin, et al. CIKM 2022] Jiarui Jin, et al. Multi-Scale User Behavior Network for Entire Space Multi-Task Learning. CIKM 2022.

Survival Model in Sequences

- We apply Recurrent Neural Networks (RNNs) to model relevance probability as:
- O



[Jin, et al. SIGIR 2020] Jiarui Jin, et al. A Deep Recurrent Survival Model for Unbiased Ranking. SIGIR 2020.

[Jin, et al. CIKM 2022] Jiarui Jin, et al. Multi-Scale User Behavior Network for Entire Space Multi-Task Learning. CIKM 2022.

Survival Model in Sequences

- We apply Recurrent Neural Networks (RNNs) to model relevance probability as:

$$h_i = P(z = i | z \geq i, x) = f_{\text{RNN}}(x_i | b_{i-1})$$

- Observation probability (survival probability) can be derived as:

$$S_i = P(z \geq i | x) = P(z \neq 1, z \neq 2, \dots, z \neq i-1 | x)$$

$$= P(z \neq 1 | x_1) \cdot P(z \neq 2 | z \neq 1, x_2) \cdots P(z \neq i-1 | z \neq 1, z \neq 2, \dots, z \neq i-2, x_{i-1})$$

$$= \prod_{\tau:\tau<i} (1 - P(z = \tau | z \geq \tau, x_\tau)) = \prod_{\tau:\tau<i} (1 - h_\tau)$$

- Click probability (death probability) can be derived as:

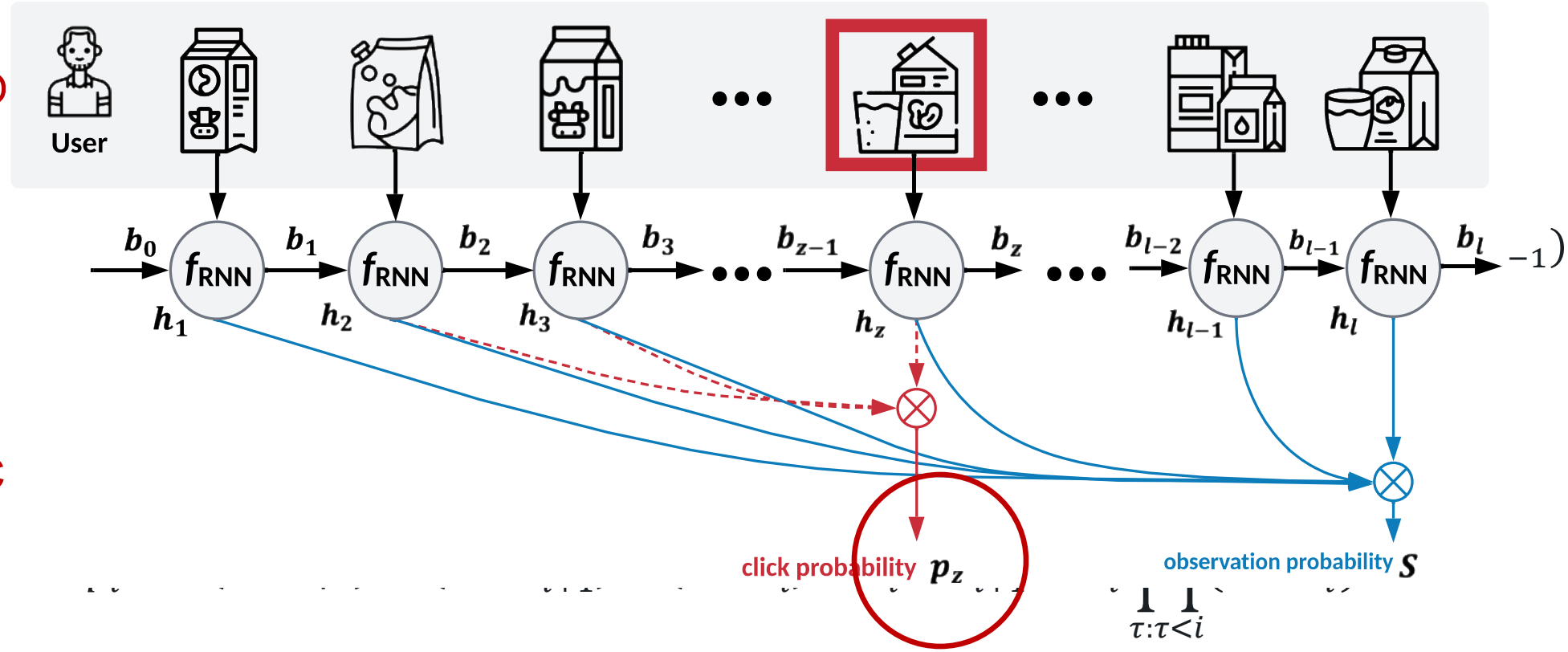
$$p_i = P(z = i | x) = (1 - S_{i+1}) - (1 - S_i) = S_i - S_{i+1} = h_i \prod_{\tau:\tau<i} (1 - h_\tau)$$

[Jin, et al. SIGIR 2020] Jiarui Jin, et al. A Deep Recurrent Survival Model for Unbiased Ranking. SIGIR 2020.

[Jin, et al. CIKM 2022] Jiarui Jin, et al. Multi-Scale User Behavior Network for Entire Space Multi-Task Learning. CIKM 2022.

Survival Model in Sequences

- We apply Recurrent Neural Networks (RNNs) to model relevance probability as:
  - O
  - C



[Jin, et al. SIGIR 2020] Jiarui Jin, et al. A Deep Recurrent Survival Model for Unbiased Ranking. SIGIR 2020.

[Jin, et al. CIKM 2022] Jiarui Jin, et al. Multi-Scale User Behavior Network for Entire Space Multi-Task Learning. CIKM 2022.

Unbiased Objective Function

- Then, we can supervise the whole model by using the click data:

$$\mathcal{L} = -\log \prod_{(x,j)\in\mathcal{D}_{\text{CLICK}}} P(z=j|x) = -\log \prod_{(x,j)\in\mathcal{D}_{\text{CLICK}}} p_j$$

$$= -\log \prod_{(x,j)\in\mathcal{D}_{\text{CLICK}}} \left(h_j \prod_{\tau:\tau<i} (1-h_\tau) \right)$$

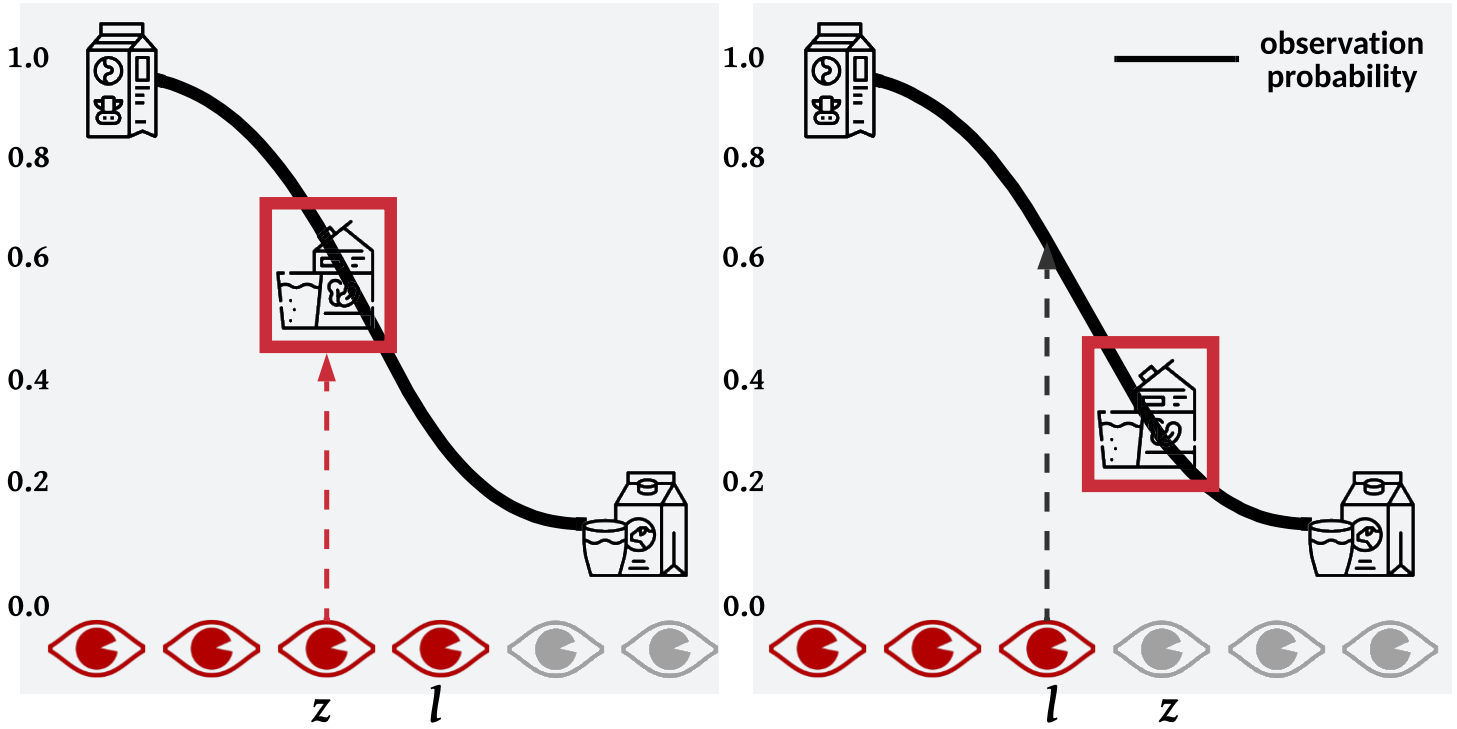
$$= -\sum_{(x,j)\in\mathcal{D}_{\text{CLICK}}} \left(\log h_j + \sum_{\tau:\tau<i} \log(1-h_\tau) \right)$$

[Jin, et al. SIGIR 2020] Jiarui Jin, et al. A Deep Recurrent Survival Model for Unbiased Ranking. SIGIR 2020.

[Jin, et al. CIKM 2022] Jiarui Jin, et al. Multi-Scale User Behavior Network for Entire Space Multi-Task Learning. CIKM 2022.

CDF vs. PDF

- Supervising observation probability pushes up the observation probability of items whose position in range of $[0, l]$ and pulls down in range of $[l, +\infty)$.
- This allows the DRSR model to leverage the sequence data without clicks as supervisions.



[Jin, et al. SIGIR 2020] Jiarui Jin, et al. A Deep Recurrent Survival Model for Unbiased Ranking. SIGIR 2020.

[Jin, et al. CIKM 2022] Jiarui Jin, et al. Multi-Scale User Behavior Network for Entire Space Multi-Task Learning. CIKM 2022.

Unbiased Objective Function

- For the sequence data with clicks, we can use observation probability as supervisions following:

$$\mathcal{L} = -\log \prod_{(x,l)\in\mathcal{D}_{\text{CLICK}}} P(z \le l|x) \approx -\sum_{(x,l)\in\mathcal{D}_{\text{CLICK}}} \log\left(1 - \prod_{\tau:\tau<l}(1-h_\tau)\right)$$

- For the sequence data without clicks, we can use observation probability as supervisions following:

$$\mathcal{L} = -\log \prod_{(x,l)\in\mathcal{D}_{\text{NONCLICK}}} P(z > l|x) \approx -\sum_{(x,l)\in\mathcal{D}_{\text{NONCLICK}}} \sum_{\tau:\tau<l} \log(1-h_\tau)$$

[Jin, et al. SIGIR 2020] Jiarui Jin, et al. A Deep Recurrent Survival Model for Unbiased Ranking. SIGIR 2020.

[Jin, et al. CIKM 2022] Jiarui Jin, et al. Multi-Scale User Behavior Network for Entire Space Multi-Task Learning. CIKM 2022.

Offline Evaluation of our DRSR Model

- Unbiased learning-to-rank algorithms always estimate click probability $P(C = 1|x)$ during training phase, as we only have access to click data; whereas they derives relevance probability $P(R = 1|x)$ for inference, because relevance estimation is free of bias.
- Evaluation of unbiased learning-to-rank algorithms regards the raw data (e.g., click data) as the unbiased relevance score; and then generate (new) click data by employing a click model.
- Position Based Model (PBM) is a click model that generates clicks based on the assumption that the bias of an item only depends on its position, which can be formulated as $P(o_i) = \rho_i^\tau$. ρ_i represents position bias at position i and τ is a parameter controlling the degree of position bias. ρ_i is obtained by an eye-tracking experiment in [Joachims et al. SIGIR 2005], and τ is set as 1.
- Cascade Click Model (CCM) is a click model, which assumes that the user browses the search results from top to bottom. The user browse behaviors are both conditioned on current and past items as $P(c_i = 1|o_i = 0) = 0$, $P(c_i = 1|o_i = 1, r_i) = P(r_i)$, $P(o_{i+1} = 1|o_i = 0) = 0, P(o_{i+1} = 1|o_i = 1, c_i = 0) = \gamma_1, P(o_{i+1} = 1|o_i = 1, c_i = 1, r_i) = \gamma_2 \cdot (1 - P(r_i)) + \gamma_3 \cdot P(r_i)$. The parameters γs are obtained from [Guo et al. WWW 2009].

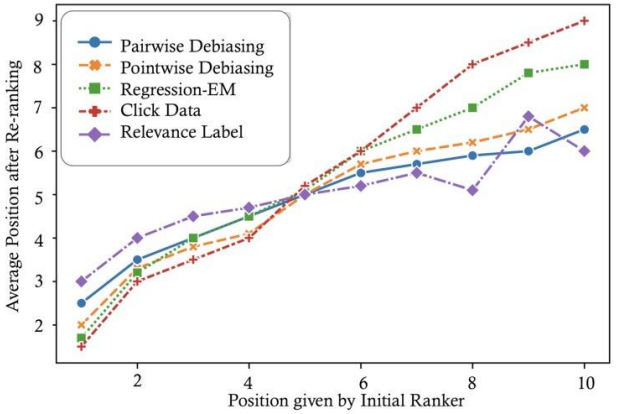
[Jin, et al. SIGIR 2020] Jiarui Jin, et al. A Deep Recurrent Survival Model for Unbiased Ranking. SIGIR 2020.

[Jin, et al. CIKM 2022] Jiarui Jin, et al. Multi-Scale User Behavior Network for Entire Space Multi-Task Learning. CIKM 2022.

[Joachims et al. SIGIR 2005] Thorsten Joachims et al. Accurately Interpreting Click-Through Data as Implicit Feedback. SIGIR 2005.

[Guo et al. WWW 2009] Fan Guo et al. Click Chain Model in Web Search. WWW 2009.

Offline Evaluation of our DRSR Model



Average position after re-ranking of items at each original position by different debiasing methods combined with the DRSR model.

Our method is point-wise debiasing method (pair-wise debiasing method is also a variant of the DRSR model that we do not mention here).

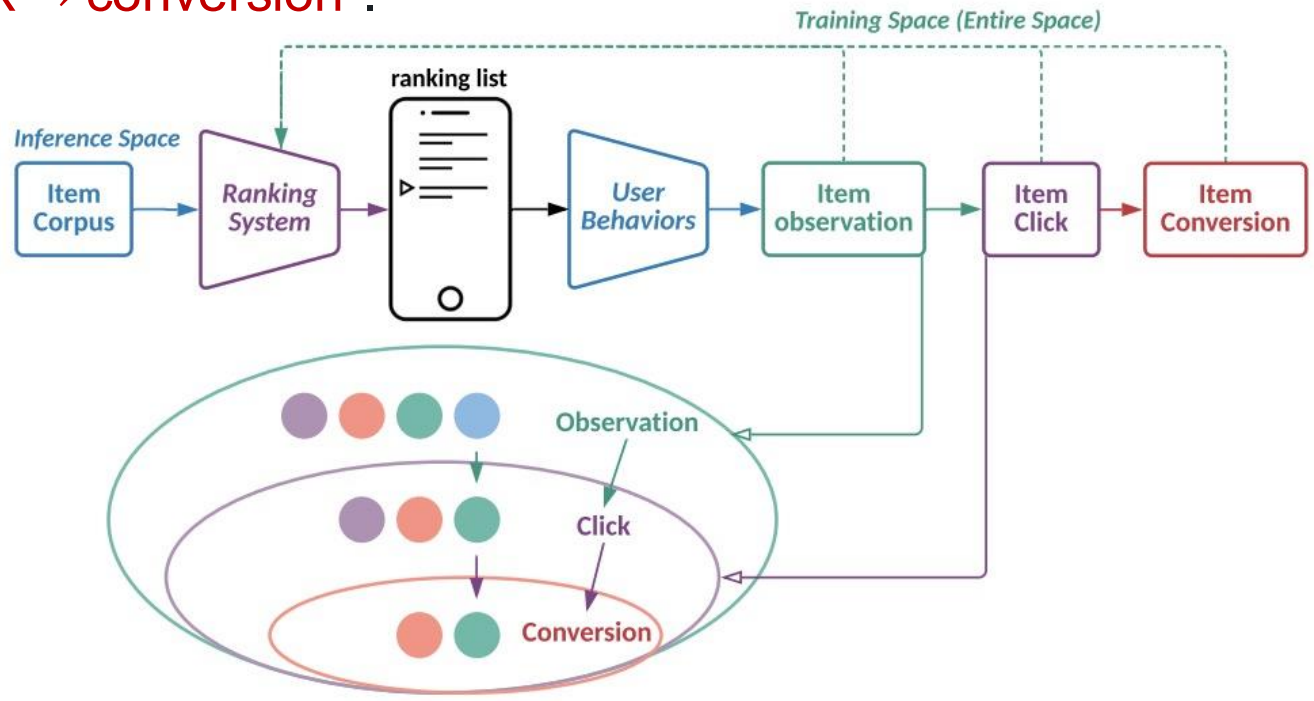
Ranker	Debiasing Method	Yahoo Search Engine (CCM)				Alibaba Recommender System (CCM)				Yahoo Search Engine (PBM)			
		MAP	NDCG@1	NDCG@3	NDCG@5	MAP	NDCG@1	NDCG@3	NDCG@5	MAP	NDCG@1	NDCG@3	NDCG@5
DRSR (Ours)	Labeled Data	0.861	0.747	0.759	0.771	0.850	0.737	0.741	0.755	0.861	0.747	0.759	0.771
	Pairwise Debiasing	**0.842***	**0.719***	**0.721***	**0.737***	**0.831***	**0.684***	**0.685***	**0.707***	**0.848***	**0.726***	**0.737***	**0.745***
	Pointwise Debiasing	**0.839***	**0.713***	**0.717***	**0.730***	**0.830***	**0.682***	**0.684***	**0.706***	**0.843***	**0.723***	**0.731***	**0.740***
	Regression-EM [43]	0.829	0.679	0.685	0.701	0.820	0.657	0.668	0.673	0.834	0.698	0.705	0.712
	Click Data	0.817	0.636	0.652	0.667	0.810	0.613	0.627	0.658	0.825	0.671	0.679	0.693
LambdaMART	Labeled Data	0.854	0.745	0.745	0.757	0.847	0.729	0.732	0.743	0.854	0.745	0.745	0.757
	Ratio Debiasing [17]	0.830	0.688	0.685	0.699	0.821	0.661	0.669	0.674	0.836	0.717	0.716	0.728
	Regression-EM [43]	0.826	0.669	0.676	0.691	0.818	0.636	0.651	0.667	0.830	0.685	0.684	0.700
	Click Data	0.813	0.628	0.646	0.673	0.804	0.603	0.618	0.646	0.820	0.658	0.669	0.672
DNN	Labeled Data	0.831	0.677	0.685	0.705	0.824	0.674	0.679	0.693	0.831	0.677	0.685	0.705
	Dual Learning Algorithm [2]	0.825	0.672	0.678	0.691	0.814	0.629	0.647	0.674	0.828	0.674	0.683	0.697
	Regression-EM [43]	0.823	0.665	0.669	0.687	0.813	0.628	0.645	0.672	0.829	0.676	0.684	0.699
	Click Data	0.809	0.611	0.619	0.648	0.801	0.600	0.612	0.641	0.819	0.637	0.651	0.667

[Jin, et al. SIGIR 2020] Jiarui Jin, et al. A Deep Recurrent Survival Model for Unbiased Ranking. SIGIR 2020.

[Jin, et al. CIKM 2022] Jiarui Jin, et al. Multi-Scale User Behavior Network for Entire Space Multi-Task Learning. CIKM 2022.

Combing Observation to Click and Click to Conversion

- The top diagram shows the pipeline of information systems, which consists of multiple user behaviors to the ranking system, and the bottom figure shows the entire space behavior path "observation→click → conversion".

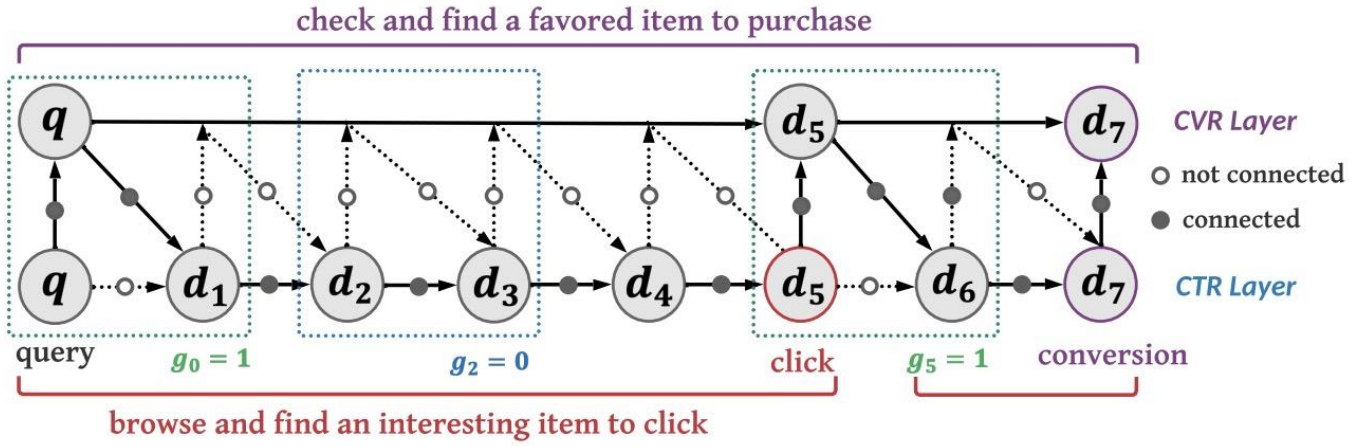


[Jin, et al. SIGIR 2020] Jiarui Jin, et al. A Deep Recurrent Survival Model for Unbiased Ranking. SIGIR 2020.

[Jin, et al. CIKM 2022] Jiarui Jin, et al. Multi-Scale User Behavior Network for Entire Space Multi-Task Learning. CIKM 2022.

Overview of our HEROES Model

- Our idea is to build a hierarchical architecture, where in the lower layer (i.e., CTR layer) for engagement objective modeling, we model through the behavior path "observation→click"; while in the upper layer (i.e., CVR layer) for satisfaction objective modeling, we model through the behavior path "click → conversion".



- To bridge these two layers, we introduce a gate mechanism, where we define g_i as a boundary detector:

$$g_i = \begin{cases} 1 & \text{if } P(c_i = 1) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

[Jin, et al. SIGIR 2020] Jiarui Jin, et al. A Deep Recurrent Survival Model for Unbiased Ranking. SIGIR 2020.

[Jin, et al. CIKM 2022] Jiarui Jin, et al. Multi-Scale User Behavior Network for Entire Space Multi-Task Learning. CIKM 2022.

Intra-Layer Mechanism in Sequences

- Let s_i denote the unit states in each cell (s_i^c for CTR layer and s_i^v for CVR layer). In each cell, we can build a hidden state h_i (correspondingly h_i^c for CTR layer and h_i^v for CVR layer). These unit states are recursively updated as:

$$s_i^c = \begin{cases} \mathrm{f}_i^c \odot s_{i-1}^c + \mathrm{i}_i^c \odot \mathrm{g}_i^c & \text{UPDATE} \\ \mathrm{i}_i^c \odot \mathrm{g}_i^c & \text{SUMMARIZE} \end{cases} \qquad s_i^v = \begin{cases} \mathrm{f}_i^v \odot s_{i-1}^v + \mathrm{i}_i^v \odot \mathrm{g}_i^v & \text{UPDATE} \\ s_{i-1}^v & \text{COPY} \end{cases}$$

- UPDATE is $g_{i-1} = 0$, and both SUMMARIZE and COPY are $g_{i-1} = 1$. Here, $\mathrm{f}_i, \mathrm{i}_i, \mathrm{o}_i$ are forget, input, output gates and g_i is a cell proposal vector which will be determined by the inter-layer mechanism (which will be introduced later). The corresponding hidden states are computed as:

$$h_i^c = \mathrm{o}_i^c \odot \tanh(s_i^c) \qquad h_i^v = \begin{cases} \mathrm{o}_i^v \odot \tanh(s_i^v) & \text{UPDATE} \\ h_{i-1}^v & \text{COPY} \end{cases}$$

[Jin, et al. SIGIR 2020] Jiarui Jin, et al. A Deep Recurrent Survival Model for Unbiased Ranking. SIGIR 2020.

[Jin, et al. CIKM 2022] Jiarui Jin, et al. Multi-Scale User Behavior Network for Entire Space Multi-Task Learning. CIKM 2022.

Inter-Layer Mechanism in Sequences

- $\mathrm{f}_i, \mathrm{i}_i, \mathrm{o}_i, \mathrm{g}_i$ are designed to encode the top-down contextual information as:

$$\mathrm{f}_i = \mathrm{i}_i = \mathrm{o}_i = \sigma(f_{\mathrm{MLP}}(\mathrm{s}_i))$$

$$\mathrm{g}_i = \tanh(f_{\mathrm{MLP}}(s_i))$$

- s_i is the top-down state, computed as:

$$\mathrm{s}_i^c = (1 - g_{i-1}) \cdot U_{i-1}^c \cdot h_{i-1}^c + g_{i-1} \cdot U_{i-1}^r \cdot h_{i-1}^v$$

$$\mathrm{s}_i^v = U_{i-1}^v \cdot h_{i-1}^v + g_i \cdot W_i^r \cdot h_i^c$$

where U_i^c, U_{i-1}^r, U_{i-1}^v, W_i^r are trainable weights.

- Our HEROES model forces the CVR layer to absorb the summary information from the CTR layer according to the top-down contexts.

[Jin, et al. SIGIR 2020] Jiarui Jin, et al. A Deep Recurrent Survival Model for Unbiased Ranking. SIGIR 2020.

[Jin, et al. CIKM 2022] Jiarui Jin, et al. Multi-Scale User Behavior Network for Entire Space Multi-Task Learning. CIKM 2022.

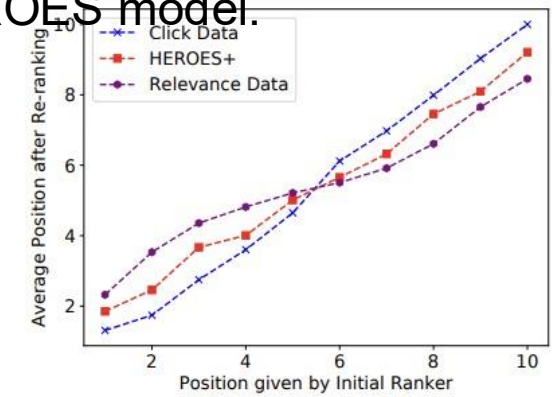
Offline Evaluation of our HEROES Model

Ranker	Task	Criteo			Taobao E-Commerce			Diantao Live Broadcast		
		AUC	LogLoss	NDCG	AUC	LogLoss	NDCG	AUC	LogLoss	NDCG
DUPN	CVR	0.9505	0.1137	0.7348	0.6747	0.5194	0.6843	0.8232	0.2345	0.7522
	CTR	0.7410	0.5863	0.7526	0.5777	0.7215	0.4576	0.7156	0.6032	0.7009
ESMM	CVR	0.8750	0.4466	0.7194	0.6443	0.6330	0.6490	0.7046	0.2743	0.6697
	CTR	0.6476	0.6511	0.7460	0.5410	0.7591	0.4166	0.6664	0.6577	0.6601
ESM^2	CVR	0.8798	0.4360	0.7235	0.6453	0.6376	0.6471	0.7039	0.2756	0.6688
	CTR	0.6740	0.6370	0.7496	0.5437	0.7573	0.4170	0.6742	0.6512	0.6608
MMoE	CVR	0.8817	0.4420	0.7182	0.6537	0.6267	0.6452	0.7283	0.2731	0.6653
	CTR	0.6779	0.6343	0.7540	0.5410	0.7463	0.4093	0.6770	0.6513	0.6618
DRSR	CVR	0.9468	0.1366	0.7644	0.6723	0.5156	0.6892	0.8140	0.2546	0.7697
	CTR	0.7452	0.5837	0.7687	0.5759	0.7171	0.4578	0.6985	0.6103	0.7053
RRN	CVR	0.9564	0.1169	0.7739	0.6732	0.5061	0.6890	0.8156	0.2698	0.7421
	CTR	0.7496	0.5797	0.7706	0.5766	0.7075	0.4575	0.6926	0.6019	0.6928
NARM	CVR	0.9524	0.1172	0.7644	0.6733	0.5160	0.6893	0.8234	0.2595	0.7612
	CTR	0.7511	0.5810	0.7724	0.5764	0.7186	0.4576	0.7082	0.5958	0.7012
STAMP	CVR	0.9406	0.1209	0.8014	0.6668	0.5210	0.6892	0.8467	0.2465	0.7689
	CTR	0.7391	0.5929	0.7702	0.5748	0.7235	0.4575	0.7123	0.5940	0.7070
Time-LSTM	CVR	0.9622	0.1132	0.7979	0.6745	0.5169	0.6889	0.8540	0.2412	0.7787
	CTR	0.7602	0.5703	0.7738	0.5776	0.7192	0.4576	0.7195	0.6040	0.7124
LSTM	CVR	0.8429	0.4841	0.6629	0.6721	0.4783	0.6885	0.7124	0.2736	0.7475
	CTR	0.6032	0.6042	0.7503	0.5749	0.7222	0.4493	0.6633	0.6542	0.6792
NHP	CVR	0.9533	0.1127	0.7682	0.6743	0.4914	0.6893	0.8267	0.2535	0.7622
	CTR	0.7428	0.5816	0.7656	0.5773	0.7214	0.4576	0.7033	0.6042	0.7068
$HEROES^{-}_{intra}$	CVR	0.8801	0.4270	0.7327	0.6917	0.5209	0.6998	0.8045	0.2675	0.7712
	CTR	0.6764	0.6612	0.7521	0.5483	0.7174	0.4682	0.7091	0.5976	0.7135
$HEROES^{-}_{inter}$	CVR	0.9682	0.1152	0.7832	0.6932	0.4918	0.7082	0.8346	0.2225	0.7883
	CTR	0.7632	0.5721	0.7882	0.5927	0.7032	0.4721	0.7138	0.6021	0.7123
$HEROES^{-}_{unit}$	CVR	0.9705	0.1016	0.8348	0.7402	0.4366	0.7106	0.8601	0.2350	0.7810
	CTR	0.7787	0.5483	0.7832	0.5920	0.7084	0.4701	0.7412	0.5942	0.7111
HEROES	CVR	**0.9759***	**0.0975***	**0.8551***	**0.7503***	**0.3519***	**0.7137***	**0.8649***	**0.2203***	**0.7893***
	CTR	**0.7870***	**0.5400***	**0.7913***	**0.5953***	**0.7024***	**0.4727***	**0.7492***	**0.5893***	**0.7166***

Comparisons of unbiased ranking and biased ranking version of HEROES under click generation model PBM.

Ranker	Task	Taobao E-Commerce (PBM)		
		AUC	LogLoss	NDCG
Relevance Data (HEROES)	CVR	0.7503	0.3519	0.7137
	CTR	0.5953	0.7024	0.4727
$HEROES^{+}$	CVR	0.7442	0.3674	0.7064
	CTR	0.5735	0.7206	0.4567
$HEROES^{+}_{comb}$	CVR	0.7463	0.3638	0.7110
	CTR	0.5738	0.7202	0.4521
Click Data (HEROES)	CVR	0.7412	0.3746	0.7024
	CTR	0.5643	0.7563	0.4284

Average position after re-ranking of items at each original position by different debiasing methods combined with the HEROES model.

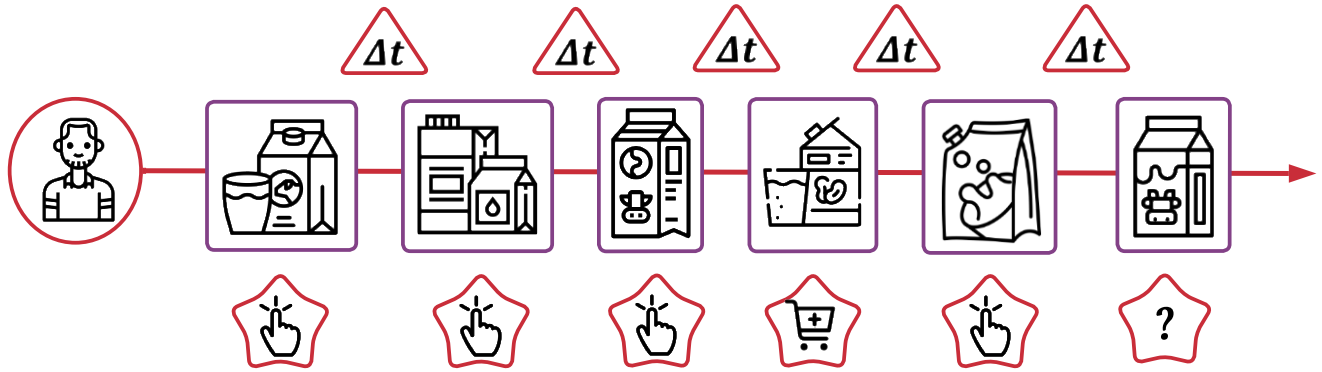


[Jin, et al. SIGIR 2020] Jiarui Jin, et al. A Deep Recurrent Survival Model for Unbiased Ranking. SIGIR 2020.

[Jin, et al. CIKM 2022] Jiarui Jin, et al. Multi-Scale User Behavior Network for Entire Space Multi-Task Learning. CIKM 2022.

Label Trick in Sequences

- Label Trick is to use the label data as the input. In the STARec model, for searched samples, we can concatenate the input feature and the corresponding user feedback (i.e., the label data) together as the input for the model.
- For discrete labels, we can build a look-up embedding table to represent each label (just like how we deal with categorical features).
- Single and Effective. It can significantly improve the recommendation performance of LSTM and STARec.

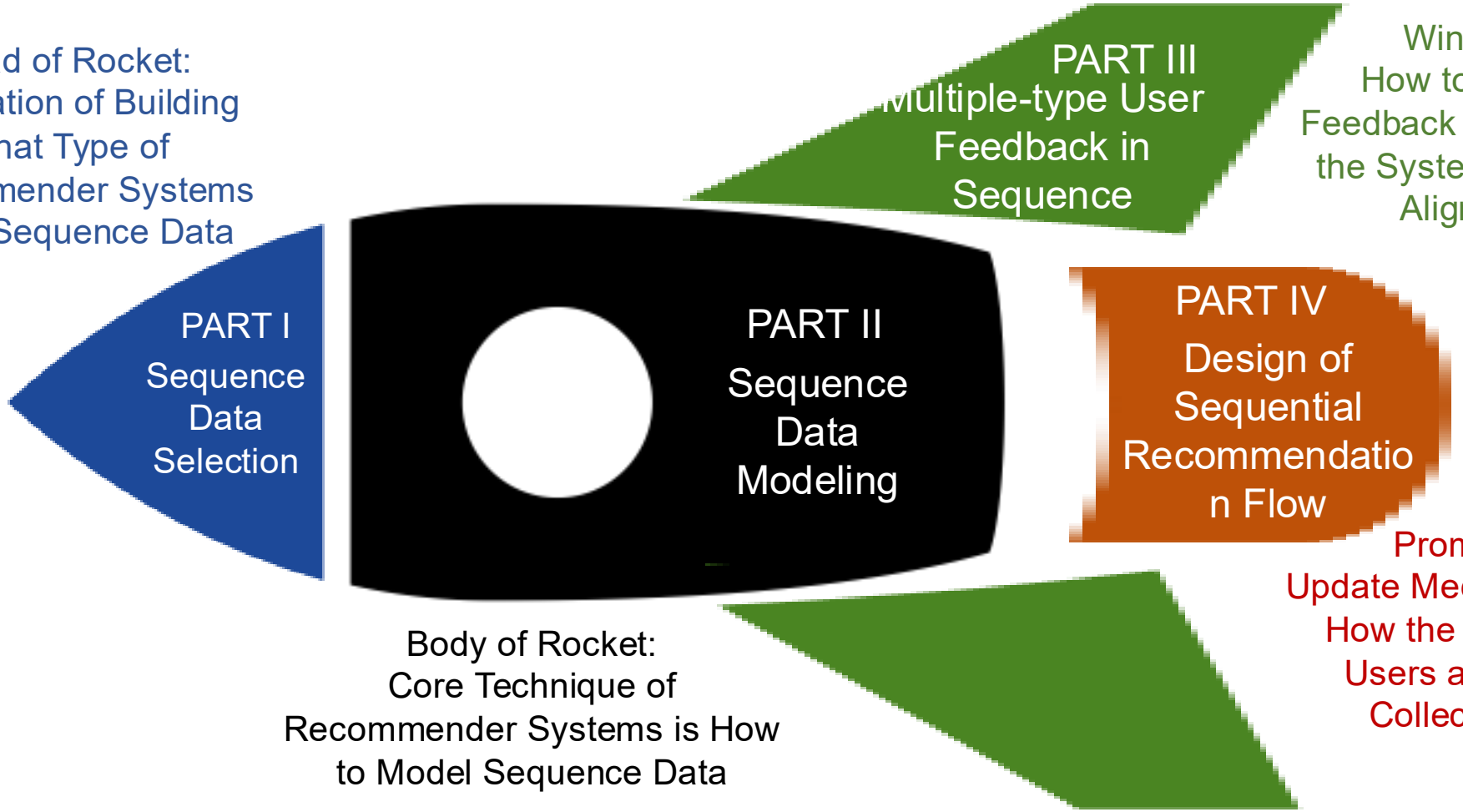


Recommender	Tmall			Alipay			Taobao		
	AUC	ACC	LogLoss	AUC	ACC	LogLoss	AUC	ACC	LogLoss
LSTM	0.6973	0.7054	0.5854	0.8357	0.7697	0.4713	0.5912	0.4516	0.4411
$\mathbf{LSTM}^{+}_{\text{label}}$	**0.7662**	**0.7324**	**0.5291**	**0.9052**	**0.8449**	**0.3738**	**0.6450**	**0.5780**	**0.4094**
STARec	**0.7204***	**0.7150***	**0.5471***	**0.8624***	**0.8142***	**0.4410***	**0.6126***	**0.4629***	**0.4211***
$\mathbf{STARec}^{+}_{\text{label}}$	**0.7986***	**0.7502***	**0.5059***	**0.9201***	**0.8661***	**0.3423***	**0.6771***	**0.6039***	**0.3862***

[Jin, et al. WWW 2022a] Jiarui Jin, et al. Learn over Past, Evolve for Future: Search-based Time-aware Recommendation with Sequential Behavior Data. WWW 2020.

Content

Data Collection → Data Selection → Architecture Design → Learning Objective → System Response



Head of Rocket:
Orientation of Building What Type of Recommender Systems using Sequence Data

Wings of Rocket:
How to Leverage User Feedback Determine Whether the System is Unbiased and Aligned with Goal

Body of Rocket:
Core Technique of Recommender Systems is How to Model Sequence Data

Promotion of Rocket:
Update Mechanism is caring about How the System Responds to Users and How the System Collects Sequence Data

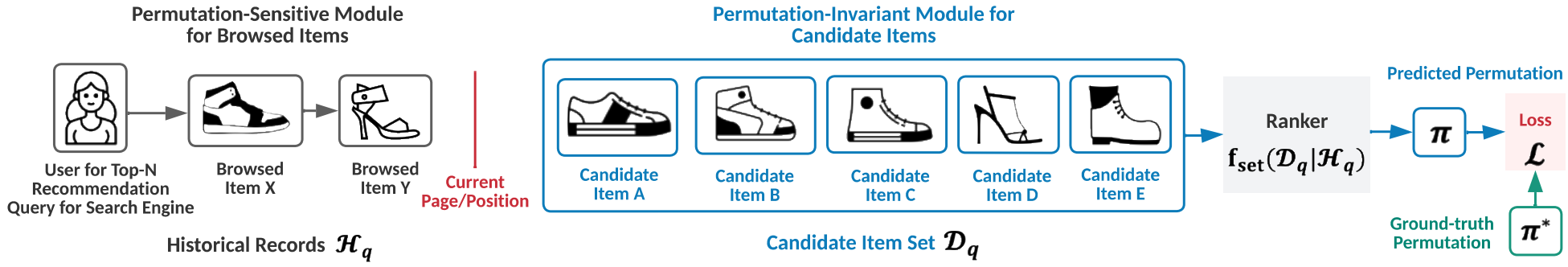
PART IV: Design of Sequential Recommendation Flow

Single-Column Flow or Double-Column Flow or Immersive Flow?

Maybe We Can Rolling Flow or Human-Communicating Flow

Masonry-Layout Recommender Systems

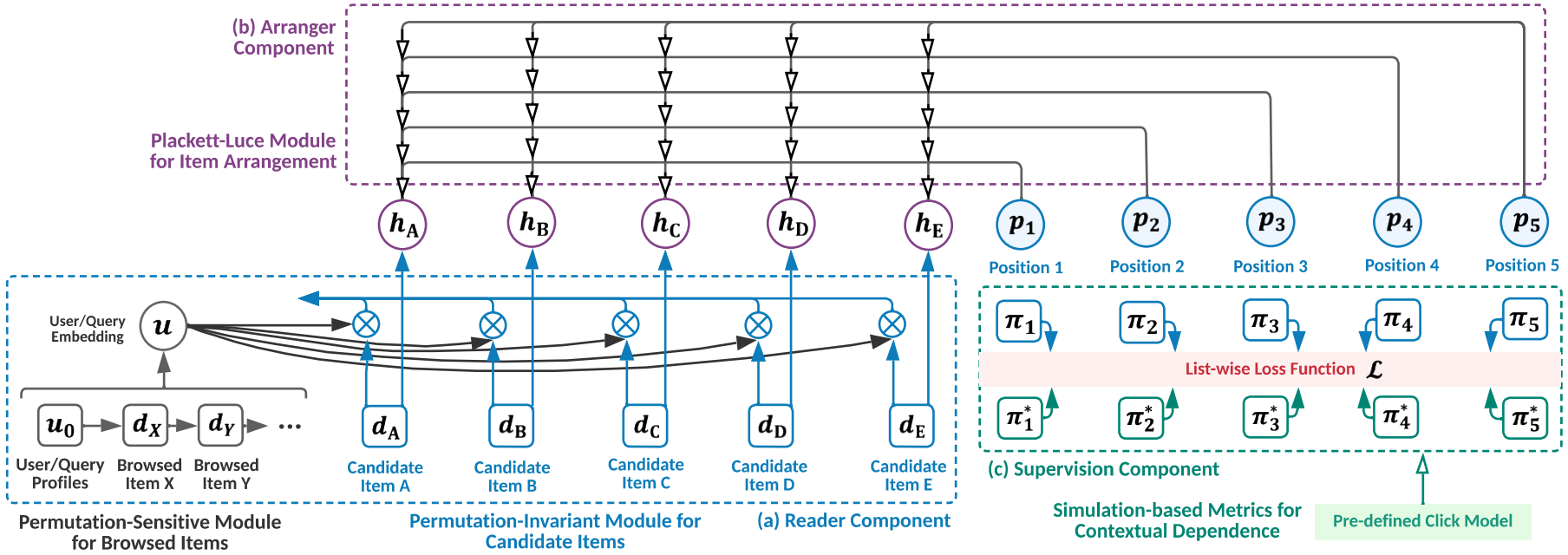
- Masonry Layout would post the recommended items to users page by page.
- Permutation Sensitive part is previous pages of items $\mathcal{H}_q$ (q is the query), because the context of user behaviors is sensitive to the posted items.
- Permutation Invariant part is the set of candidate items $\mathcal{D}_q$ for the current page.
- Our goal is to leverage $\mathcal{H}_q$ and $\mathcal{D}_q$ to generate an arrangement (i.e., permutation) of $\mathcal{D}_q$ (denoted as π).



[Jin, et al. CIKM 2023] Jiarui Jin, et al. Replace Scoring with Arrangement: A Contextual Set-to-Arrangement Framework for Learning-to-Rank. CIKM 2023.

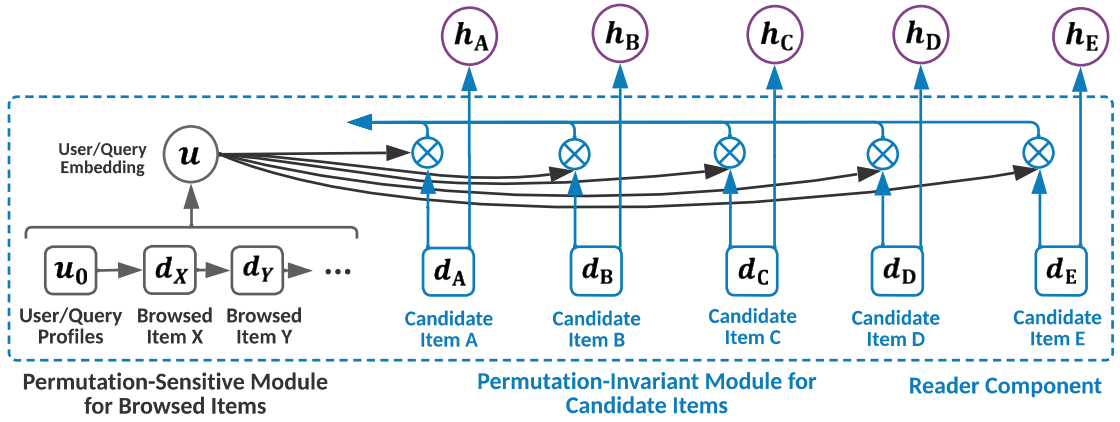
Overview of our STARank Model

The STARank model consists of the reader module (a) to deal with permutation sensitive and permutation invariant inputs), the arranger module (b) to generate the arrangement (i.e., permutation) of candidate items, and the supervision module (c) to optimize the whole model.



[Jin, et al. CIKM 2023] Jiarui Jin, et al. Replace Scoring with Arrangement: A Contextual Set-to-Arrangement Framework for Learning-to-Rank. CIKM 2023.

Reader in the STARank Model



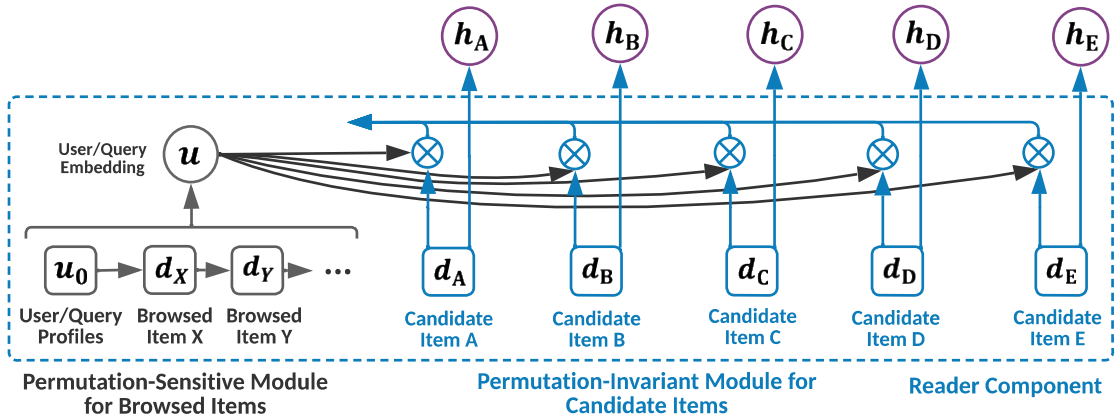
- We introduce a Permutation Sensitive (PS) module to represent the sequence data of user historical records, and the resulting user embedding can be written as:

$$u_q = \mathrm{PS}(u_0, x_1, \ldots, x_{|\mathcal{H}_q|})$$

where u_q is the embedding vector for user or query q. While practical, we apply a RNN (i.e., Long-Short Term Memory (LSTM)).

[Jin, et al. CIKM 2023] Jiarui Jin, et al. Replace Scoring with Arrangement: A Contextual Set-to-Arrangement Framework for Learning-to-Rank. CIKM 2023.

Reader in the STARank Model



- We introduce a Permutation Invariant (PI) module to encode the candidate items in $\mathcal{D}_q$:

$$\{h_d | d \in \mathcal{D}_q\} = \mathrm{PI}(\{x_d | d \in \mathcal{D}_q\}, u_q)$$

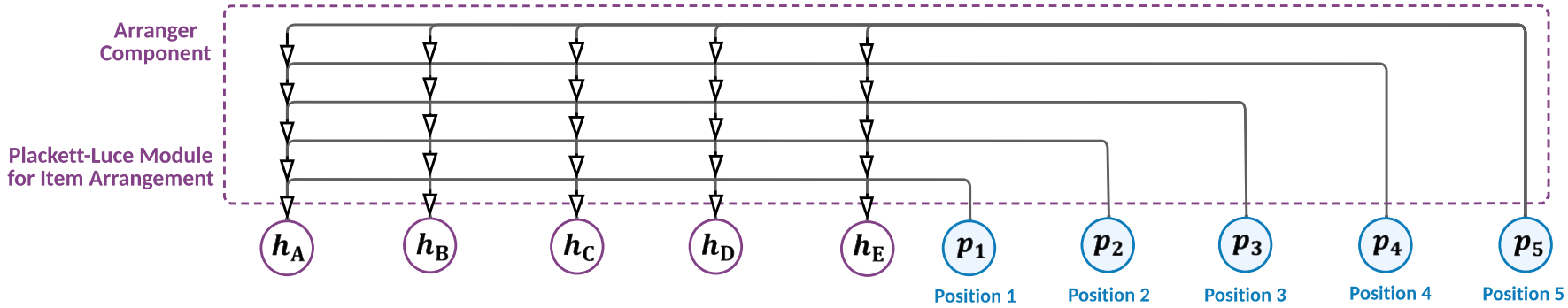
where h_d is the embedding vector for item d. In particular, our PI module is applying an attention mechanism (where w_1, W_1, b_1 are trainable matrices and vectors):

$$h'_d = w_1^{\mathrm{T}} \tanh(W_1 x_d + b_1)$$
$$h_d = \beta_d h'_d$$

$$\text{where } \beta_d = h'_d u_q^{\mathrm{T}} / \sum_{j \in \mathcal{D}_q} h' u_q^{\mathrm{T}}$$

[Jin, et al. CIKM 2023] Jiarui Jin, et al. Replace Scoring with Arrangement: A Contextual Set-to-Arrangement Framework for Learning-to-Rank. CIKM 2023.

Arranger in the STARank Model



- We use a Plackett-Luce (PL) module to generate the permutation of candidate items in $\mathcal{D}_q$. Formally, the PL module is defined as:

$$\pi = \mathrm{PL}(\{h_d | d \in \mathcal{D}_q\}, u_q)$$

- For this purpose, we first construct a batch of one-hot position embedding vectors $\{p_i\}_{i=1}^{|\mathcal{D}_q|}$ to encode the position information. This embeddings can be computed by applying a LSTM as the encoder.

[Jin, et al. CIKM 2023] Jiarui Jin, et al. Replace Scoring with Arrangement: A Contextual Set-to-Arrangement Framework for Learning-to-Rank. CIKM 2023.

Arranger in the STARank Model

- The probability distribution of item d at the i-th position can be produced as (where w_2, W_2, W_3, b_2 are trainable matrices and vectors):

$$s_d^i = v_d^i u_q^{\mathrm{T}} \text{ where } v_d^i = w_2^{\mathrm{T}} \tanh(W_2 h_d + W_3 p_i + b_2)$$

$$p_d^i = P(\pi_i = d | \pi_{<i}) = \begin{cases} e^{s_d^i} / \sum_{k \in \mathcal{D}_q \setminus \pi_{<i}} e^{s_k^i} & \text{if } d \notin \pi_{<i} \\ 0 & \text{otherwise} \end{cases}$$

where $i = 1, \ldots, |\mathcal{D}_q|$. s_d^i is the trainable attention score associated with placing item d at position i and p_d^i is the probability representing the degree to which the PL module points item d to position i, which is computed by a softmax over the remaining items.

[Jin, et al. CIKM 2023] Jiarui Jin, et al. Replace Scoring with Arrangement: A Contextual Set-to-Arrangement Framework for Learning-to-Rank. CIKM 2023.

Supervisions in the STARank Model

- Our generations should be supervised by the oracle permutation π^*

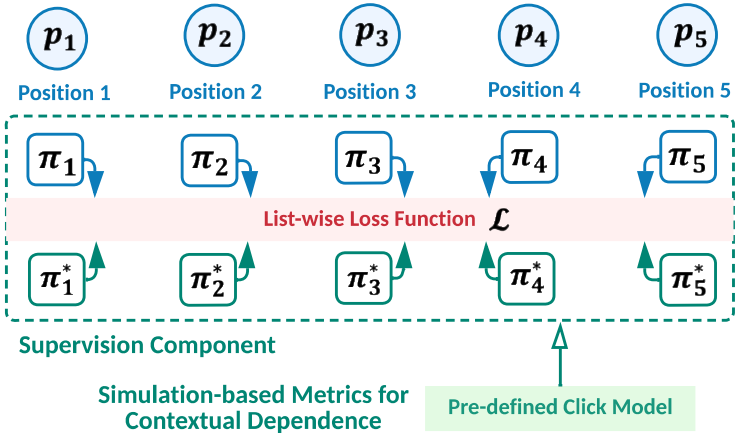
$$\mathcal{L} = \sum_{i=1}^{|\mathcal{D}_q|} \Delta(\pi_i^*|\pi_{<i}, \pi_i|\pi_{<i})$$

where $\Delta(\cdot,\cdot)$ is a point-wise loss function. However, $\pi_i^*|\pi$ is not available, since we only have the ground-truth permutation π_i^*.

Alternatively, we use:

$$\mathcal{L} = \sum_{i=1}^{|\mathcal{D}_q|} \Delta(\pi_i^*|\pi_{<i}^*, \pi_i|\pi_{<i}^*) = \sum_{i=1}^{|\mathcal{D}_q|} \Delta(\pi_i^*|\pi_{i-1}^*, \pi_i|\pi_{i-1}^*)$$

The relative permutation of π_i and π_{i-1}^* is independent of how positions from 1 to $i-2$ are arranged.



LEMMA 3.1. *Given a set of items $\mathcal{D}_q$ and one subset $\mathcal{D}_q' \subseteq \mathcal{D}_q$, the internal permutation of the items in $\mathcal{D}_q'$, denoted as π', is consistent in the context of either $\mathcal{D}_q$ or $\mathcal{D}_q'$, which can be formulated as*

$$P(\pi'|\mathcal{D}_q) = P(\pi'|\mathcal{D}_q'), \quad (14)$$

where $P(\pi'|\mathcal{D}_q)$ and $P(\pi'|\mathcal{D}_q')$ are the probabilities of the items in $\mathcal{D}_q'$ satisfying π' in the context of using the candidate item sets $\mathcal{D}_q$ and $\mathcal{D}_q'$ as inputs respectively.

[Jin, et al. CIKM 2023] Jiarui Jin, et al. Replace Scoring with Arrangement: A Contextual Set-to-Arrangement Framework for Learning-to-Rank. CIKM 2023.

Offline Evaluation of our STARank Model

Ranker	Tmall				Alipay				Taobao			
	N@5	N@10	M@5	M@10	N@5	N@10	M@5	M@10	N@5	N@10	M@5	M@10
FM	0.1233	0.1140	0.2494	0.2572	0.1342	0.1277	0.2634	0.2709	0.1124	0.1106	0.2228	0.2354
DeepFM	0.1241	0.1181	0.2515	0.2591	0.1276	0.1214	0.2551	0.2629	0.1117	0.1104	0.2209	0.2339
PNN	0.1248	0.1185	0.2525	0.2602	0.1296	0.1227	0.2601	0.2666	0.1145	0.1123	0.2266	0.2385
LSTM	0.1341	0.1287	0.2660	0.2716	0.1427	0.1348	0.2783	0.2816	0.1308	0.1284	0.2496	0.2587
GRU	0.1311	0.1255	0.2606	0.2668	0.1422	0.1345	0.2773	0.2810	0.1300	0.1275	0.2488	0.2578
DIN	0.1345	0.1305	0.2674	0.2725	0.1401	0.1365	0.2705	0.2751	0.1230	0.1227	0.2351	0.2457
DIEN	0.1243	0.1182	0.2518	0.2594	0.1351	0.1293	0.2632	0.2696	0.1132	0.1122	0.2245	0.2366
SetRank	0.2733	0.2640	0.4090	0.4284	0.2969	0.2850	0.4289	0.4458	0.2594	0.2498	0.3876	0.4062
Seq2Slate	0.2385	0.2340	0.3720	0.3948	0.2452	0.2412	0.3771	0.3993	0.2223	0.2180	0.3437	0.3690
$\text{STARank}^{-}_{\text{PI}}$	0.3042	0.3251	0.4095	0.4254	0.3214	0.3264	0.4014	0.4446	0.2632	0.2734	0.3974	0.4214
$\text{STARank}^{-}_{\text{PS}}$	0.2952	0.3052	0.3974	0.4035	0.3046	0.3035	0.4046	0.4256	0.2678	0.2742	0.4025	0.4264
STARank	**0.3353***	**0.3745***	**0.4328***	**0.4358***	**0.4509***	**0.4803***	**0.5337***	**0.5703***	**0.3479***	**0.4278***	**0.4082***	**0.4635***
$\textbf{STARank}^{+}_{\text{PBM}}$	**0.3591***	**0.4311***	**0.4472***	**0.4932***	**0.4725***	**0.5046***	**0.5402***	**0.5854***	**0.4006***	**0.4868***	**0.4683***	**0.5250***
$\textbf{STARank}^{+}_{\text{UBM}}$	**0.3482***	**0.4208***	**0.4359***	**0.4828***	**0.4481***	**0.4709***	**0.5189***	**0.5644***	**0.3930***	**0.4792***	**0.4545***	**0.5164***

Ranker	Yahoo			
	N@5	N@10	M@5	M@10
FM	0.2122	0.2434	0.3574	0.3629
DeepFM	0.2483	0.2490	0.3587	0.3662
PNN	0.2563	0.2693	0.3598	0.3602
LSTM	0.2763	0.2731	0.3602	0.3641
GRU	0.2901	0.2876.	0.3674	0.3675
DIN	0.2932	0.2975.	0.3772	0.3706
DIEN	0.2911	0.2983	0.3703	0.3688
SetRank	0.3053	0.3185	0.3932	0.3868
Seq2Slate	0.3121	0.3234	0.4001	0.3987
$\text{STARank}^{-}_{\text{PI}}$	0.3188	0.3210	0.4079	0.4154
$\text{STARank}^{-}_{\text{PS}}$	0.3110	0.3356	0.4139	0.4201
STARank	**0.3399***	**0.3525***	**0.4301***	**0.4398***
$\textbf{STARank}^{+}_{\text{PBM}}$	**0.3474***	**0.3612***	**0.4342***	**0.4450***
$\textbf{STARank}^{+}_{\text{UBM}}$	**0.3355***	**0.3543***	**0.4324***	**0.4463***

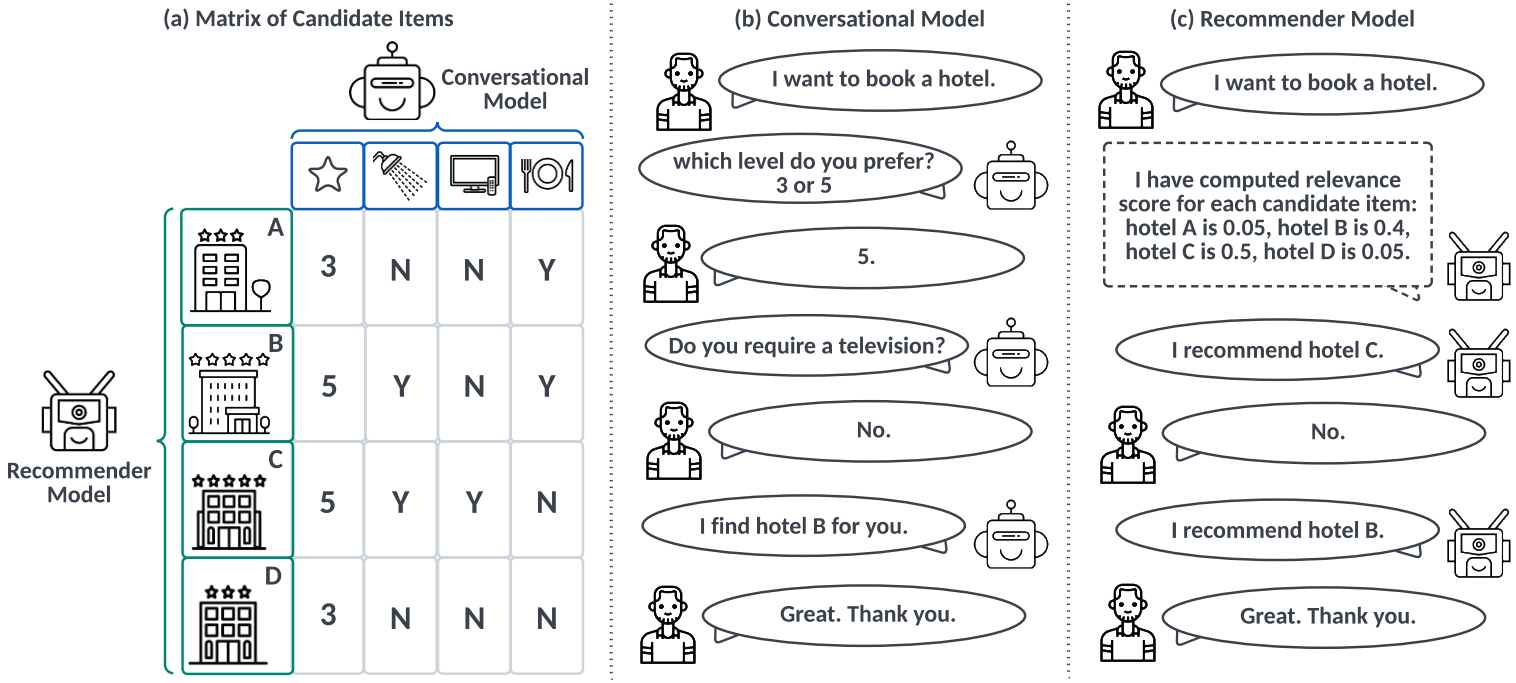
Ranker	LETOR			
	N@5	N@10	M@5	M@10
FM	0.1752	0.1721	0.3282	0.3292
DeepFM	0.1746	0.1689	0.3243	0.3204
PNN	0.1775	0.1739	0.3296	0.3301
LSTM	0.1851	0.1795	0.3405	0.3384
GRU	0.1739	0.1741.	0.3170	0.3205
DIN	0.1550	0.1671.	0.2815	0.2988
DIEN	0.1818	0.1811	0.3409	0.3402
SetRank	0.2035	0.2246	0.4045	0.4002
Seq2Slate	0.2436	0.2546	0.4122	0.4031
$\text{STARank}^{-}_{\text{PI}}$	0.2443	0.2568	0.4172	0.4075
$\text{STARank}^{-}_{\text{PS}}$	0.2467	0.2606	0.4231	0.4123
STARank	**0.2824***	**0.2622***	**0.4876***	**0.4529***
$\textbf{STARank}^{+}_{\text{PBM}}$	**0.2843***	**0.2646***	**0.4906***	**0.4573***
$\textbf{STARank}^{+}_{\text{UBM}}$	**0.2863***	**0.2668***	**0.4887***	**0.4587***

- $\text{STARank}^{-}_{\text{PI}}$ is a variant of STARank using the MLP layer as PI module to encode the input candidate items instead of our attention network.
- $\text{STARank}^{-}_{\text{PS}}$ is a variant of STARank using the MLP layer as PS module to encode the sequence data of historical records instead of our LSTM.
- $\text{STARank}^{+}_{\text{PBM}}$ and $\text{STARank}^{+}_{\text{UBM}}$ are variants of STARank using different click models (i.e., PBM and UBM) to generate the oracle permutation π^*.

[Jin, et al. CIKM 2023] Jiarui Jin, et al. Replace Scoring with Arrangement: A Contextual Set-to-Arrangement Framework for Learning-to-Rank. CIKM 2023.

Conversational Recommender Systems

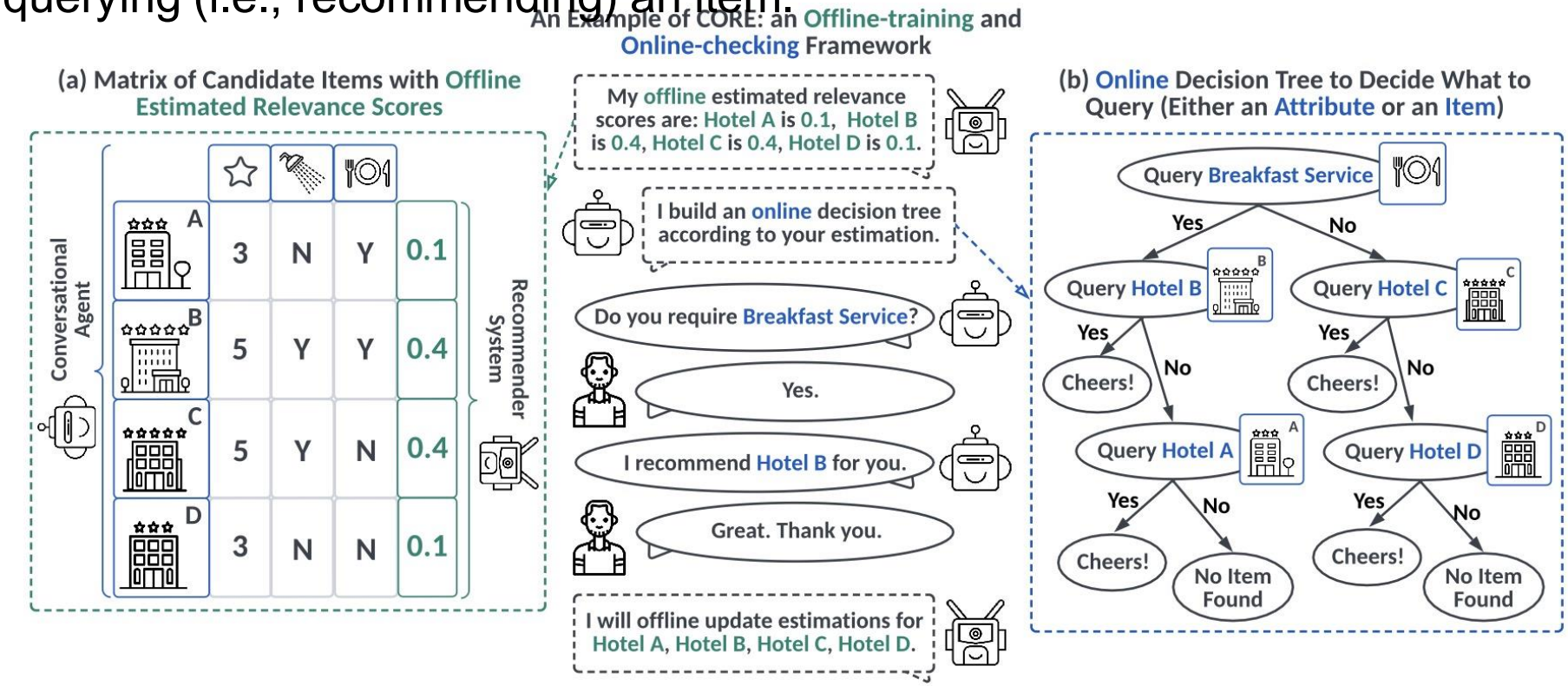
- Our main idea is to bridge the conversational model's ability of online querying user preference on each attribute and the recommendation model's ability of offline estimating user interest with the collected data for each item, allowing the whole system to either query an attribute (or attribute value) or query (i.e., recommend) an item to users.



[Jin, et al. NeurIPS 2023] Jiarui Jin, et al. Lending Interaction Wings to Recommender Systems with Conversational Agents. NeurIPS 2023.

Overview of our CORE Model

- Bridging recommendation models and conversational models through an offline-training and online-checking framework only requires recommendation APIs (i.e., estimated relevance scores) and conversation APIs.
- The key technique is how to jointly measure the gain of querying an attribute (and querying an attribute value) and querying (i.e., recommending) an item.



[Jin, et al. NeurIPS 2023] Jiarui Jin, et al. Lending Interaction Wings to Recommender Systems with Conversational Agents. NeurIPS 2023.

Online Decision Tree in our CORE Model

- Uncertainty measures how many estimated relevance scores are still unchecked, i.e.,

$$\mathrm{U}_k \coloneqq \mathrm{SUM}(\{\Psi_{\mathrm{RE}}(v_m) | v_m \in \mathcal{V}_k\})$$

where $\Psi_{\mathrm{RE}}(v_m)$ outputs the estimated relevance score for item v_m. $\mathcal{V}_k$ is the set of all the unchecked items after k interactions, and $\mathcal{V}_k$ is initialized as all candidate items $\mathcal{V}_0 = \mathcal{V}$.

- Certainty Gain of the k-th interaction is defined as $\Delta\mathrm{U}_k \coloneqq \mathrm{U}_{k-1} - \mathrm{U}_k$, i.e., how many relevance scores are checked at the k-th interaction.
- Since our goal is to find a target item, if our conversational agent successfully finds one at the k-th turn, then we set all the items checked, namely $\mathrm{U}_k = 0$. Then, our objective can be expressed as:

$$\min_{\Psi^*_{\mathrm{RE}}} K \,, \mathrm{s.t.}\,, \mathrm{U}_K = 0$$

where K is the number of turns, and Ψ^*_{RE} means that the recommender system is frozen. To this end, the design of our conversational agent can be organized as an uncertainty minimization problem.

[Jin, et al. NeurIPS 2023] Jiarui Jin, et al. Lending Interaction Wings to Recommender Systems with Conversational Agents. NeurIPS 2023.

Online Decision Tree in our CORE Model

- The resulting online decision tree algorithm is to greedily choose an attribute (or an attribute value) or an item that can maximize the Certainty Gain:

$$a_{\text{query}} = \underset{a \in \mathcal{V}_{k-1} \cup \mathcal{X}_{k-1}}{\operatorname{argmin}} \Psi_{\text{CG}}\big(\text{query}(a)\big)$$

- $\Psi_{\text{CG}}(\cdot)$ denotes the Expected Certainty Gain of querying a, and a can be either an unchecked item (from $\mathcal{V}_{k-1}$) or an unchecked attribute (from $\mathcal{X}_{k-1}$).
- $\Psi_{\text{CG}}(\cdot)$ for Query an Item. Let $\mathcal{V}^*$ denote the set of all the target items in the session. Since we only need to find one target item, therefore, if $a \in \mathcal{V}_{k-1}$, we can derive:

$$\begin{aligned}\Psi_{\text{CG}}\big(\text{query}(a)\big) &= \Psi_{\text{CG}}(a \in \mathcal{V}^*) \cdot P(a \in \mathcal{V}^*) + \Psi_{\text{CG}}(a \notin \mathcal{V}^*) \cdot P(a \notin \mathcal{V}^*) \\ &= \left(\sum_{v_m \in \mathcal{V}_{k-1}} \Psi_{\text{RE}}(v_m) \right) \cdot P(a \in \mathcal{V}^*) + \Psi_{\text{RE}}(a) \cdot P(a \notin \mathcal{V}^*)\end{aligned}$$

[Jin, et al. NeurIPS 2023] Jiarui Jin, et al. Lending Interaction Wings to Recommender Systems with Conversational Agents. NeurIPS 2023.

Online Decision Tree in our CORE Model

- $\Psi_{\mathrm{CG}}(\cdot)$ for Query an Attribute. For each queried attribute a, let $\mathcal{W}_a$ denote the set of all the candidate attribute values, and let $w_a^* \in \mathcal{W}_a$ denote the user preference on a. If $a \in \mathcal{X}_{k-1}$, we have:

$$\Psi_{\mathrm{CG}}\big(\mathrm{query}(a)\big) = \sum_{w_a \in \mathcal{W}_a} \big(\Psi_{\mathrm{CG}}(w_a = w_a^*) \cdot P(w_a = w_a^*)\big)$$

where $w_a = w_a^*$ means that when querying a, the user's answer (denoted as w_a^*) is w_a.

- Let $\mathcal{V}_{a_{\mathrm{value}}=w_a}$ denote the set of all the items whose value of a is equal to w_a, and let $\mathcal{V}_{a_{\mathrm{value}}\neq w_a}$ denote the set of rest items. $\Psi_{\mathrm{CG}}(w_a = w_a^*)$ can be computed as:

$$\Psi_{\mathrm{CG}}(w_a = w_a^*) = \mathrm{SUM}\big(\{\Psi_{\mathrm{RE}}(v_m) \big| v_m \in \mathcal{V}_{k-1} \cap \mathcal{V}_{a_{\mathrm{value}}\neq w_a}\}\big)$$

which indicates that the certainty gain, when w_a is the user's answer, is the summation of relevance scores of those items not matching the user preference.

[Jin, et al. NeurIPS 2023] Jiarui Jin, et al. Lending Interaction Wings to Recommender Systems with Conversational Agents. NeurIPS 2023.

Online Decision Tree in our CORE Model

- $\Psi_{\mathrm{CG}}(\cdot)$ for Query an Attribute Value. Querying attributes may face the following challenges: (a) a user would not have a clear picture of an attribute; (b) a user's answer would be different from all the candidate attribute values; (c) compared to querying attribute values that leads to closed questions (i.e., a user only needs to answer Yes or No), the user's answer to querying attributes may be vague.
- For this case, we need to choose a value $w_x \in \mathcal{W}_x$ where $x \in \mathcal{X}_{k-1}$, and we compute the expected certainty gain of querying attribute value w_x as:

$$\Psi_{\mathrm{CG}}(\mathrm{query}(x) = w_x) = \Psi_{\mathrm{CG}}(w_x = w_x^*) \cdot P(w_x = w_x^*) + \Psi_{\mathrm{CG}}(w_x \neq w_x^*) \cdot P(w_x \neq w_x^*)$$

where $w_x^* \in \mathcal{W}_x$ denotes the user preference on attribute x. Different from querying attributes, we only need to estimate the certainty gain for the user responding with Yes (i.e., $w_x = w_x^*$) and No (i.e., $w_x \neq w_x^*$).

[Jin, et al. NeurIPS 2023] Jiarui Jin, et al. Lending Interaction Wings to Recommender Systems with Conversational Agents. NeurIPS 2023.

Online Decision Tree in our CORE Model

- Making $\Psi_{\mathrm{CG}}(\cdot)$ Consider Dependence among Attributes. For this purpose, the expected certainty gain can expressed as:

$$\Psi_{\mathrm{CG}}^{\mathrm{D}}\big(\mathrm{query}(a)\big) = \sum_{a' \in \mathcal{X}_{k-1}} \big(\Psi_{\mathrm{CG}}(\mathrm{query}(a')) \cdot P(\mathrm{query}(a')|\mathrm{query}(a))\big)$$

where $P(\mathrm{query}(a')|\mathrm{query}(a))$ measures the probability of the user preference on a determining the user preference on a'. Then we can derive:

$$\Psi_{\mathrm{CG}}^{\mathrm{D}}\big(\mathrm{query}(a)\big) = \sum_{w_a \in \mathcal{W}_a} \big(\Psi_{\mathrm{CG}}^{\mathrm{D}}(w_a = w_a^*) \cdot P(w_a = w_a^*)\big)$$

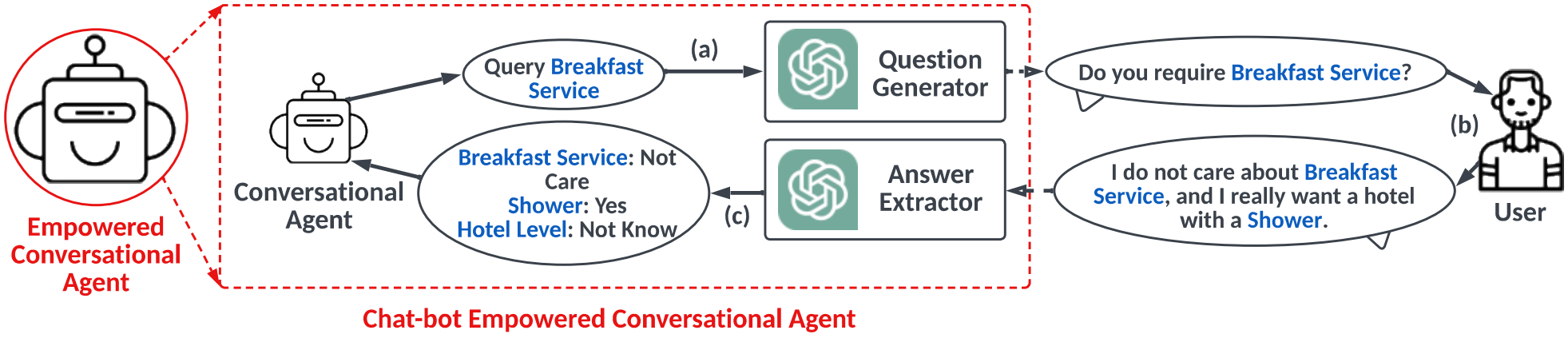
$$\Psi_{\mathrm{CG}}^{\mathrm{D}}(w_a = w_a^*) = \sum_{a' \in \mathcal{A}} \sum_{w_{a'} \in \mathcal{W}_{a'}} \big(\Psi_{\mathrm{CG}}^{\mathrm{D}}(w_{a'} = w_{a'}^*) \cdot P(w_{a'} = w_{a'}^* | w_a = w_a^*)\big)$$

where$(w_{a'} = w_{a'}^* | w_a = w_a^*)$ measures the probability of how likely getting the user preference on attribute a determines the user's preference on other attribute a'.

[Jin, et al. NeurIPS 2023] Jiarui Jin, et al. Lending Interaction Wings to Recommender Systems with Conversational Agents. NeurIPS 2023.

Plugging LLM into Conversational Agent

- We plug a Large Language Model (LLM) API to form a question generator and an answer extractor by prompting. Here, we apply LLM because (a) LLM is a powerful language model for understanding and generating conversations; (b) LLM can encode some latent features (e.g., from “a user is a student” to infer “a user may have low budget”) from conversations to guide the decision-making of our conversational agent.



[Jin, et al. NeurIPS 2023] Jiarui Jin, et al. Lending Interaction Wings to Recommender Systems with Conversational Agents. NeurIPS 2023.

Offline Evaluation of our CORE Model

- Our evaluation has two separate configurations in terms of query attributes. One is directly querying attribute IDs (e.g., querying a user what level of hotel she prefers), while the other one is querying attribute values (e.g., querying a user whether she prefers hotel with level 3).

Algorithm 1 CORE for Querying Items and Attributes

Input: A recommender system $\Psi_{\text{RE}}(\cdot)$, an item set $\mathcal{V}$, an attribute set $\mathcal{X}$, an offline dataset $\mathcal{D}$.
Output: Updated recommender system $\Psi_{\text{RE}}(\cdot)$, up-to-date dataset $\mathcal{D}$.

Train $\Psi_{\text{RE}}(\cdot)$ on $\mathcal{D}$. ▷ Offline-Training
for each session (i.e., the given user) **do**
  Initialize $k = 1$ and $\mathcal{V}_0 = \mathcal{V}$, $\mathcal{X}_0 = \mathcal{X}$.
  repeat
    Compute a_{query} following Eq. (3) for querying items and attributes or following Eq. (10) for querying items and attribute values. ▷ Online-Checking
    Query a_{query} to the user and receive the answer. ▷ Online-Checking
    Generate $\mathcal{V}_k$ and $\mathcal{X}_k$ from $\mathcal{V}_{k-1}$ and $\mathcal{X}_{k-1}$. ▷ Online-Checking
    Go to next turn: $k \leftarrow k + 1$.
  until Querying a target item or $k > K_{\text{MAX}}$ where K_{MAX} is the maximal number of turns.
  Collect session data and add to $\mathcal{D}$.
end for
Update $\Psi_{\text{RE}}(\cdot)$ using data in $\mathcal{D}$. ▷ Offline-Training

- $\text{T@}K_{\text{MAX}}$ is the average turns needed to end the sessions (where for each session, if the model successfully queries a target item within K_{MAX} turns, then return the success turn; otherwise, we enforce the model to query an item at the $(K_{\text{MAX}}+1)$-th turn, if succeeds, return $K_{\text{MAX}} + 1$; otherwise return $K_{\text{MAX}} + 3$.
- $S@K_{\text{MAX}}$ is the average success rate.
- $\text{CORE}_{\text{D}}^{+}$ is a variant of CORE computing $\Psi_{\text{CG}}^{\text{D}}(\cdot)$s instead of $\Psi_{\text{CG}}(\cdot)$s.

[Jin, et al. NeurIPS 2023] Jiarui Jin, et al. Lending Interaction Wings to Recommender Systems with Conversational Agents. NeurIPS 2023.

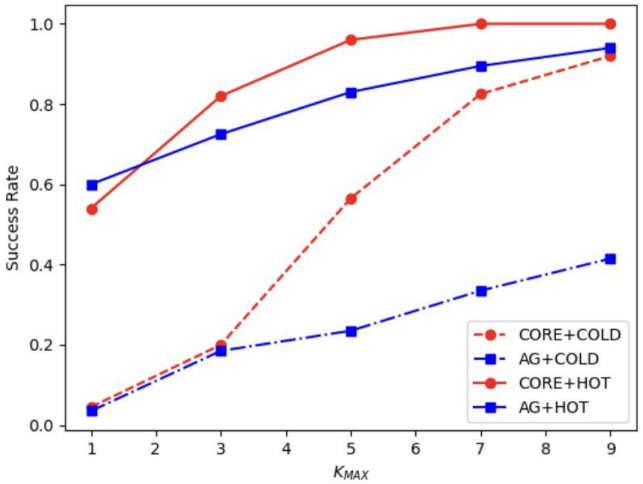
Offline Evaluation of our CORE Model

Results of querying attribute values.

$\Psi_{\text{RE}}(\cdot)$	$\Psi_{\text{CO}}(\cdot)$	Amazon				LastFM				Yelp			
		T@3	S@3	T@5	S@5	T@3	S@3	T@5	S@5	T@3	S@3	T@5	S@5
COLD START	AG	6.47	0.12	7.83	0.23	6.77	0.05	8.32	0.14	6.65	0.08	8.29	0.13
	ME	6.50	0.12	8.34	0.16	6.84	0.04	8.56	0.11	6.40	0.15	8.18	0.20
	CORE	**6.02**	**0.25**	**6.18**	**0.65**	**5.84**	**0.29**	**5.72**	**0.74**	**5.25**	**0.19**	**6.23**	**0.65**
	$\text{CORE}_{\text{D}}^{+}$	6.00	0.26	6.01	0.67	5.79	0.30	5.70	0.75	5.02	0.21	6.12	0.68
FM	AG	**2.76**	0.74	**2.97**	0.83	4.14	0.52	4.67	0.64	3.29	0.70	3.39	0.81
	CRM	4.58	0.28	6.42	0.38	4.23	0.34	5.87	0.63	4.12	0.25	6.01	0.69
	EAR	4.13	0.32	6.32	0.42	4.02	0.38	5.45	0.67	4.10	0.28	5.95	0.72
	CORE	3.26	**0.83**	3.19	**0.99**	**3.79**	**0.72**	**3.50**	**0.99**	**3.14**	**0.84**	**3.20**	**0.99**
	$\text{CORE}_{\text{D}}^{+}$	3.16	0.85	3.22	1.00	3.75	0.74	3.53	1.00	3.10	0.85	3.23	1.00
DEEP FM	AG	**3.07**	0.71	3.27	0.82	3.50	0.68	3.84	0.79	3.09	0.74	3.11	0.88
	CRM	4.51	0.29	6.32	0.40	4.18	0.38	5.88	0.63	4.11	0.23	6.02	0.71
	EAR	4.47	0.30	6.35	0.43	4.01	0.37	5.43	0.69	4.01	0.32	5.74	0.75
	CORE	3.23	**0.85**	**3.22**	**0.99**	**3.47**	**0.81**	**3.34**	**1.00**	**2.98**	**0.93**	**3.11**	**1.00**
PNN	AG	3.02	0.74	3.10	0.87	3.44	0.67	3.53	0.84	2.83	0.77	2.82	0.91
	CORE	**3.01**	**0.88**	**3.04**	**0.99**	**3.10**	**0.87**	**3.20**	**0.99**	**2.75**	**0.88**	**2.76**	**1.00**

Results of querying attribute IDs.

$\Psi_{\text{RE}}(\cdot)$	$\Psi_{\text{CO}}(\cdot)$	Amazon			
		T@3	S@3	T@5	S@5
COLD START	ME	3.04	0.98	5.00	1.00
	CORE	**2.88**	**1.00**	**2.87**	**1.00**
	$\text{CORE}_{\text{D}}^{+}$	2.84	1.00	2.86	1.00
FM	AG	2.76	0.74	2.97	0.83
	CRM	3.07	0.98	3.37	1.00
	EAR	2.98	0.99	3.13	1.00
	CORE	**2.17**	**1.00**	**2.16**	**1.00**
	$\text{CORE}_{\text{D}}^{+}$	2.14	1.00	2.14	1.00



Comparisons between CORE and Abs Greedy (AG) that always queries an item with the highest relevance score at each turn, in both cold- and hot-start setting.

[Jin, et al. NeurIPS 2023] Jiarui Jin, et al. Lending Interaction Wings to Recommender Systems with Conversational Agents. NeurIPS 2023.

Open-Sourced System of our CORE Model

https://github.com/CORE-Labet/CORE

Definition A1 (Conversational Agent and Human User Interactions). *Our conversational agent is designed to act in the following four ways.*

(i) Query an item $v \in \mathcal{V}_{k-1}$, where $\mathcal{V}_{k-1}$ is the set of unchecked items after $k-1$ interactions (e.g., recommend `Hotel A` *to the user).*

(ii) Query an attribute $x \in \mathcal{X}_{k-1}$, where $\mathcal{X}_{k-1}$ is the set of unchecked attributes after $k-1$ interactions (e.g., query what `Hotel Level` *does the user want).*

(iii) Query whether the user's preference on an attribute $x \in \mathcal{X}_{k-1}$ is equal to a specific attribute value $w_x \in \mathcal{W}_x$ where $\mathcal{W}_x$ is the set of values of attribute x (e.g., query whether the user likes a hotel with `Hotel Level`*=5).*

(iv) Query whether the user's preference on an attribute is not smaller than a specific value $w_x \in \mathbb{R}$ (e.g., query whether the user likes a hotel with `Hotel Level`*$\geq$3.5).*

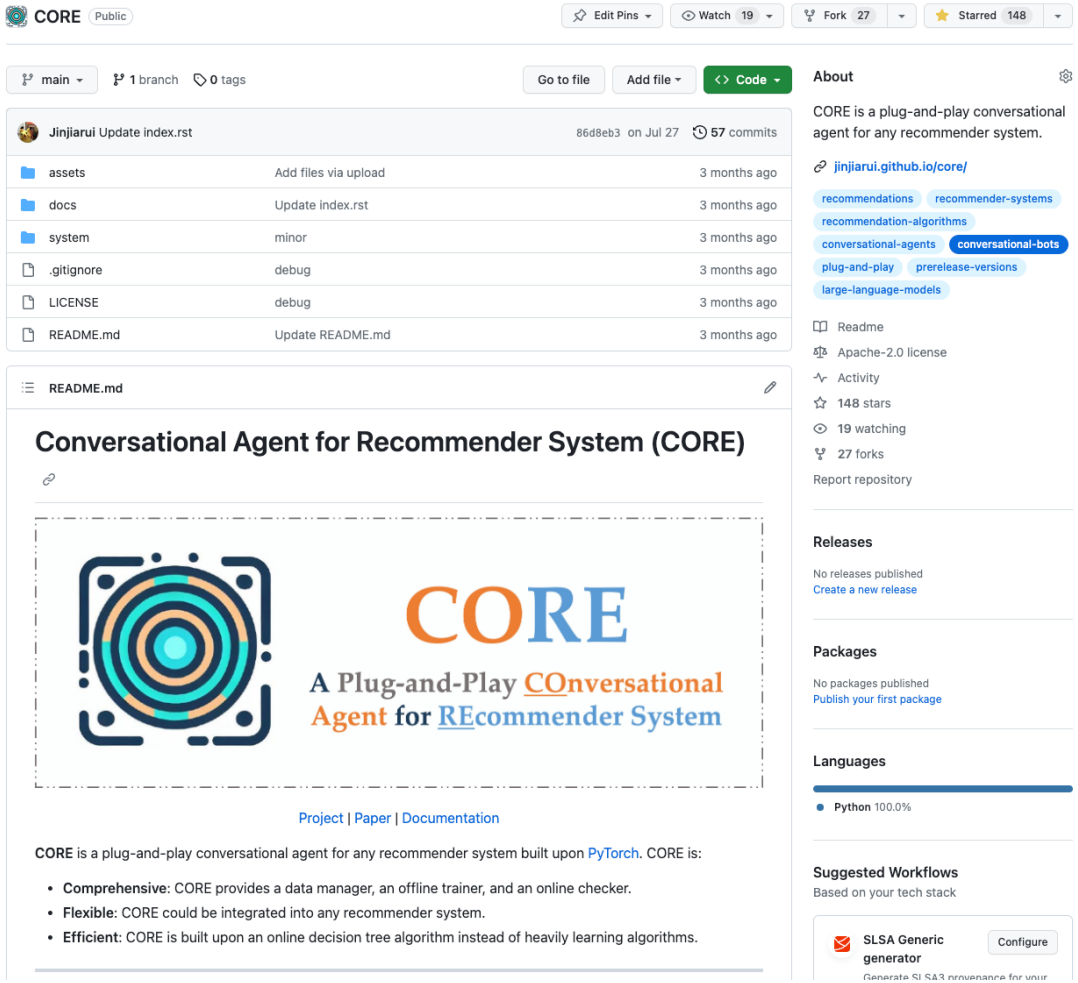
The human user is supposed to respond in the following ways.

(i) For queried item v, the user should answer `Yes` *(if v satisfies the user) or* `No` *(otherwise) (e.g., answer* `Yes`*, if the user likes* `Hotel A`*).*

(ii) For queried attribute x, the user should answer her preferred attribute value, denoted as $w_x^* \in \mathcal{W}_x$ (e.g., answer 3 for queried attribute `Hotel Level`*), or answer* `Not Care` *to represent that any attribute value works.*

(iii) For queried attribute value w_x, the user should answer `Yes` *(if w_x matches the user preference) or* `No` *(otherwise) (e.g., answer* `Yes`*, if the user wants a hotel with* `Hotel Level`*=5), or answer* `Not Care` *to represent that any attribute value works.*

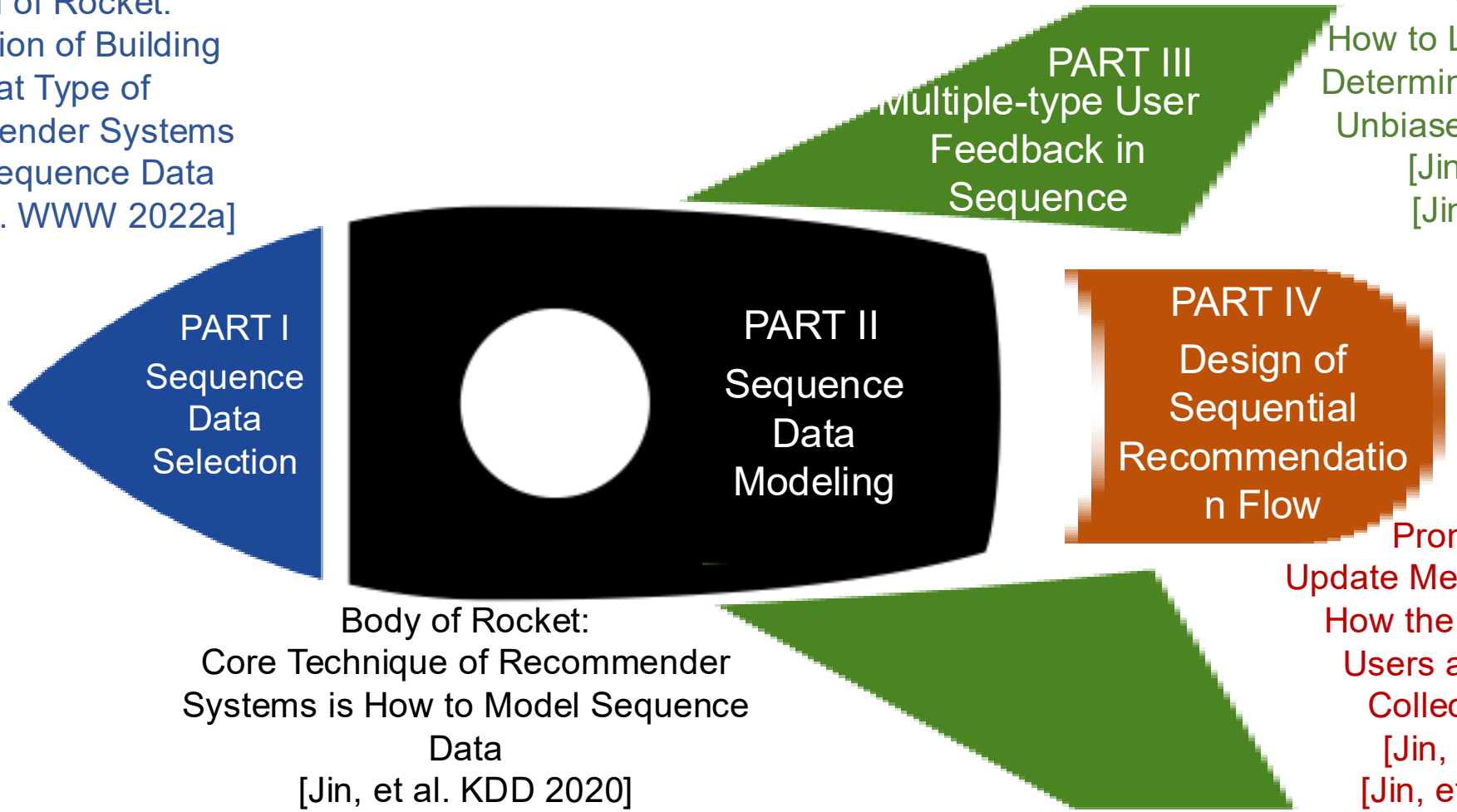
(iv) For queried attribute value w_x, the user should answer `Yes` *(if the user wants an item whose value of attribute x is not smaller than w_x) or* `No` *(otherwise) (e.g., answer* `Yes`*, if the user wants a hotel with* `Hotel Level`*=5), or answer* `Not Care` *to represent that any attribute value works.*



[Jin, et al. NeurIPS 2023] Jiarui Jin, et al. Lending Interaction Wings to Recommender Systems with Conversational Agents. NeurIPS 2023.

Content

Head of Rocket:
Orientation of Building What Type of Recommender Systems using Sequence Data
[Jin, et al. WWW 2022a]



Wings of Rocket:
How to Leverage User Feedback Determine Whether the System is Unbiased and Aligned with Goal
[Jin, et al. SIGIR 2020]
[Jin, et al. CIKM 2022]

Body of Rocket:
Core Technique of Recommender Systems is How to Model Sequence Data
[Jin, et al. KDD 2020]
[Jin, et al. TOIS 2022]
[Jin, et al. WWW 2022b]

Promotion of Rocket:
Update Mechanism is caring about How the System Responds to Users and How the System Collects Sequence Data
[Jin, et al. CIKM 2023]
[Jin, et al. NeurIPS 2023]

PART V: Conclusions & Take-aways

How Can We Benefit From These Methods?

May We Sell SANDSTONE to You

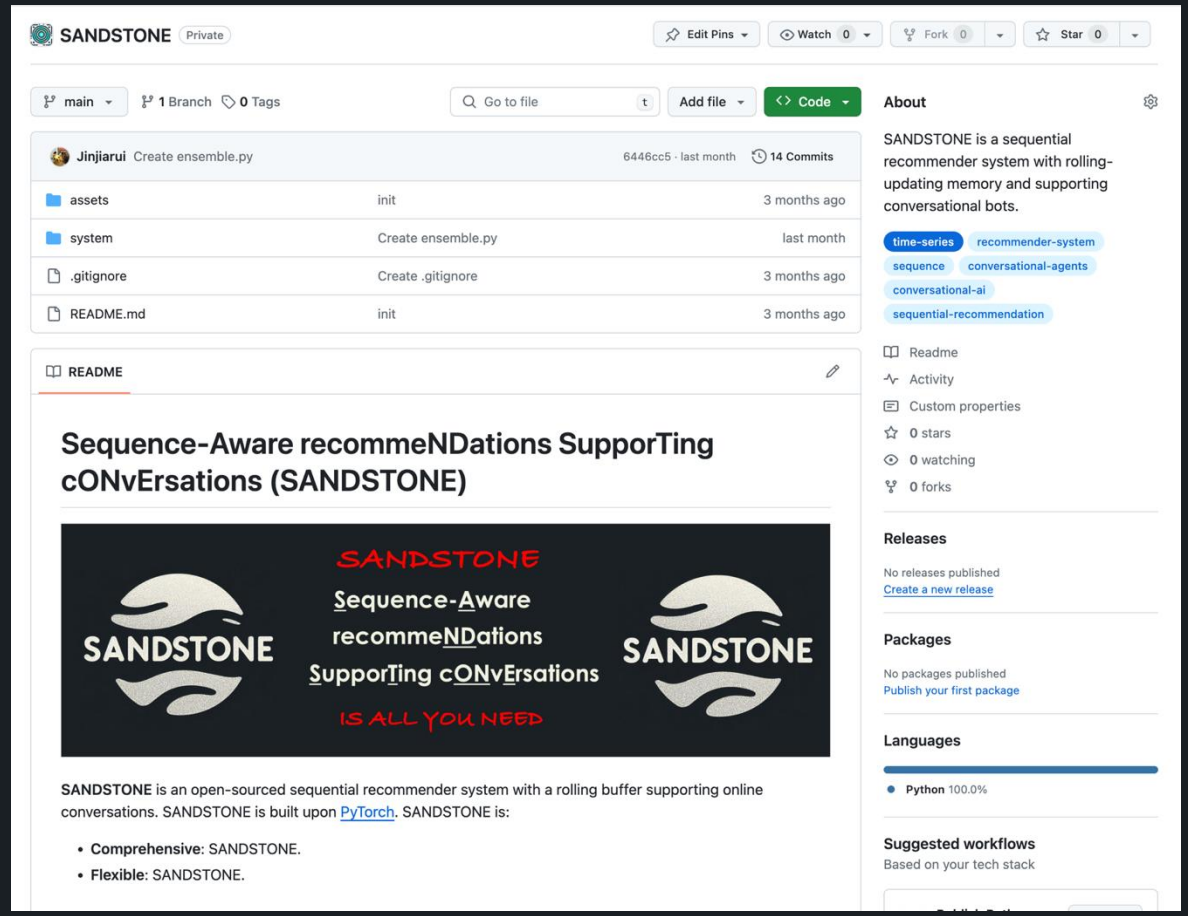
(a) Defining “Relevant” Data with Category and Time in Data Selection.

(b) Formulating “Interaction” Operations with Fast Fourier Transform in Architecture Design.

(c) Discovering “Relationship” over User Feedback (i.e., Task Objective) with Survival Model in Learning Objective.

(d) Plugging “Conversations” with Offline-Training and Online-Checking in System Response.

https://github.com/CORE-Labet/SANDSTONE



SHANGHAI JIAO TONG UNIVERSITY

APEX 数据和知识管理实验室
DATA & KNOWLEDGE MANAGEMENT LAB

UCL

Thanks for Your Listening

Jiarui Jin

Ph.D. Student, Shanghai Jiao Tong University

https://jinjiarui.github.io/

alimama

aws

招商银行
CHINA MERCHANTS BANK

淘宝直播
TAOBAO LIVE